

Automation Testing

SELENIUM

Agenda

Module-1 [Introduction to Automation Testing]

Module -2 [Core Java]

Module-3 [Selenium Web Driver]

Module-4 [Automation Testing Framework]

Module -5 [Maven Project Management Tool]

Module-1 [Introduction to Automation Testing]

- Introduction to Automation Testing
- Selenium
- QTP V/S Selenium

Automation Testing

Introduction

- Test automation is the use of software to control the execution of tests, the comparison of actual outcomes to predicted outcomes, the setting up of test preconditions, and other test control and test reporting functions.
- Commonly, test automation involves automating a manual process already in place that uses a formalized testing process.
- Although manual tests may find many defects in a software application, it is a laborious and time consuming process.
- In addition it may not be effective in finding certain classes of defects.
- Test automation is a process of writing a computer program to do testing that would otherwise need to be done manually.
- Once tests have been automated, they can be run quickly.
- This is often the most cost effective method for software products that have a long maintenance life, because even minor patches over the lifetime of the application can cause features to break which were working at an earlier point in time.

Automation Testing

Why Automation?

- **Speed**—Automation Scripts run very fast when compared to human users
- **Reliable**—Tests perform precisely the same operations each time they are run, there by eliminating human error.
- **Repeatable**—We can test how the application reacts after repeated execution of the same operation
- **Comprehensive**—We can build a suite of tests that covers every feature in our application
- **Reusable**—We can reuse tests on different versions of an application, even if the user interface changes.
- Automate your testing procedure when you have lot of regression work.
- Automate your load testing work for creating virtual users to check load capacity of your application.

Difference between Manual and Automation Testing

Manual Testing

- Manual testing is not accurate at all times due to human error, hence it is less reliable.
- Manual testing is time-consuming, taking up human resources.
- Investment is required for human resources.
- Manual testing is only practical when the test cases are run once or twice, and frequent repetition is not required.
- Manual testing allows for human observation, which may be more useful if the goal is user-friendliness or improved customer experience.

Automated Testing

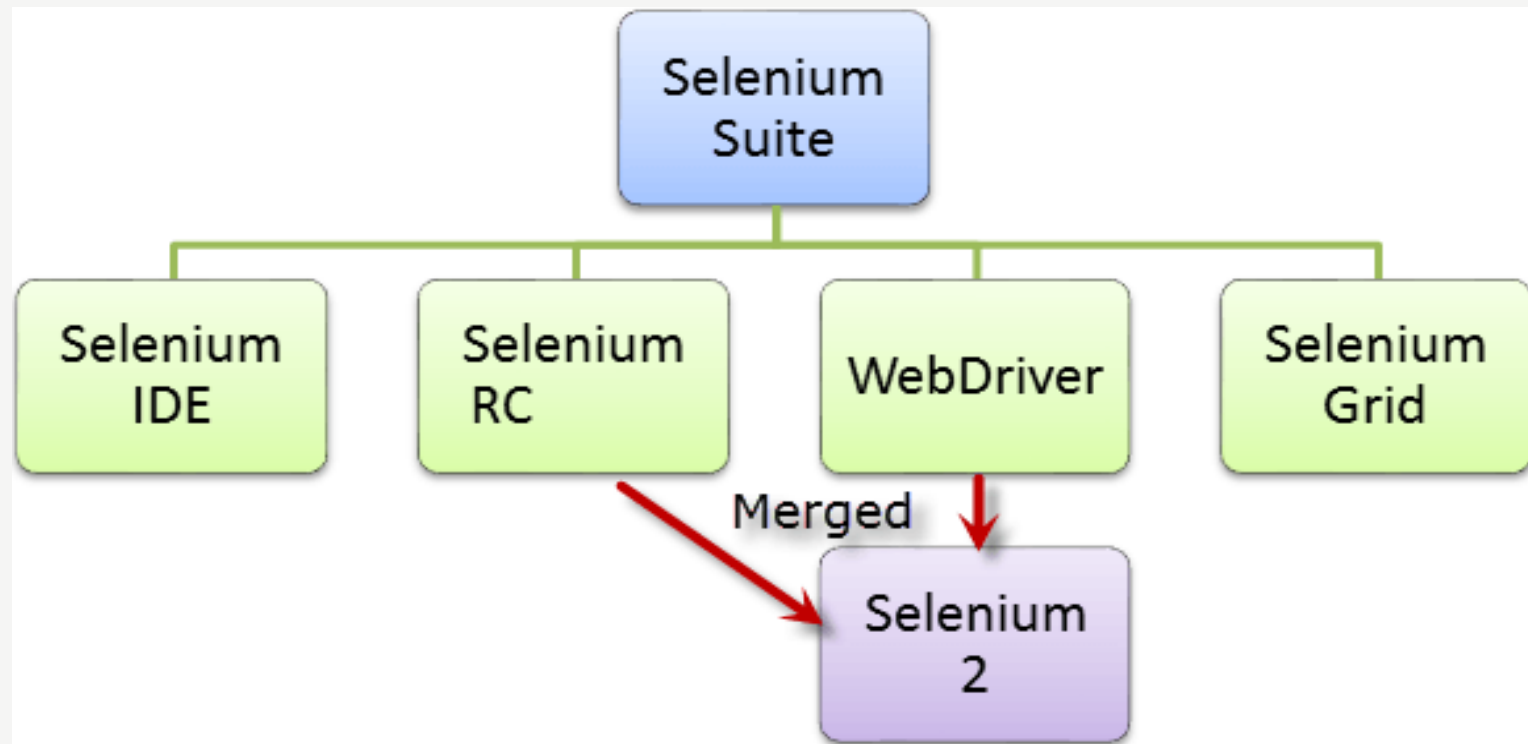
- Automated testing is more reliable, as it is performed by tools and/or scripts.
- Automated testing is executed by software tools, so it is significantly faster than a manual approach.
- Investment is required for testing tools.
- Automated testing is a practical option when the test cases are run repeatedly over a long time period.
- Automated testing does not entail human observation and cannot guarantee user-friendliness or positive customer experience.

Introduction to Selenium

- Selenium is a free (open source) automated testing suite for web applications across different browsers and platforms.
- It is quite similar to HP Quick Test Pro (QTP) only that Selenium focuses on automating web-based applications.

Introduction to Selenium

Selenium is not just a single tool but a suite of software's, each catering to different testing needs of an organisation. It has four components.



Selenium Components

Introduction Selenium IDE

- Selenium Integrated Development Environment (IDE) is the simplest framework in the Selenium suite and is the easiest one to learn.
- It is a Firefox plugin that you can install as easily as you can with other plugins.
- However, because of its simplicity, Selenium IDE should only be used as a prototyping tool.
- If you want to create more advanced test cases, you will need to use either Selenium RC or WebDriver.

Selenium (Pros and Cons)



<u>PROS</u>	<u>CONS</u>
Very easy to use and install.	Available only in Firefox.
No programming experience is required, though knowledge of HTML and DOM are needed.	Designed only to create prototypes of tests.
Can export tests to formats usable in Selenium RC and WebDriver.	No support for iteration and conditional operations.
Has built-in help and test results reporting module.	Test execution is slow compared to that of Selenium RC and WebDriver.
Provides support for extensions.	

Introduction Selenium Remote Control (Selenium RC)

Selenium RC was the **flagship testing framework** of the whole Selenium project for a long time. This is the first automated web testing tool that **allowed users to use a programming language they prefer**. As of version 2.25.0, RC can support the following programming languages:

Java

C#

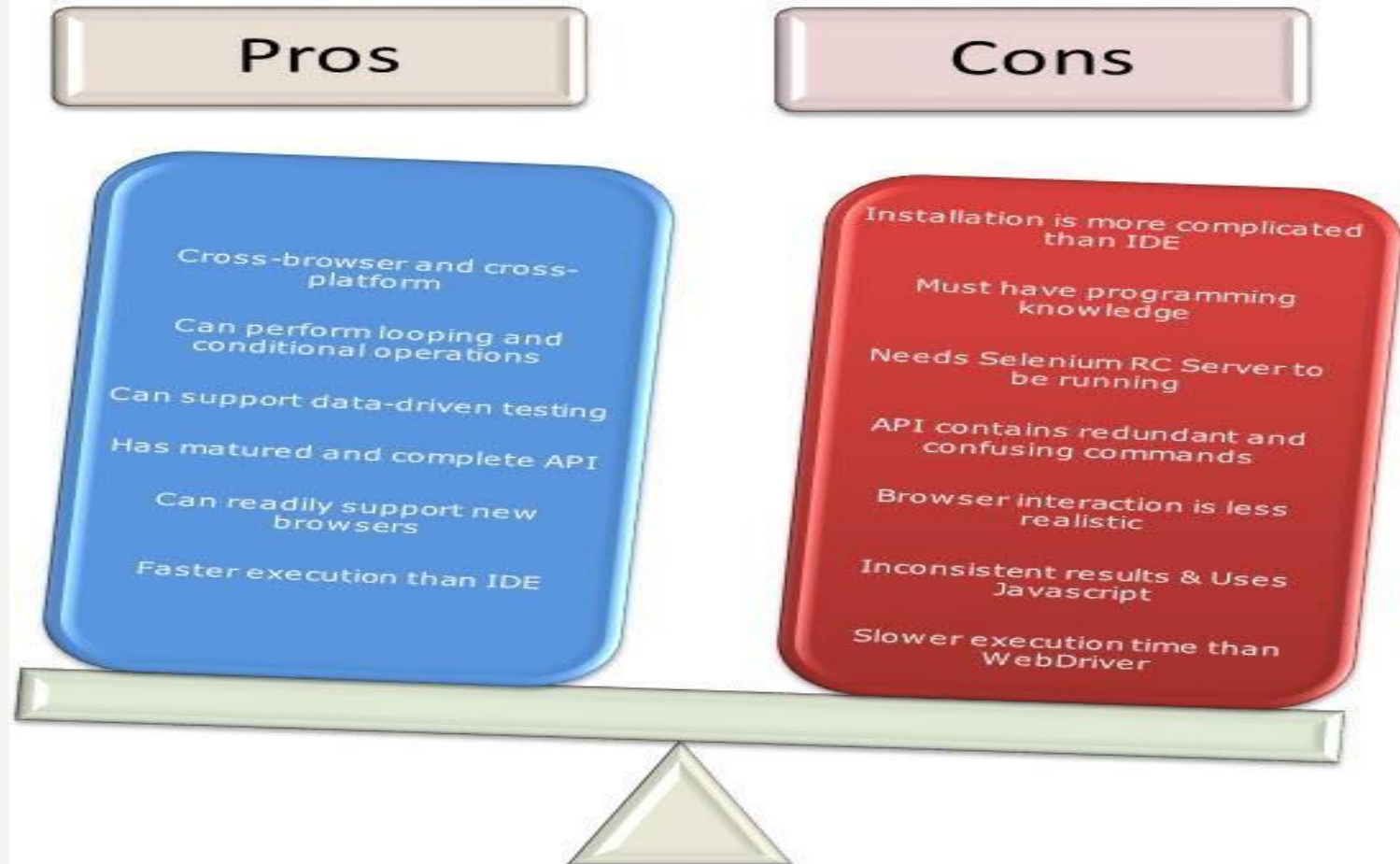
PHP

Python

Perl

Ruby

Selenium RC(Pros and Cons)



Introduction WebDriver

The WebDriver proves itself to be **better than both Selenium IDE and Selenium RC** in many aspects. It implements a more modern and stable approach in automating the browser's actions. WebDriver, unlike Selenium RC, does not rely on JavaScript for automation. **It controls the browser by directly communicating to it.**

The supported languages are the same as those in Selenium RC.

Java

C#

PHP

Python

Perl

Ruby

WebDriver(Pros and Cons)

Pros	Cons
Simpler installation than Selenium RC	Installation is more complicated than Selenium IDE
Communicates directly to the browser	Requires programming knowledge
Browser interaction is more realistic	Cannot readily support new browsers
No need for a separate component such as the RC Server	Has no built-in mechanism for logging runtime messages and generating test results
Faster execution time than IDE and RC	

Selenium Grid

- Selenium Grid is a tool **used together with Selenium RC to run parallel tests** across different machines and different browsers all at the same time. Parallel execution means running multiple tests at once.

Features:

- Enables **simultaneous running of tests** in **multiple browsers and environments**.
- **Saves time** enormously.
- Utilizes the **hub-and-nodes** concept. The hub acts as a central source of Selenium commands to each node connected to it.

Browser and Environment Support

	Selenium IDE	Selenium RC	WebDriver
Browser Support	Mozilla Firefox	Mozilla Firefox Internet Explorer Google Chrome Safari Opera Konqueror Others	Internet Explorer versions 6 to 9, both 32 and 64-bit Firefox 3.0, 3.5, 3.6, 4.0, 5.0, 6, 7 and above (current version is 16.0.1)

Browser and Environment Support

Selenium IDE	Selenium RC	WebDriver
		Google Chrome 12.0.712.0 and above (current version is 22.0.1229.94 m) Opera 11.5 and above (current version is 12.02) Android - 2.3 and above for phones and tablets (devices & emulators) iOS 3+ for phones (devices & emulators) and 3.2+ for tablets (devices & emulators) HtmlUnit 2.9 and above(current version is 2.10)

Browser and Environment Support

	Selenium IDE	Selenium RC	WebDriver
Operating System	Windows Mac OS X Linux	Windows Mac OS X Linux Solaris	All operating systems where the browsers above can run.

Selenium Features

Open source	- Works on multiple browsers and multiple operating systems as compared to other tools in market.
Supports multiple platform and browser	- You can develop selenium code and make it run parallelly on multiple machines using different browsers.
Mobile testing	- Support for Android and Iphone Testing.
Simple	- Selenium IDE is a simple tool which comes as an addon in firefox and is easy to use. It has the record and run feature which is very strong.

Selenium Features

	- You can also extend the functionality/scope of IDE with the help of many plugins available
	- You can also create your own Selenium IDE plugins
	Webdriver is the latest version of selenium and is very strong. Its removed lots of drawbacks in RC and introduced many more features in selenium. - Selenium when used with Hudson can be used for Continuous integration
	- Object oriented datadriven or hybrid testing framework can be made very easily.
	- You can use open source frameworks such as junit, testng, NUnit etc and can write selenium test cases in them

QTP VS SELENIUM

Quick Test Professional(QTP) is a proprietary automated testing tool previously owned by the company **Mercury Interactive** before it was **acquired by Hewlett-Packard in 2006**. The Selenium Tool Suite has many advantages over QTP (as of version 11) as detailed below -

QTP VS SELENIUM

QTP	SELENIUM
Open source, free to use, and free of charge.	Commercial.
Highly extensible	Limited add-ons
Can run tests across different browsers	Can only run tests in Firefox, Internet Explorer and Chrome
Supports various operating systems	Can only be used in Windows
Supports mobile devices	Supports mobile device using 3rd party software
Can execute tests while the browser is minimized	Needs to have the application under test to be visible on the desktop
Can execute tests in parallel.	Can only execute in parallel but using Quality Center which is again a paid product.

Advantages of QTP over Selenium

QTP	SELENIUM
Can test both web and desktop applications	Can only test web applications
Comes with a built-in object repository	Has no built-in object repository
Automates faster than Selenium because it is a fully featured IDE.	Automates at a slower rate because it does not have a native IDE and only third party IDE can be used for development
Data-driven testing is easier to perform because it has built-in global and local data tables.	Data-driven testing is more cumbersome since you have to rely on the programming language's capabilities for setting values for your test data
Can access controls within the browser(such as the Favorites bar, Address bar, Back and Forward buttons, etc.)	Cannot access elements outside of the web application under test

Advantages of QTP over Selenium

QTP	SELENIUM
Provides professional customer support	No official user support is being offered.
Has native capability to export test data into external formats	Has no native capability to export runtime data onto external formats
Parameterization Support is in built	Parameterization can be done via programming but is difficult to implement.
Test Reports are generated automatically	No native support to generate test /bug reports.

Advantages of QTP over Selenium

Though clearly, QTP has more advanced capabilities, Selenium outweighs QTP in three main areas:

- **Cost** (because Selenium is completely free)
- **Flexibility** (because of a number of programming languages, browsers, and platforms it can support)
- **Parallel testing** (something that QTP is capable of but only with use of Quality Center)

Who uses Selenium



Scope and Growth of Selenium Testing

- Nowadays, software quality assurance (QA) has become an integral part of each project.
- Many organizations even adopt test-based development approach to make the coding
 - compatible with testing modules.
- The QA professionals further use several test automation tools and frameworks to
 - facilitate test management.
- As a compact and robust software testing framework, Selenium is used widely to web
 - application.
- The open source framework was originally developed by a team of testers at ThoughtWorks in 2014.

Scope and Growth of Selenium Testing

- But within a decade, Selenium has become synonymous with internet application testing.
- A number of recent reports have highlighted that Selenium skills has become one of the most commonly and widely required test automation technology.
- As each enterprise has to optimize the look, feel and performance of a web application to deliver user experience, Selenium testing is expected to grow steadily over next few years.

Scope and Growth of Selenium Testing

Why Selenium will Grow Steadily Over Next Few Years?

- Open Source and Free
- Selenium was released by ThoughtWork under the Apache 2.0 license. So the testing framework is open source, and can be downloaded by users without paying in charges.

Scope and Growth of Selenium Testing

Support for Multiple Web Browsers and Operating Systems

Each internet application must run across several web browsers and operating systems to keep the users engaged. So QA professionals look for a set of tools that allows them to quickly evaluate the performance of the application on some of the widely used web browsers.

Scope and Growth of Selenium Testing

A Complete IDE for Internet Application Testing	The Selenium IDE comes with a set of advanced tools that allows users to record, edit and debug tests without putting any extra effort. As the IDE was earlier known as Selenium Recorder.
Client API for Several Programming Languages	The Selenium IDE records scripts in a special test scripting language called Selenese. But the testers have option to write the test scripts in a variety of programming languages in addition to Selenese. The Selenium Client API supports a number of widely used programming languages like C#, Java, Perl, PHP, Ruby and Python.

Scope and Growth of Selenium Testing

Comprehensive Documentation and Support

While migrating to a new web application testing framework, the QA professionals require additional training. The tools provided by Selenium are well documented. So the QA professionals can refer to a complete documentation and use guide to understand how the tools function.

Advantages of Selenium

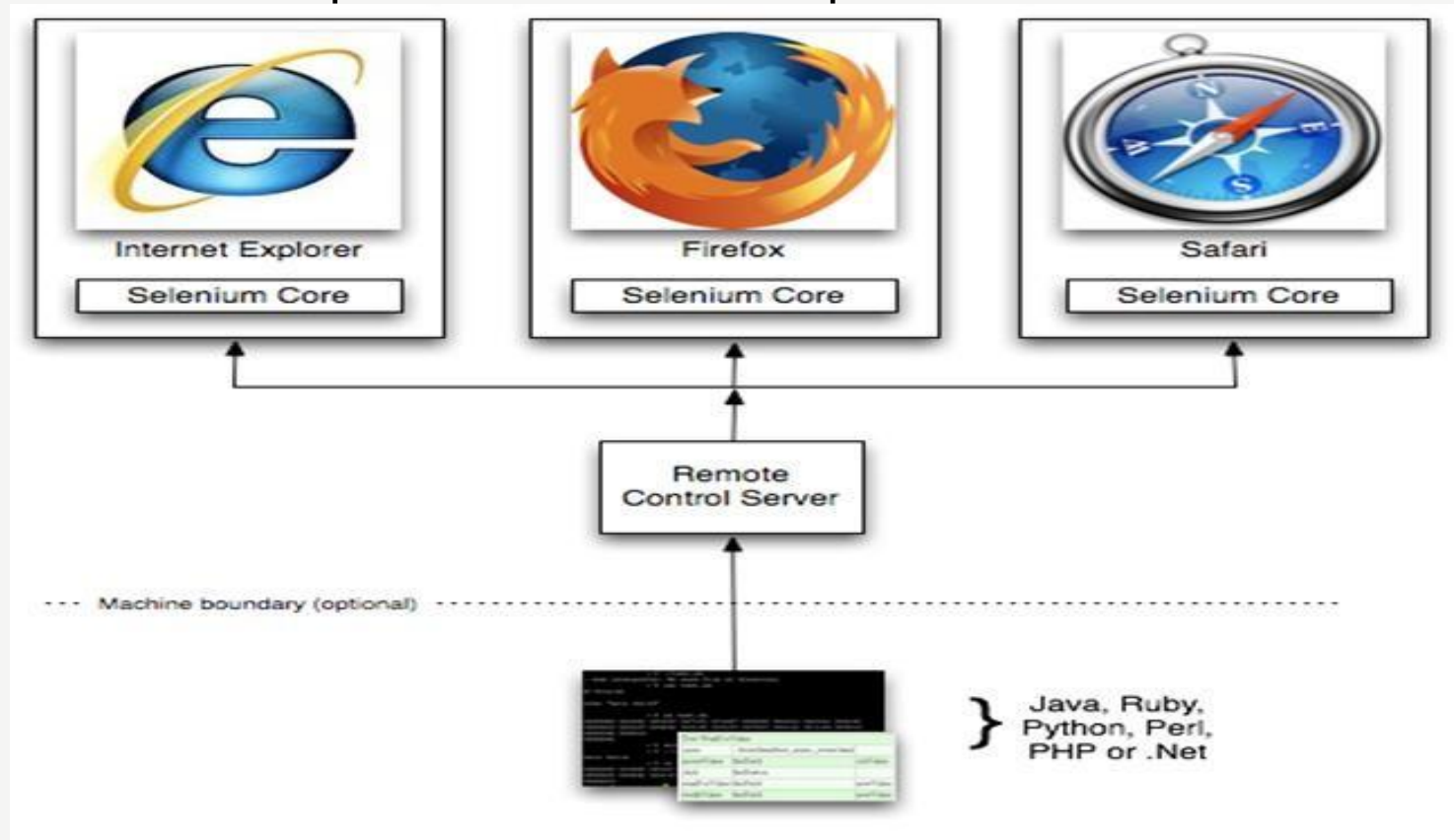
- Open Source
- Supports all browsers like IE, Firefox, Mozilla, Safari
- Supports all Operating Systems.
- Supports all programming languages Java, Ruby, C# and Python.
- Run multiple tests at a time.

Selenium RC Architecture

- Selenium Remote Control is a test tool that allows you to write automated web application UI tests in any programming language against any HTTP website using any mainstream JavaScript-enabled browser.
- Selenium RC comes in two parts:
 - A server which automatically launches and kills browsers, and acts as a HTTP proxy for web requests from them.
 - Client libraries for your favourite computer language.

Selenium RC Architecture

Below is a simplified architectural representation.



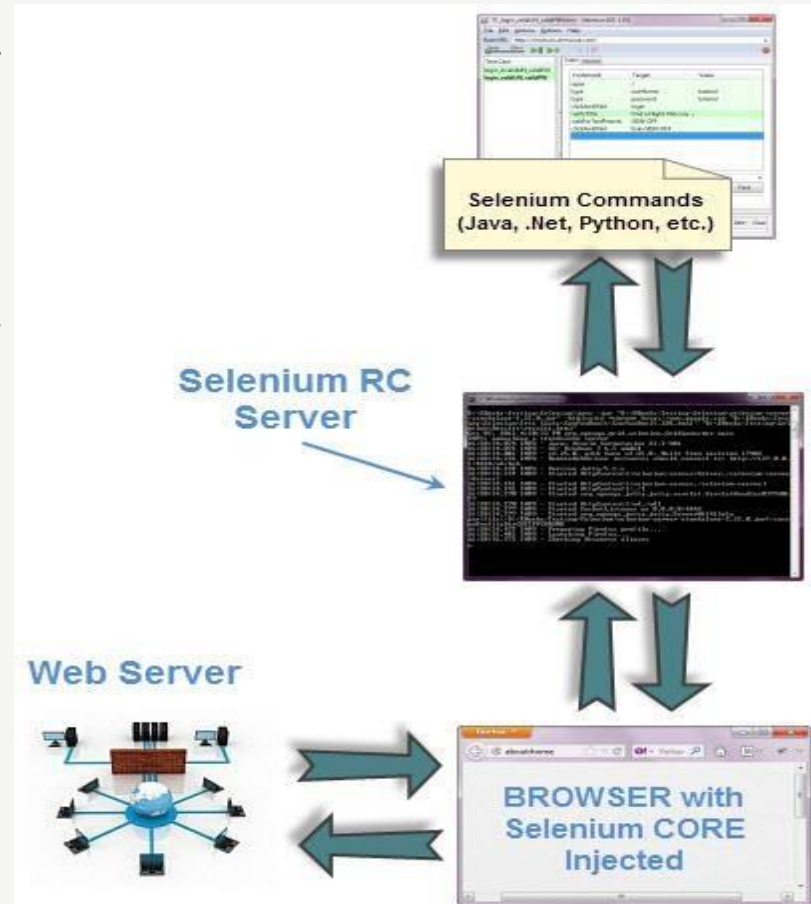
Selenium RC Architecture

Selenium RC's architecture is way more complicated.

- You first need to launch a separate application called Selenium Remote Control (RC) Server before you can start testing
- The Selenium RC Server acts as a "middleman" between your Selenium commands and your browser
- When you begin testing, Selenium RC Server "injects" a JavaScript program called Selenium Core into the browser.
- Once injected, Selenium Core will start receiving instructions relayed by the RC Server from your test program. When the instructions are received, Selenium Core will execute them as JavaScript commands.

Selenium RC Architecture

- The browser will obey the instructions of Selenium Core, and will relay its response to the RC Server.
- The RC Server will receive the response of the browser and then display the results to you.
- RC Server will fetch the next instruction from your test script to repeat the whole cycle.



Web driver Architecture

- WebDriver is a web automation framework that allows you to execute your tests
- against different browsers, not just Firefox (unlike Selenium IDE).
- WebDriver also enables you to use a programming language in creating your test scripts(not possible in Selenium IDE).
- You can now use conditional operations like if-then-else or switch-case.
- You can also perform looping like do-while.
- Following programming languages are supported by WebDriver: Java, .Net, PHP, Python, Perl, Ruby.

Web driver Architecture



The diagram illustrates the Selenium WebDriver architecture. At the top, a yellow box labeled "Selenium Commands (Java, .Net, Python, etc.)" represents the client code. A double-headed arrow connects this box to a "Web Server" icon, which is depicted as a central server with multiple client machines. Another double-headed arrow connects the "Web Server" to a "BROWSER" window, which is shown displaying the URL "http://localhost:8080/".

Web driver Architecture

Web Driver	Selenium RC
WebDriver is faster than Selenium RC since it speaks directly to the browser uses the browser's own engine to control it.	Selenium RC is slower since it uses a Javascript program called Selenium Core. This Selenium Core is the one that directly controls the browser, not you.
	

Web driver Architecture

Brief explanation about advantages of web driver

- Selenium uses JavaScript to automate web pages. This lets it interact very tightly with web content, and was one of the first automation tools to support Ajax and other heavily dynamic pages.
- However, this also means Selenium runs inside the JavaScript sandbox. This means you need to run the Selenium-RC server to get around the same-origin policy, which can sometimes cause issues with browser setup.
- WebDriver on the other hand uses native automation from each language. While this means it takes longer to support new browsers/languages, it does offer a much closer 'feel' to the browser.
- If you're happy with WebDriver, stick with it, it's the future. There are limitations and bugs right now, but if they're not stopping you, go for it.

Selenium Benefits Over WebDriver

- Supports many browsers and many languages, WebDriver needs native implementations for each new language/browser combo.
- Very mature and complete API
- Currently (Sept 2010) supports JavaScript alerts and confirms better

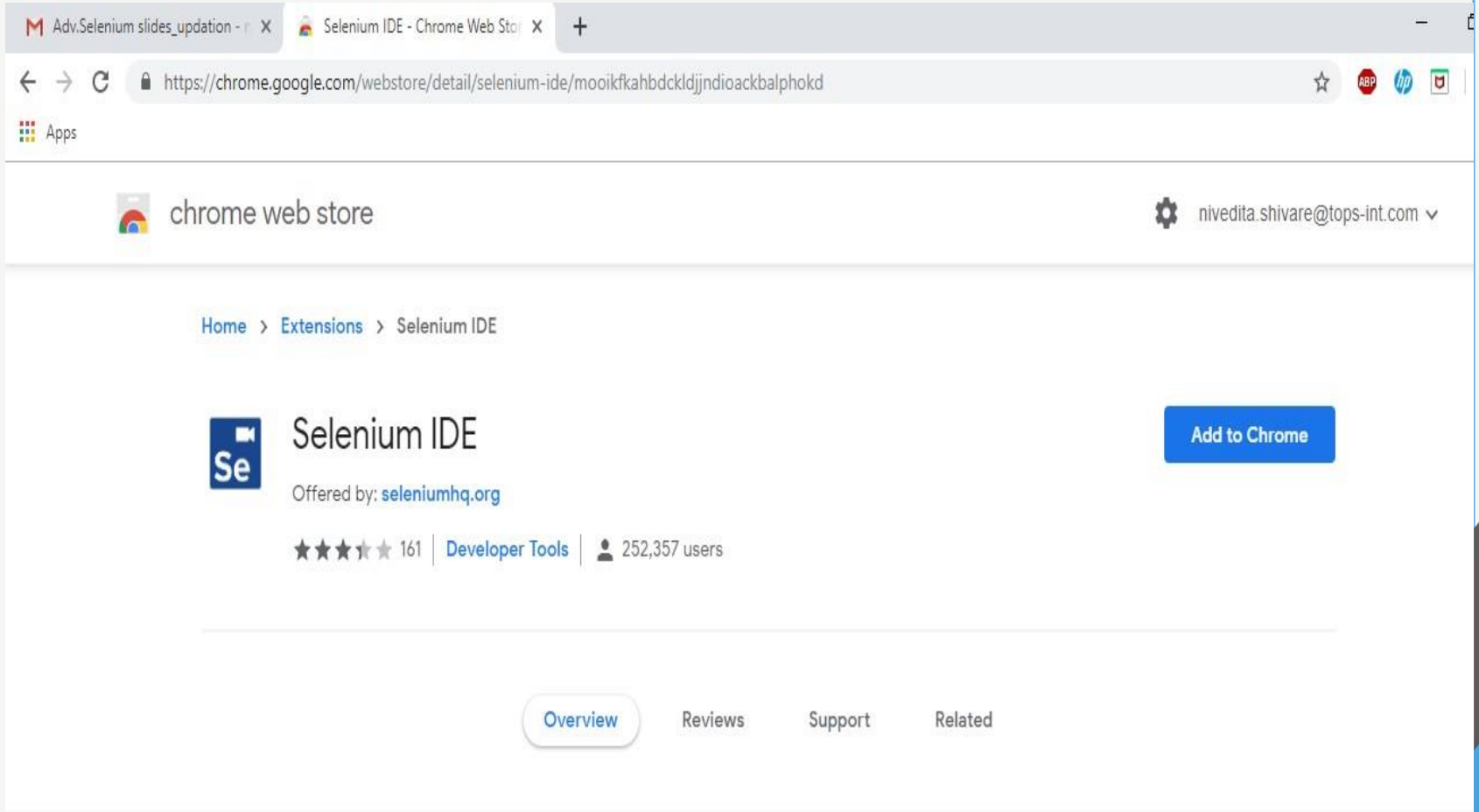
Benefits of WebDriver Compared to Selenium

- Native automation faster and a little less prone to error and browser configuration
- Does not Requires Selenium-RC Server to be running
- Access to headless HTML Unit can allow really fast tests
- Great API

Selenium IDE

- Download and Installation
- Record and playback techniques
- Modifying the script using IDE
- Object, CSS, XPath, Elements Indentify Process
- Validate the locator value using IDE
- Commenting Code
- Fire Bug and Fire path

Add extension of Selenium IDE(Chrome)



The screenshot shows a web browser with two tabs: 'Adv.Selenium slides_update -' and 'Selenium IDE - Chrome Web Store'. The address bar displays the URL 'https://chrome.google.com/webstore/detail/selenium-ide/mooikfahbdckldjndioackbalphokd'. The page header includes the 'chrome web store' logo and a user profile 'nivedita.shivare@tops-int.com'. The main content area shows the 'Selenium IDE' extension page, which is part of the 'Extensions' category. The extension is offered by 'seleniumhq.org' and has a rating of 4.5 stars from 161 reviews. It is categorized as a 'Developer Tool' and has 252,357 users. A blue 'Add to Chrome' button is visible. At the bottom, there are tabs for 'Overview', 'Reviews', 'Support', and 'Related'.

Adv.Selenium slides_update - X Selenium IDE - Chrome Web Store X

https://chrome.google.com/webstore/detail/selenium-ide/mooikfahbdckldjndioackbalphokd

chrome web store nivedita.shivare@tops-int.com

Home > Extensions > Selenium IDE

 **Selenium IDE** [Add to Chrome](#)

Offered by: seleniumhq.org

★★★★☆ 161 | [Developer Tools](#) | 252,357 users

[Overview](#) [Reviews](#) [Support](#) [Related](#)

Process #1: Recording a test script

Scenario

Open “https://accounts.google.com”.

Assert Title of the application

Enter a valid username and password and submit the details to login.

Verify that the user is re-directed to the Home page.

Step 1 – Launch the Firefox and open Selenium IDE from the menu bar.

Step 2 – Enter the address of application under test (“https://accounts.google.com”) inside the Base URL textbox.

Selenium IDE Commands

Each Selenium IDE test step can chiefly be split into following three components:

- Command
- Target
- Value

Command	Target	Value
type	id=Passwd	TestSelenium
Action needs to be performed	The web element to interact with	String that needs to be entered in the web element

Types of Selenium IDE commands:

There are three flavors of Selenium IDE commands. Each of the test step in Selenium IDE falls under any of the following category.

- Actions
- Accessors
- Assertions

Actions:

Actions are those commands which interact directly with the application by either altering its state or by pouring some test data.

For Example, “type” command lets the user to interact directly with the web elements like text box. It allows them to enter a specific value in the text box and as when the value is entered; it is showed on the UI as well.

Another example is “click” command. “click” command lets the user to manipulate with the state of the application.

In case of failure of an action type command, the test script execution halts and rest of the test steps would not be executed.

Accessors

Accessors are those commands which allows user to store certain values to a user defined variable. These stored values can be later on used to create assertions and verifications.

For example, “storeAllLinks” reads and stores all the hyperlinks available within a web page into a user defined variable. Remember the variable is of array type if there are multiple values to store.

Assertions

Similar to Accessors as they do not interact with the application directly. Assertions are used to verify the current state of the application with an expected state.

Forms of Assertions:

- **assert** – the “assert” command makes sure that the test execution is terminated in case of failure.
- **verify** – the “verify” command lets the Selenium IDE to carry on with the test script execution even if the verification is failed.
- **waitFor** – the “waitFor” command waits for a certain condition to be met before executing the next test step. The conditions are like page to be loaded, element to be present. It allows the test execution to proceed even if the condition is not met within the stipulated waiting period.

Commonly used Selenium IDE Commands

Command	Description	Arguments
open	Opens a specified URL in the browser.	1
assertTitle, VerifyTitle	Returns the current page title and compares it with the specified title	1
assertElementPresent, verifyElementPresent	Verify / Asserts the presence of an element on a web page.	1
type, typeKeys, sendKeys	Enters a value (String) in the specified web element.	2

Commonly used Selenium IDE Commands

Command	Description	Arguments
Click, clickAt, clickAndWait	Clicks on a specified web element within a web page.	1
waitForPageToLoad	Sleeps the execution and waits until the page is loaded completely.	1
waitForElement Present	Sleeps the execution and waits until the specified element is present	1
chooseOkOnNext Confirmation, chooseCancelOn NextConfirmation	Click on "OK" or "Cancel" button when next confirmation box appears.	0

Introduction to Firebug

Firebug is a Mozilla Firefox add-on. This tool helps us in identifying or to be more particular inspecting HTML, CSS and JavaScript elements on a web page. It helps us identifying the elements uniquely on a webpage.

The elements can be found uniquely based on their locator types which we would be discussing later...

How to Install Firebug?

For the ease of understanding, we would bifurcate the installation process into the following steps.

Step -1: Launch the Mozilla Firefox browser and navigate to this Firebug add-on download page. The URL takes us to Firefox add-ons section.

Step -2: Click on the “Add to Firefox” button present on the webpage. Refer the following figure for the same.

Installation of Firebug

For the ease of understanding, we would bifurcate the installation process into the following steps.

Step -1: Launch the Mozilla Firefox browser and navigate to this Firebug add-on download page. The URL takes us to Firefox add-ons section.

Step -2: Click on the “Add to Firefox” button present on the webpage. Refer the following figure for the same.

Creating Selenium Script using Firebug

Unlike Selenium IDE, In Firebug, we create automated test scripts manually by adding multiple test steps to form a logical and consistent test script.

Let us follow a progressive approach and understand the process step by step.

Scenario:

Open “<https://accounts.google.com>”.

Assert Title of the application

Enter an invalid username and invalid password and submit the details to login.

Module – 2 [Core Java]

- Introduction to Java
- Features of Java
- Data Type in Java
- Modifier in Java
- Operators
- Array
- OOPs
- File I/O
- Collection
- Sterilization
- Exception Handling

Core Java

Introduction:

- Java programming language was originally developed by Sun Microsystems.
- It is initiated by James Gosling and released in 1995 as core component of Sun Microsystems' Java platform (Java 1.0 [J2SE]).
- Sun Microsystems was acquired by the Oracle Corporation.
- Now it is part of Oracle.
- Java is guaranteed to be Write Once, Run Anywhere.
- Platforms available :
 - J2SE for Standard Edition
 - J2EE for Enterprise Applications,
 - J2ME for Mobile Applications.

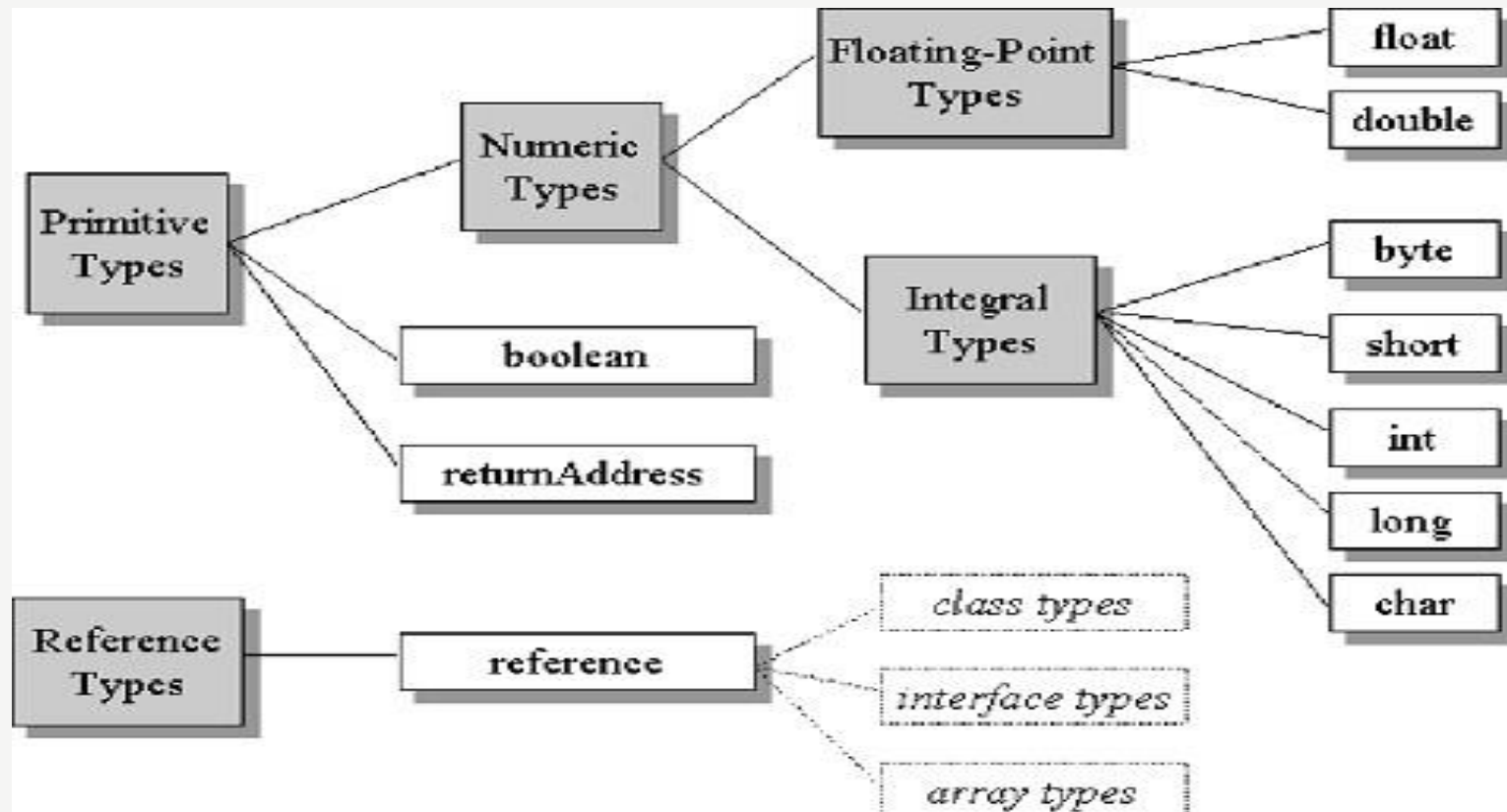
Features of Java

- Java is simple
- Java is object-oriented
- Java is distributed
- Java is interpreted
- Java is robust/powerful
- Java is secure
- Java is architecture-neutral
- Java is portable
- Java's performance
- Java is multithreaded
- Java is dynamic

Data types in Java

- Variables are nothing but reserved memory locations to store values.
- Based on the data type of a variable, the operating system allocates memory and decides what can be stored in the reserved memory.
- There are two data types available in Java:
 - Primitive Data Types
 - Reference/Object Data Types

Data types in Java



Data types in Java

Datatypes	Discription
byte	8-bit signed two's complement integer. Range $(-128 (-2^7) \text{ to } 127 (\text{inclusive})(2^7 - 1))$. Default value is 0
Short	16-bit signed two's complement integer. Range $-32,768 (-2^{15}) \text{ to } 32,767 (\text{inclusive}) (2^{15} - 1)$. Default value is 0.
Int	32-bit signed two's complement integer. Range $-2,147,483,648.(2^{31}) \text{ to } 2,147,483,647 (\text{inclusive}).(2^{31} - 1)$. Default value is 0.
Long	64-bit signed two's complement integer. Range $-9,223,372,036,854,775,808.(-2^{63}) \text{ to } 9,223,372,036,854,775,807 (\text{inclusive}). (2^{63} - 1)$. Default value is 0L.

Data types in Java

Datatypes	Discription
Float	single-precision 32-bit IEEE 754 floating point. Default value is 0.0f.
Double	double-precision 64-bit IEEE 754 floating point. Default value is 0.0d.
Boolean	boolean data type represents one bit of information either true or false. Default value is false.
char	single 16-bit Unicode character. Range '\u0000' (or 0) to '\uffff' (or 65,535 inclusive).

Data types in Java

In Java a reference data type is a variable that can contain the reference or an address of dynamically created object. The reference data types are arrays, classes and interfaces.

- **Class type**

e.g.

```
public class Person
```

```
{ .....
```

```
Person p=new Person();//p is of class type variable.
```

```
}
```

- **Array Type**

An array is a special kind of object that contains values called elements. The java array enables the user to store the values of the same type in contiguous memory allocations.

e.g.

```
int [] a = new int [10];
```

```
String [] cities = ,"Ahmedabad","Mumbai", "Pune"-;
```


Keywords in Java

Keywords or Reserved words used for some internal process or represent some predefined actions. These words are therefore not allowed to use as a variable names or objects. Doing this will result into a **compile time error**.

57 keywords

Keywords in Java				
abstract	default	if	private	this
assert	do	implements	protected	throw
boolean	double	import	public	throws
break	else	instanceof	return	transient
byte	enum	int	short	try
case	extends	interface	static	void
catch	final	long	strictfp	volatile
char	finally	native	super	while
class	float	new	switch	
continue	for	package	synchronized	

Modifiers in Java

Modifier	Same Class	Same Package	Within Subclass outside the package	World
public	Y	Y	Y	Y
protected	Y	Y	Y	N
no modifier	Y	Y	N	N
private	Y	N	N	N

Assignment Operator(=) in Java

- **The Simple Assignment Operator**

One of the most common operators that you'll encounter is the simple assignment operator "=". It assigns the value on its right to the operand on its left:

```
int cadence = 0;
```

```
int speed = 0;
```

```
int gear = 1;
```

This operator can also be used on objects to assign *object references*.

Operators in Java

Java provides a rich set of operators to manipulate variables. We can divide all the Java operators into the following groups:

- Arithmetic Operators
- Relational Operators
- Bitwise Operators
- Logical Operators
- Assignment Operators
- Misc Operators

The Arithmetic Operators:

Arithmetic operators are used in mathematical expressions in the same way that they are used in algebra. The following table lists the arithmetic operators:

Assume integer variable A holds 10 and variable B holds 20, then:

Arithmetic Operators in Java

Operator	Description	Example
+	Addition - Adds values on either side of the operator	A + B will give 30
-	Subtraction - Subtracts right hand operand from left hand operand	A - B will give -10
*	Multiplication - Multiplies values on either side of the operator	A * B will give 200
%	Modulus - Divides left hand operand by right hand operand and returns remainder	B % A will give 0
/	Division - Divides left hand operand by right hand operand	B / A will give 2
++	Increment - Increases the value of operand by 1	B++ gives 21
--	Decrement - Decreases the value of operand by 1	B-- gives 19

Relational Operators in Java

- There are following relational operators supported by Java language
- Assume variable A holds 10 and variable B holds 20, then:

Operator	Description	Example
==	Checks if the values of two operands are equal or not, if yes then condition becomes true.	(A == B) is not true.
!=	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.	(A != B) is true.
>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	(A > B) is not true.

Bitwise Operators in Java

- Java defines several bitwise operators, which can be applied to the integer types, long, int, short, char, and byte.
- Bitwise operator works on bits and performs bit-by-bit operation. Assume if a = 60; and b = 13; now in binary format they will be as follows:

a = 0011 1100

b = 0000 1101

a&b = 0000 1100

a|b = 0011 1101

a^b = 0011 0001

~a = 1100 0011

Logical Operators in Java

- The following table lists the logical operators:
- Assume Boolean variables A holds true and variable B holds false, then:

Operator	Description	Example
&&	Called Logical AND operator. If both the operands are non-zero, then the condition becomes true.	(A && B) is false.
	Called Logical OR Operator. If any of the two operands are non-zero, then the condition becomes true.	(A B) is true.
!	Called Logical NOT Operator. Use to reverses the logical state of its operand. If a condition is true then Logical NOT operator will make false.	!(A && B) is true.

Assignment Operators in Java

Operator	Description	Example
=	Simple assignment operator, Assigns values from right side operands to left side operand	$C = A + B$ will assign value of $A + B$ into C
+=	Add AND assignment operator, It adds right operand to the left operand and assign the result to left operand	$C += A$ is equivalent to $C = C + A$
-=	Subtract AND assignment operator, It subtracts right operand from the left operand and assign the result to left operand	$C -= A$ is equivalent to $C = C - A$
*=	Multiply AND assignment operator, It multiplies right operand with the left operand and assign the result to left operand	$C *= A$ is equivalent to $C = C * A$

Assignment Operators in Java

Operator	Description	Example
/=	Divide AND assignment operator, It divides left operand with the right operand and assign the result to left operand	C /= A is equivalent to C = C / A
%=	Modulus AND assignment operator, It takes modulus using two operands and assign the result to left operand	C %= A is equivalent to C = C % A
<<=	Left shift AND assignment operator	C <<= 2 is same as C = C << 2
>>=	Right shift AND assignment operator	C >>= 2 is same as C = C >> 2
&=	Bitwise AND assignment operator	C &= 2 is same as C = C & 2
^=	bitwise exclusive OR and assignment operator	C ^= 2 is same as C = C ^ 2

Misc. Operator in Java

Conditional Operator

Conditional operator is also known as the ternary operator. This operator consists of three operands and is used to evaluate Boolean expressions. The goal of the operator is to decide which value should be assigned to the variable. The operator is written as:

variable x = (expression) ? value if true : value if false

Introduction to Eclipse IDE

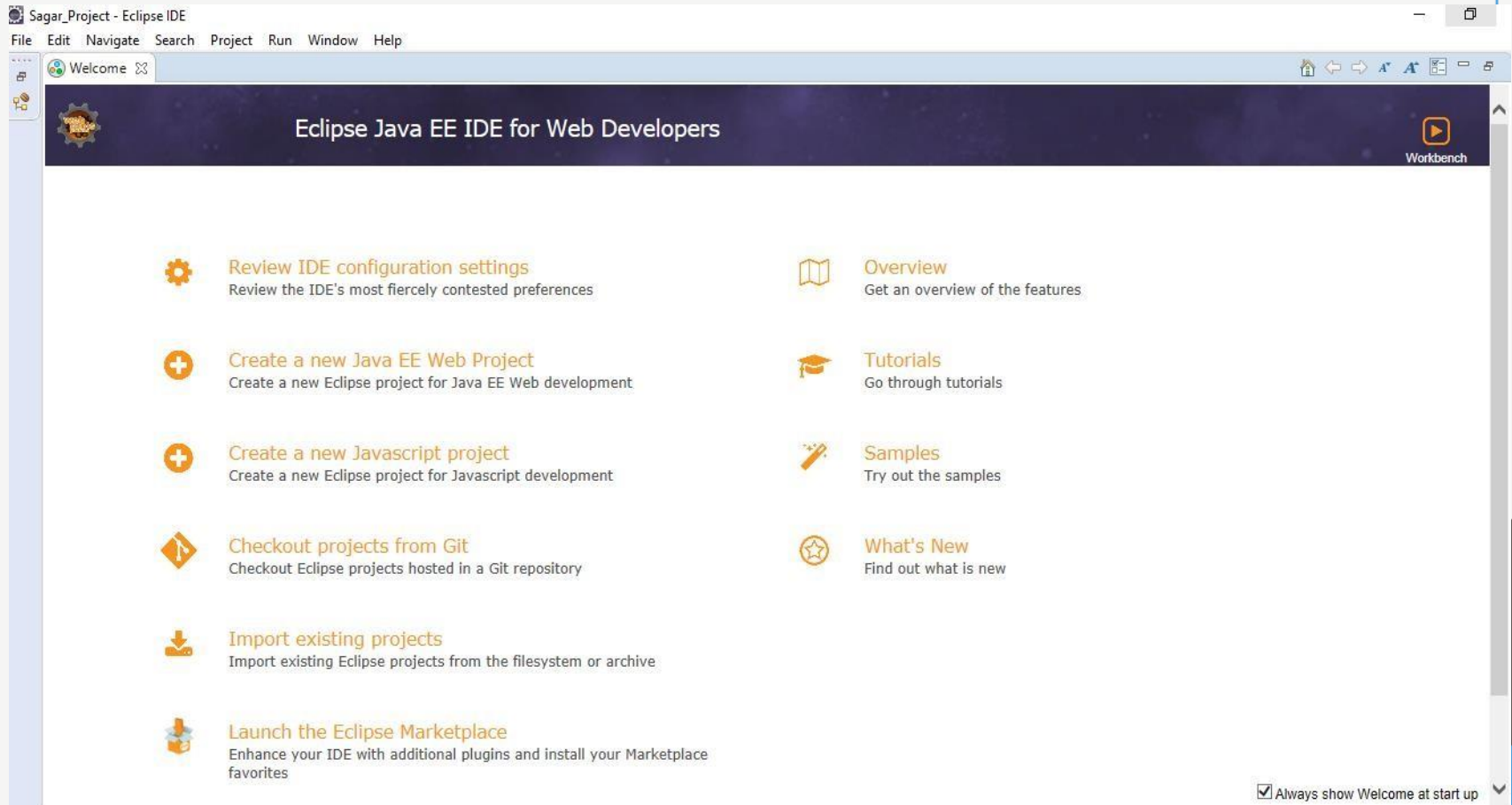
Java IDE Tools

- To write your Java programs, you will need a text editor. There are even more sophisticated IDEs available in the market.
- **Notepad:** On Windows machine you can use any simple text editor like Notepad, TextPad.
- **Netbeans:** is a Java IDE that is open-source.
- **Eclipse:** is also a Java IDE developed by the eclipse open-source community.

About Eclipse IDE

- Most people know **Eclipse** as an **integrated development environment (IDE)** for Java.
- Today it is the leading development environment for Java with a market share of approximately 65%.
- The Eclipse IDE can be extended with additional software components.
- Eclipse calls these software components plug-ins. Several Open Source projects and companies have extended the Eclipse IDE

Eclipse IDE



Java program Structure

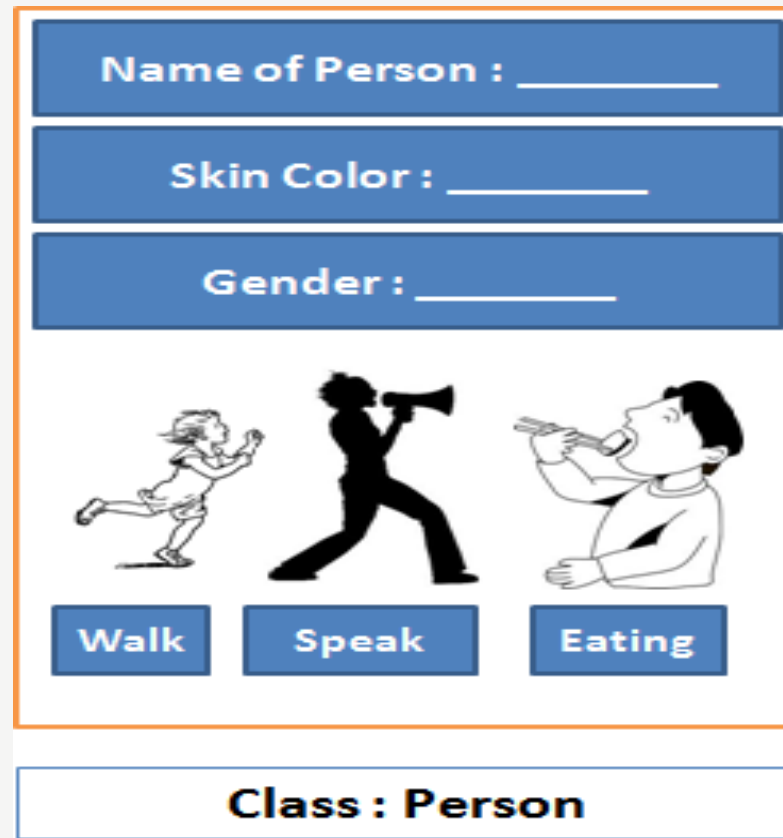
Introduction of

- Class, Object , Class Members
- Constructor

Class, Object and members

● Class

- A class can be defined as a template/blue print that describes the behaviors/states that object of its type support.



Class, Object and members

- Objects are key to understanding object-oriented technology.
- Examples of real-world objects:
 - your dog,
 - your desk,
 - your television set,
 - your bicycle.
- Real-world objects share two characteristics:
 - They all have state and behavior.
 - Example:
 - A dog has states - color, name, breed as well as behaviors -wagging, barking, eating.
 - An object is an instance of a class.

Class, Object and members

- An **object** is an instance of a class created using a new operator.
- The new operator returns a reference to a new instance of a class.
- Eg `Person me=new Person();`

Class Members

- When we create a class its totally incomplete without defining any member of this class.

Class, Object and members

- **Field:** field is nothing but the property of the class or object which we are going to create .
 - for example if we are creating a class called computer then they have property like model, mem_size, hd_size, os_type etc
- **Method:** method is nothing but the operation that an object can perform it define the behavior of object how an object can interact with outside world .
 - For example : startMethod (), shutdownMethod ().

Refer Example here-

https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.0%20Class%20and%20Object/2.0.1%20Class_ObjectDemo.java

Constructors

- Every class has a constructor. If we do not explicitly write a constructor for a class the Java compiler builds a default constructor for that class.
- Each time a new object is created, at least one constructor will be invoked.
- The main rule of constructors is that they should have the same name as the class.
- A class can have more than one constructor.
- Constructor declarations look like method declarations—except that they use the name of the class and have no return type.

Constructors

- They are called or invoked when an object of class is created and can't be called explicitly.
- E.g.

```
public class Person{  
    //attribute declaration  
    Person(){  
    }  
    //method declaration }
```
- Like other methods in java constructor can be overloaded i.e. we can create as many constructors in our class as desired.
- Number of constructors depends on the information about attributes of an object we have while creating objects.



TOPSTECHNOLOGIES

Training | Outsourcing | Placement | Study Abroad

Constructors

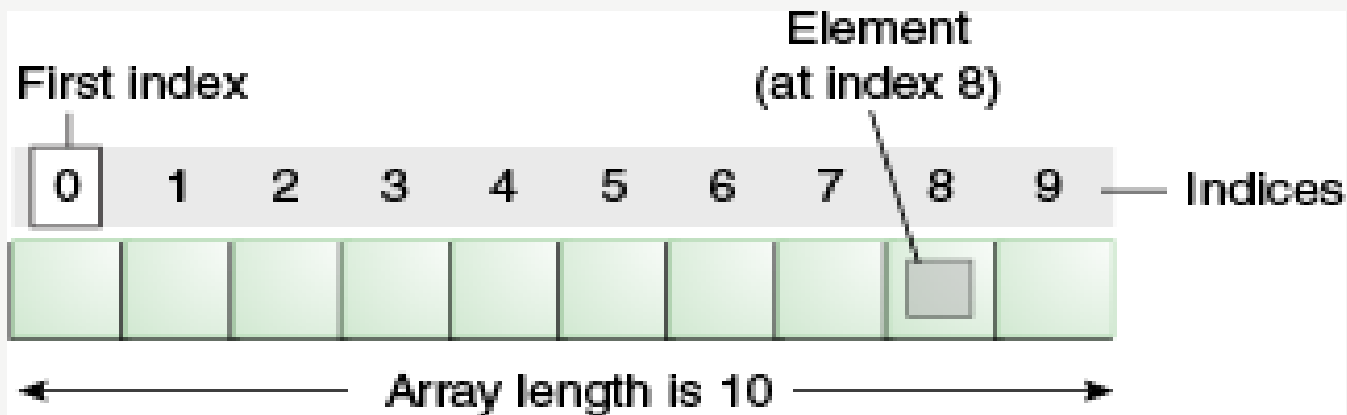
Refer Example here-

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.1%20Constructor/2.1.1%20ConstructorDemo.java>

Array

•Introduction:

- An array is a container object that holds a fixed number of values of a single type.
- The length of an array is established when the array is created.
- After creation, its length is fixed.



Array

Advantages of Array:

- Arrays are most appropriate for storing a fixed amount of data which will be accessed in an unpredictable fashion.
- Arrays that has become very important on modern architectures is that iterating through an array has good locality of reference.
- Arrays are much faster than iterating through a linked list of the same size, which tends to jump around in memory.
- Arrays also are among the most compact data structures; storing 100 integers in an array takes only 100 times the space required to store an integer

Array

Array Types

There are 2 types of array available in java.

1) Primitive type,

- 1. byte b; // byte is a primitive type
- 2. byte[] arrayOfBytes; // byte[] is an array type: array of byte
- 3. byte[][] arrayOfArrayOfBytes; // byte[][] is another type: array of byte[]

2) Class/Object type.

Point[] points;

Array

Primitive Types and Reference type Arrays.

- char s[];
- Point p[];
- char[] s;
- Point[] p;

Primitive Array

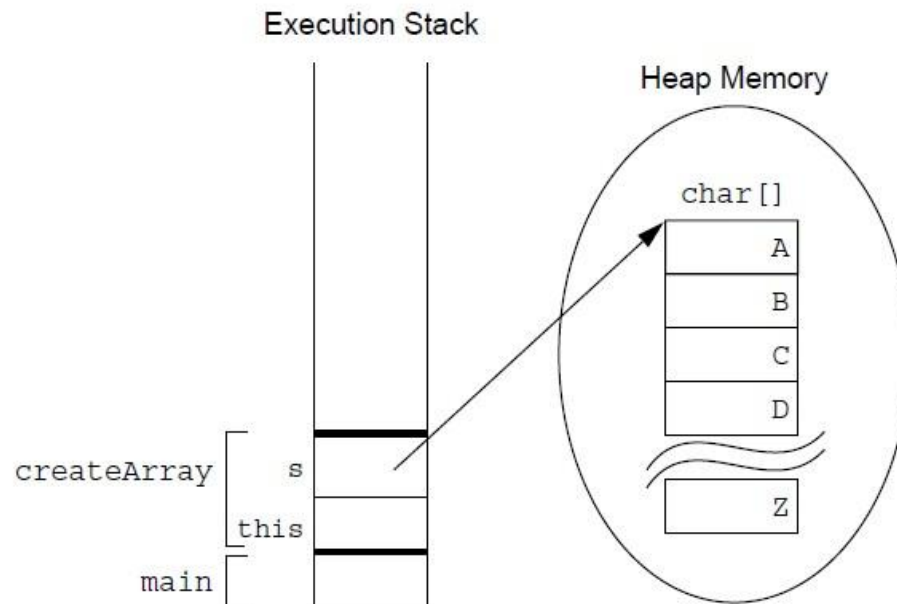
```
public char[] createArray() {  
    char[] s= new char[26];  
    for ( int i=0; i<26; i++ ) {  
        s[i]= (char) ('A' + i);  
    }  
    return s;}  

```



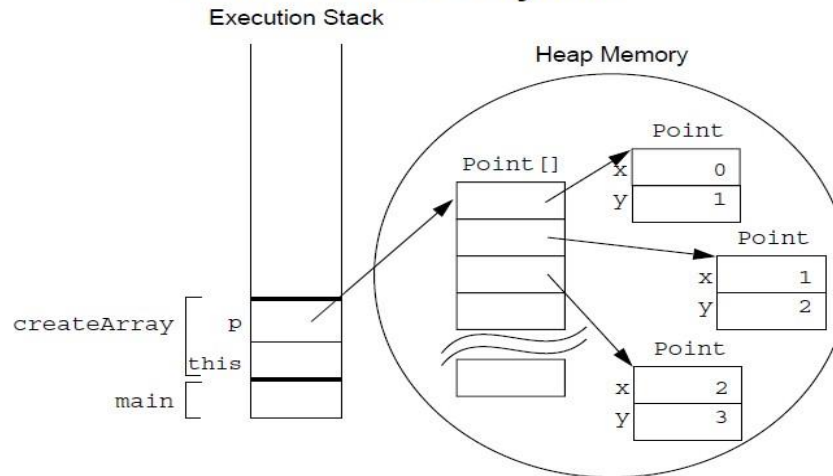
Array

Creating an Array of Character Primitives



Array

Creating an Array of Character Primitives With Point Objects



Array

Refer Example here-

https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.2%20Array/2.2.1%20Array_Demo.java

String in Java

- The String class represents character strings.
- Strings are constant;
- Their values cannot be changed after they are created.
- It's immutable/fixed class.



Methods of String class

Function Name	Use
<code>char charAt(int index)</code>	Returns the char value at the specified index. An index ranges from 0 to <code>length() - 1</code>
<code>public void getChars(int srcBegin, int srcEnd, char[] dst, int dstBegin)</code>	Copies characters from this string into the destination character array.

Methods of String class

Function Name	Use
public void getBytes(int srcBegin, int srcEnd, byte[] dst, int dstBegin)	Copies characters from this string into the destination byte array.
public boolean equalsIgnoreCase(String anotherString)	Compares this String to another String, ignoring case considerations.

Methods of String class

Function Name	Use
<code>public String substring(int beginIndex)</code>	Returns a new string that is a substring of this string. The substring begins with the character at the specified index and extends to the end of this string.
<code>public String concat(String str)</code>	Concatenates the specified string to the end of this string.
<code>public String replace(char oldChar, char newChar)</code>	Returns a new string resulting from replacing all occurrences of oldChar in this string with newChar.

Methods of String class

Function Name	Use
<code>public String trim()</code>	Returns a copy of the string, with leading and trailing whitespace omitted.
<code>public char[] toCharArray()</code>	Converts this string to a new character array.
<code>public static String valueOf(char c)</code>	Returns the string representation of the char argument.

Methods of String class

Refer Example here-

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.3%20String/2.3.1%20StringDemo.java>

StringBuffer and StringBuilder

Strings objects of type StringBuffer and StringBuilder can be modified over and over again without leaving behind a lot of new unused objects.

StringBuffer	StringBuilder
StringBuffer methods are thread safe.	StringBuilders methods are not thread safe(not Synchronised).
Less efficient than StringBuilder.	More efficient than StringBuffer.
When the application needs to run on multiple threads then it is better to use StringBuffer.	When the application needs to be run only in a single thread then it is better to use StringBuilder.

StringBuffer and StringBuilder

StringBuffer:Refer Example here-

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.3%20String/2.3.3%20StringBufferDemo.java>

StringBuilder:Refer Example here-

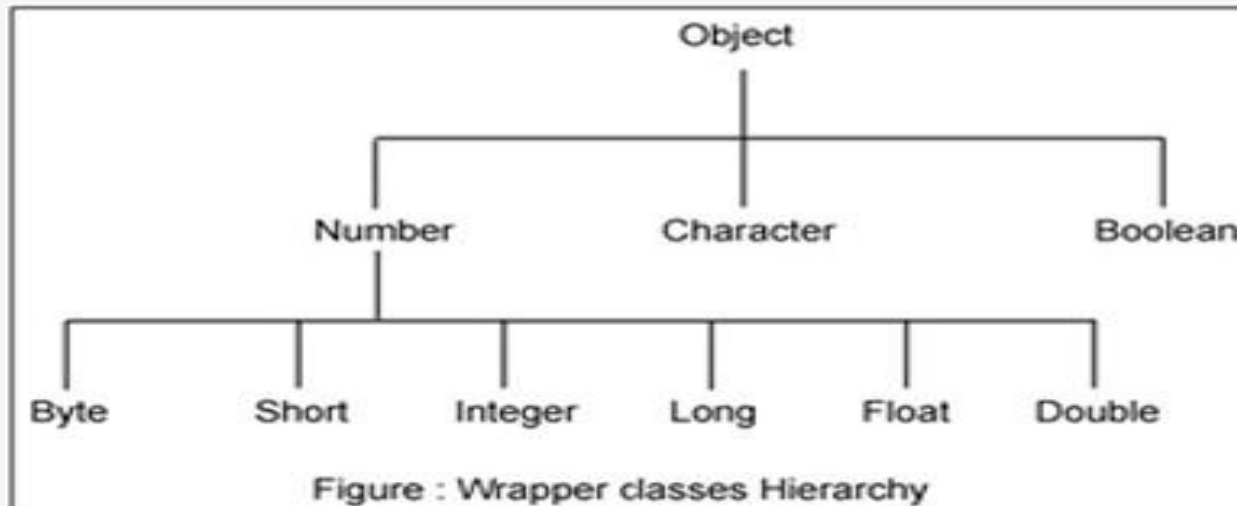
<https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.3%20String/2.3.2%20StringBuilderDemo.java>

Wrapper Class

- Each of Java's eight primitive data types has a class dedicated to it.

Several possible reasons for wrapper classes availability,

- For the null value is possible,
- To include in a Collection,
- To treat generically / polymorphically as an Object along with other Objects



Boxing

- Some Terms:
- **Boxing**
 - Boxing/wrapping, is the process of placing a primitive type within an object so that the primitive can be used as a reference object.
- **Autoboxing**
 - Autoboxing is the term for getting a reference type out of a value type just through type conversion (either implicit or explicit).
 - The compiler automatically supplies the extra source code which creates the object.
 - `Integer i = new Integer(9);`
 - `Integer l = 9;`

Boxing

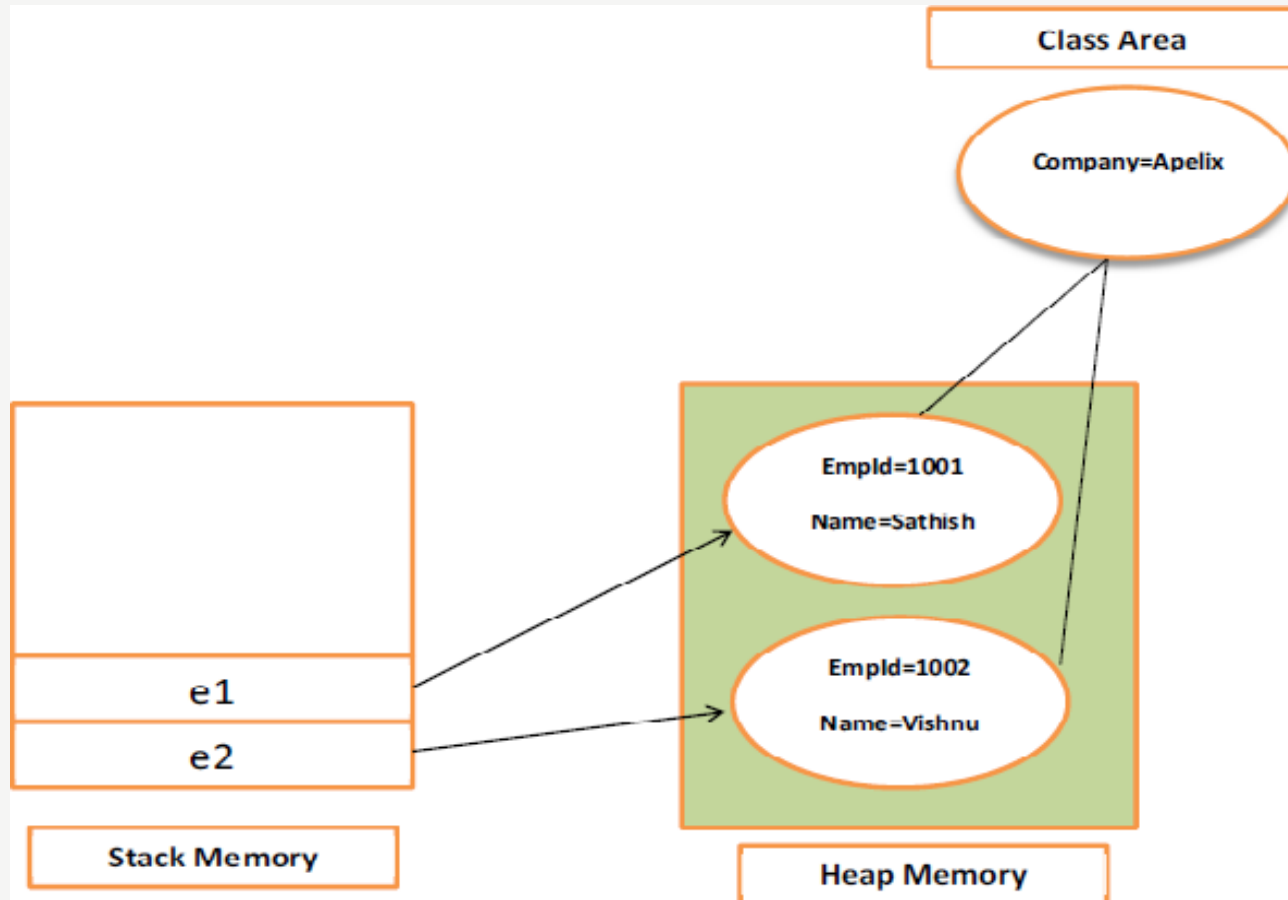
● Unboxing

- Unboxing refers to getting the value which is associated to a given object, just through type conversion (either implicit or explicit).
- The compiler automatically supplies the extra source code which retrieves the value out of that object, either by invoking some method on that object, or by other means.
- Integer k = new Integer(4);
- int l = k.intValue();
- int m = k;

Refer Example here-

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.4%20Wrapper%20Class/2.4.1%20WrapperClassDemo.java>

Keyword-static



static variable

- Java instance variables are given separate memory for storage.
- If there is a need for a variable to be common to all the objects of a single java class, then the static modifier should be used in the variable declaration.
- Any java object that belongs to that class can modify its static variables.
- Also, an instance is not a must to modify the static variable and it can be accessed using the java class directly.
- Static variables can be accessed by java instance methods also.

static method

- static methods are also common to classes and not tied to a java instance.
- static methods should be invoked with using the class name though it can be invoked using an object.

ClassName.methodName(arguments)

or

objectName.methodName(arguments)

- General use for java static methods is to access static fields.
- Static methods can be accessed by java instance methods.
- Java static methods cannot access instance variables or instance methods directly.
- Java static methods cannot use the “this” keyword.

static initialize block

- A class can contain code in a static block that does not exist within a method body.
- Static block code executes once only, when the class is loaded.
- Usually, a static block is used to initialize static (class) attributes.

Refer Example here-

https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.5%20Keywords/2.5.1%20Keyword_staticDemo.java

Keyword-final

- **final** modifier can be used with
Class,
Variable &
Method.

- **final class**

This class can not extended.

```
public final class MyFinalClass  
{...  
}
```

// forbidden

```
public class subclass extends MyFinalClass  
{...  
}
```

Keyword-final

final variable

- A final variable is a constant.
- A variable that is declared as final and not initialized is called a blank final variable.
- A blank final variable forces the constructors to initialise it.

Keyword-final

final method

- You cannot override a final method.
- A blank final method variable must be set in the method body before being used.

Refer Example here-

https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.5%20Keywords/2.5.2%20Keyword_finalDemo.java

OOPS

- Object Oriented Programming System
- Object means a real word entity such as pen, chair, table etc.
- Object-Oriented Programming is a methodology or paradigm to design a program using classes and objects.
- It simplifies the software development and maintenance by providing some concepts:



TOPSTECHNOLOGIES

Training | Outsourcing | Placement | Study Abroad

OOP Concepts

- Object
- Class
- Inheritance
- Polymorphism
- Abstraction
- Encapsulation

OOP Concepts

Object	Any entity that has state and behavior is known as an object. For example: chair, pen, table, keyboard, bike etc. It can be physical and logical.
Class	Collection of objects is called class. It is a logical entity.

Refer Example here-

https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.6%20OOPS/2.6.1%20Class%20and%20Object/2.6.1.1%20Class_ObjectDemo.java

OOP Concepts

Encapsulation	Binding (or wrapping) code and data together into a single unit is known as encapsulation. For example: capsule, it is wrapped with different medicines. A java class is the example of encapsulation. Java bean is the fully encapsulated class because all the data members are private here.

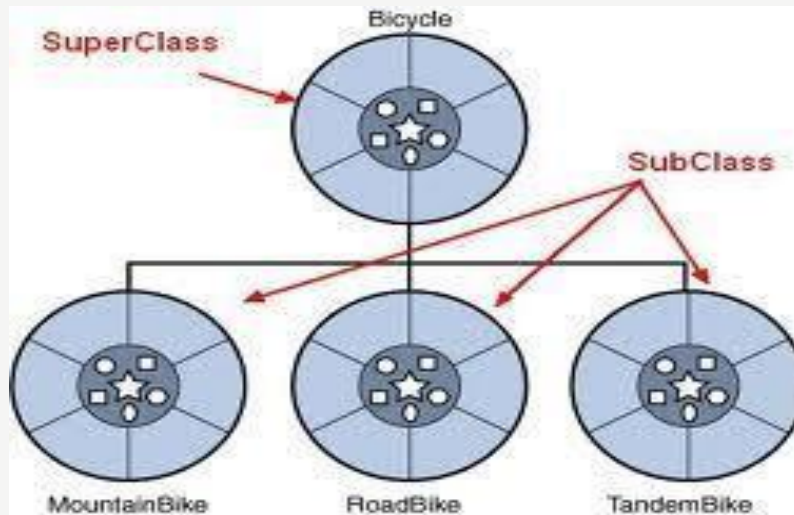
Refer Example here-

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.6%20OOPS/2.6.2%20Encapsulation/2.6.2.1%20EncapsulationDemo.java>

OOP Concepts

Inheritance

- Inheritance is a mechanism wherein a new class is derived from an existing class.
- In Java, classes may inherit or acquire the properties and methods of other classes.
- A class derived from another class is called a sub class, where as the class from which a subclass is derived is called a super class.



OOP Concepts

Inheritance-Use

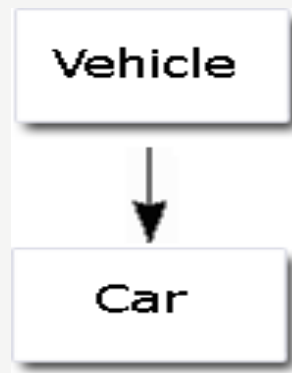
Reusability	facility to use public methods of base class without rewriting the same.
Extensibility	extending the base class logic as per business logic of the derived class.
Data hiding	base class can decide to keep some data private so that it cannot be altered by the derived class.
Overriding	With inheritance, we will be able to override the methods of the base class so that meaningful implementation of the base class method can be designed in the derived class.



OOP Concepts

Inheritance Types-

Simple Inheritance



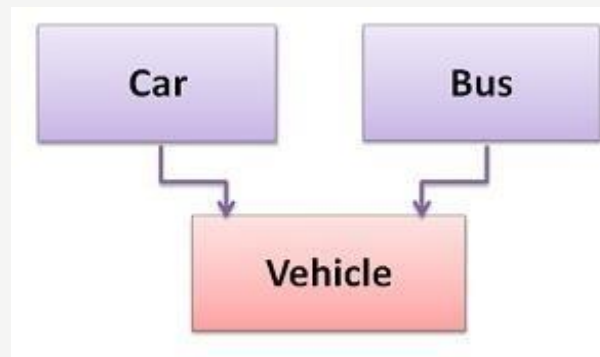
Multilevel Inheritance



OOP Concepts

Inheritance Types-

Multiple Inheritance



OOP Concepts

Simple Inheritance

- When a subclass is derived simply from its parent class then this mechanism is known as simple inheritance.
- In case of simple inheritance there is only a sub class and its parent class.
- It is also called single inheritance or one level inheritance.

Refer Example here-

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.6%20OOPS/2.6.3%20Inheritance/2.6.3.1%20SimpleInheritanceDemo.java>

OOP Concepts

- Multilevel Inheritance
 - It is the enhancement of the concept of inheritance. When a subclass is derived from a derived class then this mechanism is known as the multilevel inheritance.
 - The derived class is called the subclass or child class for its parent class and this
 - parent class works as the child class for its just above (parent) class.
 - Multilevel inheritance can go up to any number of level.
- Refer Example here-
- <https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.6%20OOPS/2.6.3%20Inheritance/2.6.3.2%20MultilevelInheritanceDemo.java>

OOP Concepts

Multiple Inheritance

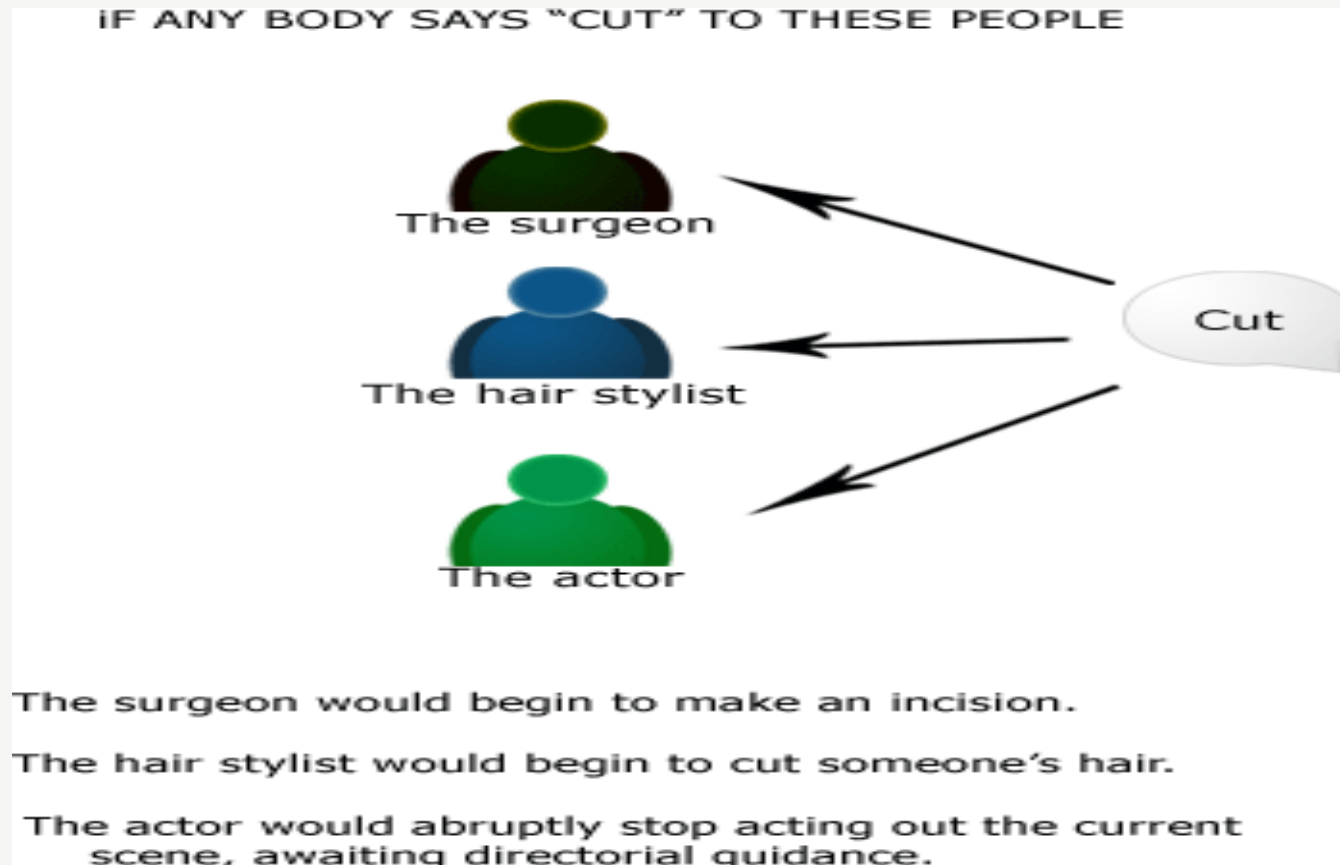
- Java does not support multiple Inheritance.
- The mechanism of inheriting the features of more than one base class into a single class is known as multiple inheritance.
- Java does not support multiple inheritance but the multiple inheritance can be achieved by using the interface.
- In Java Multiple Inheritance can be achieved through use of Interfaces by implementing more than one interfaces in a class.

OOP Concepts

Polymorphism	When one task is performed by different ways i.e. known as polymorphism. For example: to converse the customer differently, to draw something e.g. shape or rectangle etc. In java, we use method overloading and method overriding to achieve polymorphism. Another example can be to speak something e.g. cat speaks meaw, dog barks woof etc.

- Types of Polymorphism
 - Method Overloading
 - Method Overriding

OOP Concepts



OOP Concepts

Polymorphism

- Polymorphism is the ability of an object to take on many forms.
- Any Java object that can pass more than one IS-A test is considered to be polymorphic.
- For e.g.

```
public interface Vegetarian{  
    }  
public class Animal{  
    }  
public class Deer extends Animal implements Vegetarian{}
```

OOP Concepts

Polymorphism

- The Deer class is considered to be polymorphic since this has multiple inheritance.
- Following are true for the above example:
 - A Deer IS-A Animal
 - A Deer IS-A Vegetarian
 - A Deer IS-A Deer
 - A Deer IS-A Object
- Polymorphism is essentially considered into two versions.
 - Compile time polymorphism /static binding/method overloading.
 - Runtime polymorphism/dynamic binding/method overriding.

OOP Concepts

Method overloading	Method overriding
In case of method overloading in Java, Signature of method changes .	while in case of method overriding signature of method (including return type, number of method parameters, type of parameters and order of parameters)remain same.
Java is that You can overload method in one class .	While overriding can only be done on subclass.
You can overload static, final and private method in Java.	While you can not override static, final or private method in Java.
Overloaded method in Java is bonded by static binding .	Overridden methods are subject to dynamic binding.

OOP Concepts

Method Overloading:Refer Example here-

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.6%20OOPS/2.6.4%20Polymorphism/2.6.4.1%20MethodOverloadingDemo.java>

Method Overriding:Refer Example here-

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.6%20OOPS/2.6.4%20Polymorphism/2.6.4.2%20MethodOverridingDemo.java>

OOP Concepts

Abstraction	Hiding internal details and showing functionality is known as abstraction. For example: phone call, we don't know the internal processing. In java, we use abstract class and interface to achieve abstraction.

Interface

- An interface is a collection of abstract methods.
- A class implements an interface, thereby inheriting the abstract methods of the interface.
- Class describes the attributes and behaviours of an object.
- An interface contains behaviours that a class implements.

OOP Concepts

Class vs. Interface

- You cannot instantiate an interface.
- An interface does not contain any constructors.
- All of the methods in an interface are abstract.
- An interface cannot contain instance fields. The only fields that can appear in an interface must be declared both static and final.
- An interface is not extended by a class; it is implemented by a class.
- An interface is not extended by a class; it is implemented by a class.
- An interface can extend multiple interfaces.

OOP Concepts

Abstract Class : Refer Example here-

https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.6%20OOPS/2.6.5%20Abstraction/2.6.5.1%20Abstraction_AbstractClassDemo.java

Interface: Refer Example here-

https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.6%20OOPS/2.6.5%20Abstraction/2.6.5.2%20Abstraction_InterfaceDemo.java

Exception in Java

- Introduction
- Types of Exception
- try catch and finally block
- Multi catch Exceptions
- throw and throws keywords
- Method overriding with Exceptions
- Custom Exceptions

Exception in Java

Introduction

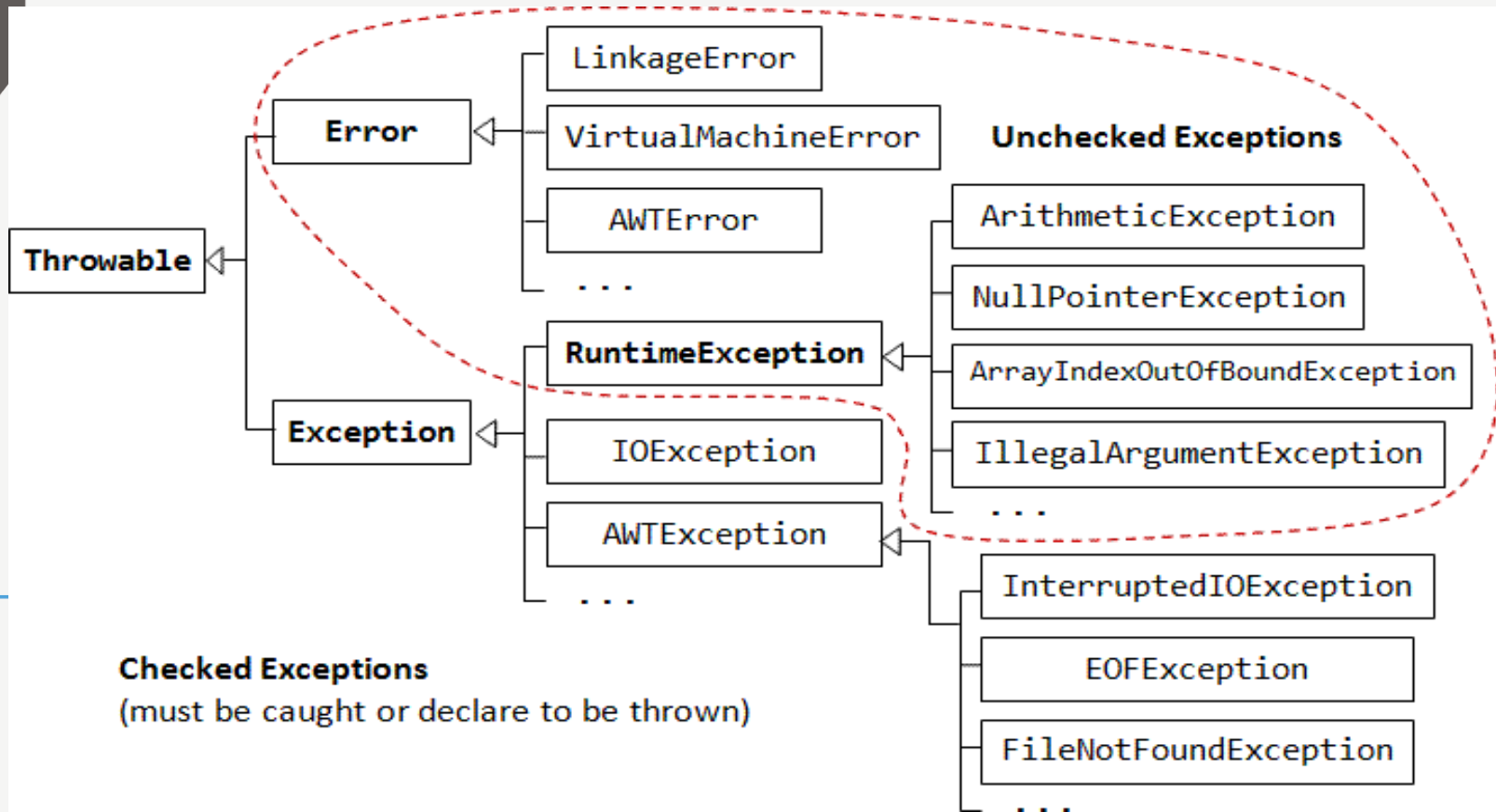
- An exception is a problem that arises during the execution of a program.
- An exception can occur for many different reasons:
 - A user has entered invalid data.
 - A file that needs to be opened cannot be found.
 - A network connection has been lost in the middle of communications or the JVM has run out of memory.

Exception in Java

- There are three categories of exceptions:

Exception	Description
Checked exceptions:	A checked exception is an exception that is typically a user error or a problem that cannot be foreseen by the programmer. For example, if a file is to be opened, but the file cannot be found, an exception occurs. These exceptions cannot simply be ignored at the time of compilation.
Runtime exceptions:	A runtime exception is an exception that occurs that probably could have been avoided by the programmer. As opposed to checked exceptions, runtime exceptions are ignored at the time of compilation.
Errors:	These are not exceptions at all, but problems that arise beyond the control of the user or the programmer. For example, if a stack overflow occurs, an error will arise. They are also ignored at the time of compilation.

Exception in Java



Exception in Java

Exception Handling

try block

- The first step in constructing an exception handler is to enclose the code that might throw an exception within a try block.
- try
 - {
 - *code*
 - }
 - *catch and finally blocks . . .*

Exception in Java

catch block

- You associate exception handlers with a try block by providing one or more catch blocks directly after the try block.

```
try {  
    }  
catch (ExceptionType name) {  
    }
```

Exception in Java

finally block

- The finally keyword is used to create a block of code that follows a try block.
- A finally block of code always executes, whether or not an exception has occurred.

Refer Example here-

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.7%20Exception/2.4.1%20ExceptionDemo.java>

Exception in Java

Multiple catch blocks

```
try
{ //Protected code }
catch(ExceptionType1 e1)
{ //Catch block }
catch(ExceptionType2 e2)
{ //Catch block }
catch(ExceptionType3 e3)
{ //Catch block }
```

Exception in Java

- If an exception occurs in the protected code, the exception is thrown to the first catch block in the list.
- If the data type of the exception thrown matches ExceptionType1, it gets caught there.
- If not, the exception passes down to the second catch statement.

Refer Example here-

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.7%20Exception/2.4.2%20MultiplaCatchDemo.java>

Exception in Java

throw & throws keyword

- If a method does not handle a checked exception, the method must declare it using the **throws** keyword.
- The throws keyword appears at the end of a method's signature.
- You can throw an exception, either a newly instantiated one or an exception that you just caught, by using the **throw** keyword

Exception in Java

throw	throws
throw is use to explicitly throw the exceptions.	throws is use to declare the exceptions.
throw is followed by instance	throws is followed by class
throw is used within method.	throws is used with the method signature.
You can not throw multiple exception .	you can declare multiple exceptions using throws.

Exception in Java

Custom Exception

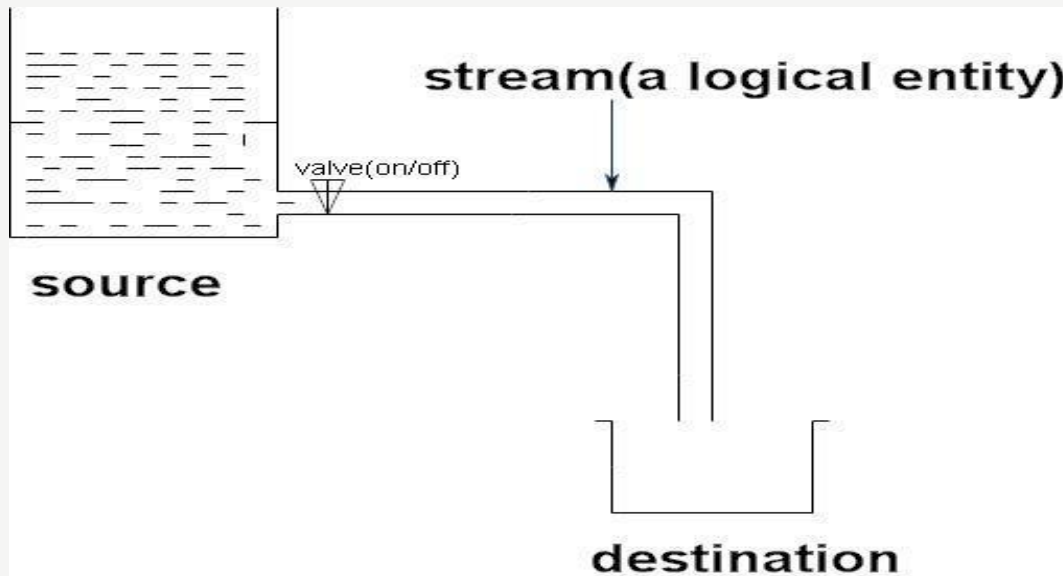
- You can create your own exceptions in Java.
- For that,
 - All exceptions must be a child of Throwable.
 - If you want to write a checked exception that is automatically enforced by the Handle or Declare Rule, you need to extend the **Exception** class.
 - If you want to write a runtime exception, you need to extend the RuntimeException class.
 - Eg class MyException extends Exception{ }

Refer Example here-

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.7%20Exception/2.4.3%20User%20Exception/2.4.3.1%20BankDemo.java>

File IO in Java

- What is Stream and Types of Stream
- File Input and Output Stream and its Methods
- File class
- Command Line Arguments.

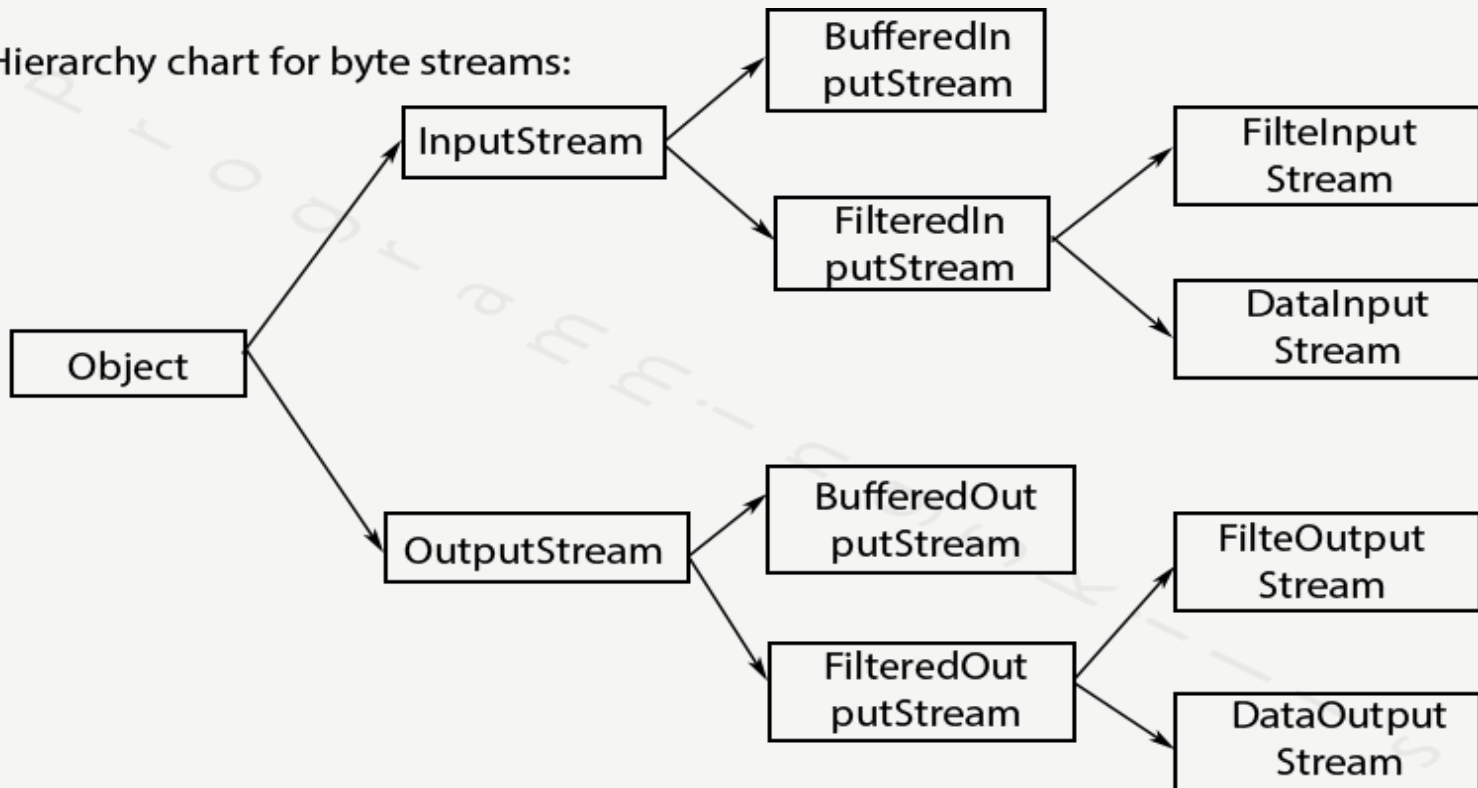


File IO in Java

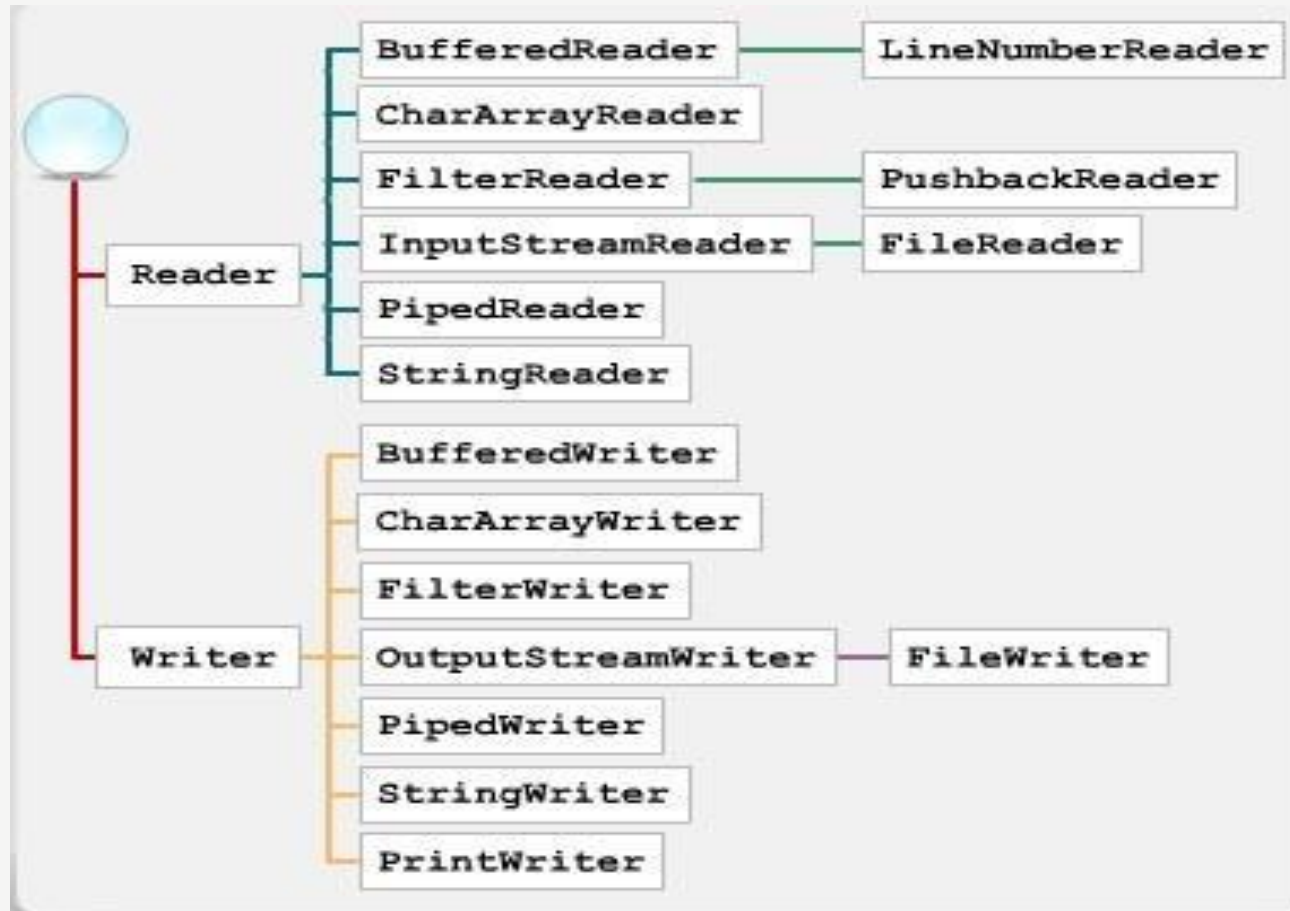
- A stream can be defined as sequence of data from one place to another.
- A stream can represent many different kinds of sources and destinations, including disk files, devices, other programs, and memory arrays.
- Streams are used read/write data from file resides on network or hard disk or other storage device.
- An input stream/Source Stream initiates the flow of data, used to read data from Source.
- An output stream/Sink Stream terminates flow of data, used for writing data to a destination.

File IO in Java

Hierarchy chart for byte streams:



File IO in Java



InputStream Methods

Method details

public int **available**() throws IOException

Returns the number of bytes that can be read (or skipped over) from this input stream without blocking by the next caller of a method for this input stream.

public void **close**() throws IOException

Closes this input stream and releases any system resources associated with the stream.

public void **mark**(int readlimit)

Marks the current position in this input stream.

public void **reset**() throws IOException

Repositions this stream to the position at the time the mark method was last called on this input stream.

InputStream Methods

Method details

public boolean **markSupported()**

Tests if this input stream supports the mark and reset methods.

public abstract int **read()** throws
IOException

Reads the next byte of data from the input stream. The value byte is returned as an int in the range 0 to 255. If no byte is available because the end of the stream has been reached, the value - 1 is returned.

InputStream Methods

Method details

public int **read**(byte[] b) throws
IOException

Reads some number of bytes from the input stream and stores them into the buffer array b.

public int **read**(byte[] b, int off, int len)
throws IOException

Reads up to len bytes of data from the input stream into an array of bytes. An attempt is made to read as many as len bytes. The number of bytes actually read is returned as an integer.

OutputStream Methods

Method details

public abstract void **write**(int b) throws
IOException

Writes the specified byte to this output stream.

public void **write**(byte[] b) throws
IOException

Writes b.length bytes from the specified byte array to this output stream.
The general contract for write(b) is that it should have exactly the same effect as the call write(b, 0, b.length).

public void **write**(byte[] b, int off, int len)
throws IOException

Writes len bytes from the specified byte array starting at offset off to this output stream.

public void **flush**() throws IOException

Flushes this output stream and forces any buffered output bytes to be written out.

InputStream & OutputStream

Refer Example here-

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.8%20File%20Handling/2.8.1%20InputOutputStreamDemo.java>

Reader & Writer

- Java IO's Reader and Writer work much like the InputStream and OutputStream with the exception that Reader and Writer are character based.
- They are intended for reading and writing text.
- The InputStream and OutputStream were byte based.
- **Reader**
- The Reader is the baseclass of all Reader's in the Java IO API. Subclasses include a BufferedReader, PushbackReader etc.

Reader & Writer

Methods	Description
abstract void close()	Closes the stream and releases any system resources associated with it.
void mark(int readAheadLimit)	Marks the present position in the stream.
boolean markSupported()	Tells whether this stream supports the mark() operation.
Int read()	Reads a single character.
Int read(char[] cbuf)	Reads characters into an array.

Reader & Writer

Methods	Description
Int read(char[] cbuf,int off,int len)	Reads characters into a portion of an array.
Int read(CharBuffer targert)	Attempts to read characters into the specified character buffer.
void reset()	Resets the stream.
long skip()	Skips characters.

Reader & Writer

Writer

- You will normally use a Writer subclass rather than a Writer directly. Subclasses of Writer include OutputStreamWriter, CharArrayWriter, FileWriter, plus many others

Methods	Description
Writer append(char c)	Appends the specified character to this writer.
Writer append(CharSequence seq)	Appends the specified character sequence to this writer.
Writer append(CharSequence seq,int start,int end)	Appends a subsequence of the specified character sequence to this writer.
void close()	Closes the stream, flushing it first.

Reader & Writer

Methods	Description
<code>void flush()</code>	Flushes the stream.
<code>void write(char[] cbuf)</code>	Writes an array of characters.
<code>void write(char[] cbuf,int off,int len)</code>	Writes a portion of an array of characters.
<code>void write(int c)</code>	Writes a single character.
<code>void write(String str)</code>	Writes a string.
<code>Void write(String str,int off,int len)</code>	Writes a portion of a string.

Refer Example here-

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.8%20File%20Handling/2.8.2%20ReaderWriterClassDemo.java>

File Class

- Java File class represents the files and directory pathnames in an abstract manner.
- This class is used for creation of files and directories, file searching, file deletion etc.
- The File object represents the actual file/directory on the disk.
- There are following constructors to create a File object:
 - File(File parent, String child);
 - File(String pathname)
 - File(String parent, String child)
 - File(URI uri)

File Class

Method details

public boolean **createNewFile()**
throws IOException

Atomically creates a new, empty file named by this abstract pathname if and only if a file with this name does not yet exist.

public boolean **delete()**

Deletes the file or directory denoted by this abstract pathname. If this pathname denotes a directory, then the directory must be empty in order to be deleted.

public String[] **list()**

Returns an array of strings naming the files and directories in the directory denoted by this abstract pathname.

public boolean **mkdir()**

Creates the directory named by this abstract pathname.

File Class

Method details

public long length()	Returns the length of the file denoted by this abstract pathname. The return value is unspecified if this pathname denotes a directory.
public boolean isFile()	Tests whether the file denoted by this abstract pathname is a normal file.
public boolean isDirectory()	Tests whether the file denoted by this abstract pathname is a directory.
public boolean exists()	Tests whether the file or directory denoted by this abstract pathname exists.
public boolean canWrite()	Tests whether the application can modify to the file denoted by this abstract pathname.
public boolean canRead()	Tests whether the application can read the file denoted by this abstract pathname.

File Class

Method details

public String getName()	Returns the name of the file or directory denoted by this abstract pathname.
public String getParent()	Returns the pathname string of this abstract pathname's parent, or null if this pathname does not name a parent directory.
public String getPath()	Converts this abstract pathname into a pathname string.

Refer Example here-

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.8%20File%20Handling/2.8.3%20FileDemo.java>

Java Serialization & Deserialization

Refer Example here-

https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.8%20File%20Handling/2.8.4%20Serialization_DeserializationDemo.java

Collection in Java

- **Collection Framework Introduction**
- **Collection API**
 - **Arraylist**
 - **Hashset**
 - **Iterator**
 - **HashMap**
 - **Vector and Enumration**
- **Generics**
 - **Genrics Example**
- **Comprator and Comprable.**
- **Vector vs Arraylist**
- **HashMap vs Hashtable**
- **Comprator vs Comprable**

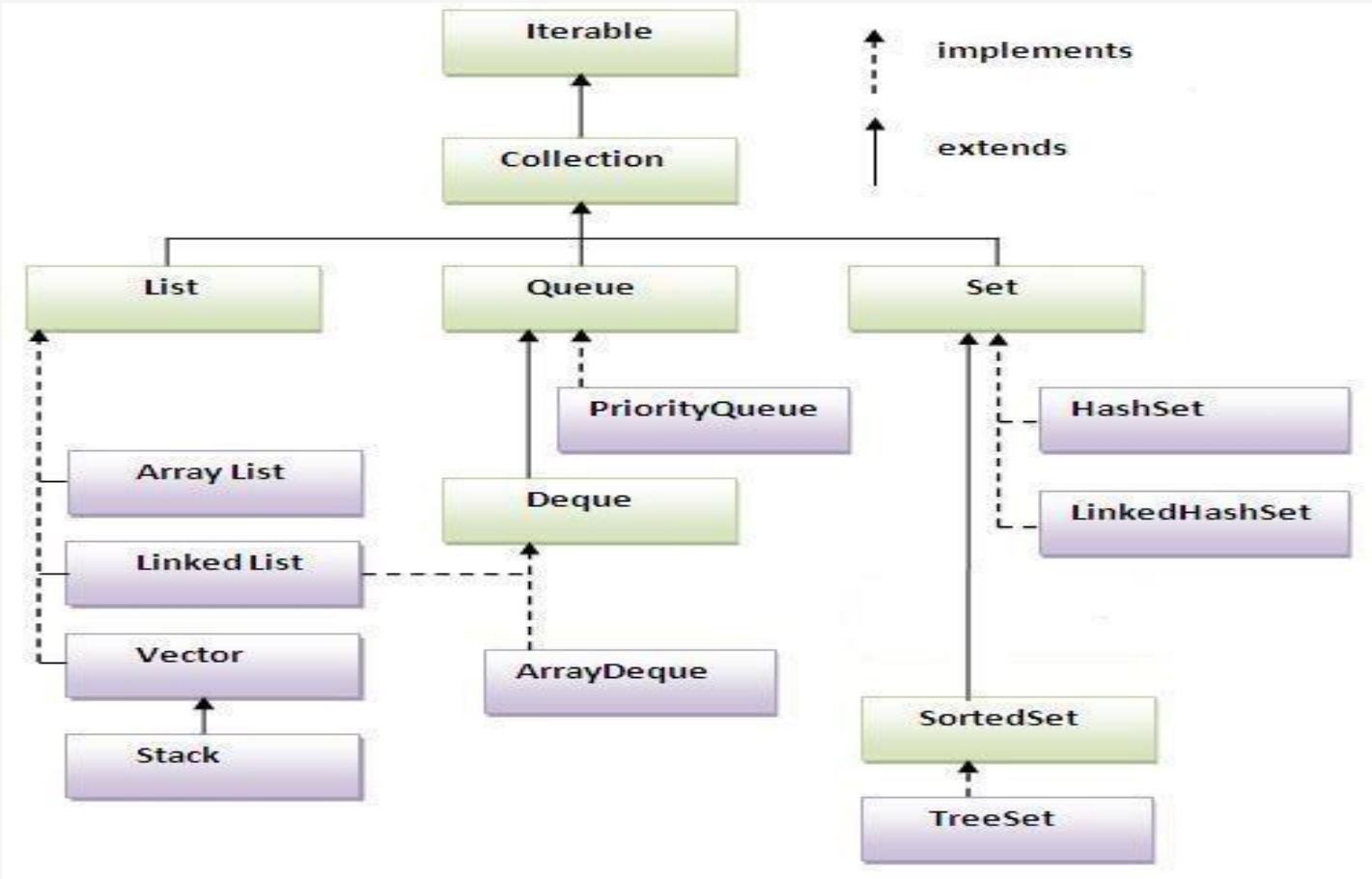
Collection in Java

- Towards this end, the entire collections framework is designed around a set of standard interfaces. Several standard implementations such as **LinkedList**, **HashSet**, and **TreeSet**, of these interfaces are provided that you may use as-is and you may also implement your own collection, if you choose.
- A collections framework is a unified architecture for representing and manipulating collections. All collections frameworks contain the following:
 - **Interfaces:** These are abstract data types that represent collections. Interfaces allow collections to be manipulated independently of the details of their representation. In object-oriented languages, interfaces generally form a hierarchy.

Collection in Java

- **Implementations, i.e., Classes:** These are the concrete implementations of the collection interfaces. In essence, they are reusable data structures.
- **Algorithms:** These are the methods that perform useful computations, such as searching and sorting, on objects that implement collection interfaces. The algorithms are said to be polymorphic: that is, the same method can be used on many different implementations of the appropriate collection interface.
- In addition to collections, the framework defines several map interfaces and classes. Maps store key/value pairs. Although maps are not *collections* in the proper use of the term, but they are fully integrated with collections.

Collection API



Collection in Java

ArrayList

- The ArrayList class extends AbstractList and implements the List interface. ArrayList supports dynamic arrays that can grow as needed.
- Standard Java arrays are of a fixed length. After arrays are created, they cannot grow or shrink, which means that you must know in advance how many elements an array will hold.
- Array lists are created with an initial size. When this size is exceeded, the collection is automatically enlarged. When objects are removed, the array may be shrunk.

Refer Example here-

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.9%20Collection/2.9.1%20ArrayListDemo.java>

Collection in Java

HashSet

- HashSet extends AbstractSet and implements the Set interface. It creates a collection that uses a hash table for storage.
- A hash table stores information by using a mechanism called hashing. In hashing, the informational content of a key is used to determine a unique value, called its hash code.
- The hash code is then used as the index at which the data associated with the key is stored. The transformation of the key into its hash code is performed automatically.

Refer Example here-

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.9%20Collection/2.9.2%20HashSetDemo.java>

Collection in Java

Iterator Interface

- Iterator interface will be use for the fetching the element from the collections.
- The easiest way to do this is to employ an iterator, which is an object that implements either the Iterator or the ListIterator interface.
- Iterator enables you to cycle through a collection, obtaining or removing elements. ListIterator extends Iterator to allow bidirectional traversal of a list, and the modification of elements.
- Before you can access a collection through an iterator, you must obtain one. Each of the collection classes provides an iterator() method that returns an iterator to the start of the collection.

Collection in Java

Iterator Interface

- By using this iterator object, you can access each element in the collection, one element at a time.
- In general, to use an iterator to cycle through the contents of a collection, follow these steps:
 - Obtain an iterator to the start of the collection by calling the collection's `iterator( )` method.
 - Set up a loop that makes a call to `hasNext( )`. Have the loop iterate as long as `hasNext( )` returns true.
 - Within the loop, obtain each element by calling `next( )`.

Collection in Java

HashMap

- The HashMap class uses a hashtable to implement the Map interface. This allows the execution time of basic operations, such as get() and put(), to remain constant even for large sets.

Refer Example here-

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.9%20Collection/2.9.3%20HashMapDemo.java>

Collection in Java

Generics

- Java **Generic** methods and generic classes enable programmers to specify, with a single method declaration, a set of related methods or, with a single class declaration, a set of related types, respectively.
- Generics also provide compile-time type safety that allows programmers to catch invalid types at compile time.
- Using Java Generic concept, we might write a generic method for sorting an array of objects, then invoke the generic method with Integer arrays, Double arrays, String arrays and so on, to sort the array elements.

Collection in Java

Generics Method

- You can write a single generic method declaration that can be called with arguments of different types. Based on the types of the arguments passed to the generic method, the compiler handles each method call appropriately. Following are the rules to define Generic Methods:
 - All generic method declarations have a type parameter section delimited by angle brackets (< and >) that precedes the method's return type (< E > in the next example).
 - Each type parameter section contains one or more type parameters separated by commas. A type parameter, also known as a type variable, is an identifier that specifies a generic type name.

Collection in Java

Generics Method

- The type parameters can be used to declare the return type and act as placeholders for the types of the arguments passed to the generic method, which are known as actual type arguments.
- A generic method's body is declared like that of any other method. Note that type parameters can represent only reference types, not primitive types (like int, double and char).

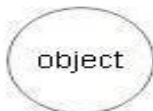
Collection in Java

Collection Framework Without Generic

```
ArrayList myDogList = new ArrayList();
```



row type conversion by compiler



Collection

Collection in Java

Comparator and Comparable

- **Comparator** is capable of comparing two different objects. The method required for implementation is *compare()*.
- **Comparable** is implemented by a class in order to be able to comparing object of itself with some other objects. The class itself must implement the interface in order to be able to compare its instance(s). The method required for implementation is *compareTo()*.

Collection in Java

Difference between comparable and comparator

Parameter	Comparable	Comparator
Sorting logic	Sorting logic must be in same class whose objects are being sorted. Hence this is called natural ordering of objects	Sorting logic is in separate class. Hence we can write different sorting based on different attributes of objects to be sorted. E.g. Sorting using id,name etc.
Implementation	Class whose objects to be sorted must implement this interface. e.g Country class needs to implement comparable to collection of country object by id	Class whose objects to be sorted do not need to implement this interface. Some other class can implement this interface. E.g.- CountrySortByIdComparator class can implement Comparator interface to sort collection of country object by id

Collection in Java

Difference between comparable and comparator

Parameter	Comparable	Comparator
Sorting method	<code>int compareTo(Object o1)</code> This method compares this object with o1 object and returns a integer. Its value has following meaning.	<code>int compare(Object o1, Object o2)</code> This method compares o1 and o2 objects. and returns a integer. Its value has following meaning.
Calling method	<code>Collections.sort(List)</code> Here objects will be sorted on the basis of <code>compareTo</code> method	<code>Collections.sort(List, Comparator)</code> Here objects will be sorted on the basis of <code>compare</code> method in Comparator
	<code>Java.lang.Comparable</code>	<code>Java.util.Comparator</code>

Comparable and Comparator

Comparable Interface:Refer Example here-

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.9%20Collection/2.9.4%20ComparableDemo.java>

Comparator Interface:Refer Example here-

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-2/2.9%20Collection/2.9.5%20ComparatorDemo.java>

Module – 3 [Web Driver]

- Web Driver Introduction
- Browser Tools
- Web Table
- Junit
- TestNG
- Working with Multiple Browser

Selenium WebDriver

- Selenium suite is comprised of 4 basic components; Selenium IDE, Selenium RC, WebDriver, Selenium Grid.
- WebDriver allows user to perform web based automation testing. WebDriver is a different tool altogether that has various advantages over Selenium RC.
- WebDriver supports a wide range of web browsers, programming languages and test environments.
- WebDriver directly communicates with the web browser and uses its native compatibility to automate.

Selenium WebDriver

- WebDriver's support doesn't only limits in the periphery of traditional user actions. Instead it supports efficient handling mechanisms for complex user actions like dealing with dropdowns, Ajax calls, switching between windows, navigation, handling alerts etc.
- WebDriver enables user to perform web based mobile testing. To support the same, WebDriver introduces AndroidDriver and IphoneDriver.
- WebDriver is faster than other tools of Selenium Suite because it makes direct calls to browser without any external intervention.

Available Drivers

- There are a number of driver classes available in WebDriver, each catering a specific web browser. Each browser has a different driver implementation in WebDriver.
- In WebDriver, a few of the browsers can be automated directly where as some of the web browsers require an external entity to be able to automate and execute the test script. This external entity is known as Driver Server. Thus, user is required to download the Driver Server for different web browsers.
- Notice that there is a separate Driver Server for each of the web browser and user cannot use one Driver Server for web browsers other than the one it is designated for.

Below is the list of available web browsers and their corresponding Server Drivers.

Web-Browser	Driver Server
Mozilla Firefox	No (No external server is required to spin the Firefox browser)
Google Chrome	Yes (ChromeDriver)
Internet Explorer	Yes (Internet Explorer Driver Server)
Opera	Yes (OperaDriver)
Safari	Yes (SafariDriver)
HTML Unit	No (No external entity is required to spin the HTML Unit)

Script Creation

For script creation, we would be using “Learning_Selenium” project created in the previous tutorial and “gmail.com” as the application under test (AUT).

Code Walkthrough

```
import org.openqa.selenium.WebDriver;
```

```
import org.openqa.selenium.firefox.FirefoxDriver;
```

```
import org.openqa.selenium.WebElement;
```

```
import org.openqa.selenium.By;
```

Object Instantiation

```
WebDriver driver = new FirefoxDriver();
```

We create a reference variable for WebDriver interface and instantiate it using FirefoxDriver class. A default Firefox profile will be launched which means that no extensions and plugins would be loaded with the Firefox instance and that it runs in the safe mode.

Launching the Web browser

```
driver.get(appUrl);
```

A *get()* method is called on the WebDriver instance to launch a fresh web browser instance. The string character sequence passed as a parameter into the *get()* method redirects the launched web browser instance to the application URL.

Maximize Browser Window

```
driver.manage().window().maximize();
```

The *maximize()* method is used to maximize the browser window soon after it is re-directed to the application URL.

Fetch the page Title

```
driver.getTitle();
```

The *getTitle()* method is used to fetch the title of the current web page. Thus, the fetched title can be loaded to a string variable.

Comparison between Expected and Actual Values:

```
if (expectedTitle.equals(actualTitle)) {
```

```
    System.out.println("Verification Successful - The correct title is displayed on the web  
    page."); }
```

```
Else
```

```
{
```

```
    System.out.println("Verification Failed - An incorrect title is displayed on the web  
    page.");}
```

The above code uses the conditional statement java constructs to compare the actual value and the expected value. Based on the result obtained, the print statement would be executed.

WebElement Instantiation

```
WebElement username = driver.findElement(By.id("Email"));
```

In the above statement, we instantiate the `WebElement` reference with the help of `"driver.findElement(By.id("Email"))"`. Thus, `username` can be used to reference the Email textbox on the user interface every time we want to perform some action on it.

Clear Command

```
username.clear();
```

The `clear()` method/command is used to clear the value present in the textbox if any. It also clears the default placeholder value.

sendKeys Command

```
username.sendKeys("TestSelenium");
```

The *sendKeys()* method/command is used to enter/type the specified value (within the parentheses) in the textbox. Notice that the *sendKeys()* method is called on the WebElement object which was instantiated with the help of element property corresponding to the UI element.

The above block of code enters the string "TestSelenium" inside the Email textbox on the Gmail application.

sendKeys is one of the most popularly used commands across the WebDriver scripts.

Click Command

SignInButton.click();

Like *sendKeys()*, *click()* is another excessively used command to interact with the web elements. *Click()* command/method is used to click on the web element present on the web page. The above block of code clicks on the “Sign in” button present on the Gmail application.

Notes:

Unlike *sendKeys()* method, *click()* methods can never be parameterized.

At times, clicking on a web element may load a new page altogether. Thus to sustain such cases, *click()* method is coded in a way to wait until the page is loaded.

Close the Web Browser

```
driver.close();
```

The close() is used to close the current browser window.

Terminate the Java Program

```
System.exit(0);
```

The Exit() method terminates the Java program forcefully. Thus, remember to close all the browser instances prior terminating the Java Program.

Refer Example here-

https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.0%20WebDriver_Sample_Program/3.0.1%20WebDriverFirstDemo.java

Test Execution

The test script or simply the java program can be executed in the following ways:

#1. Under the Eclipse's menu bar, there is an icon to execute the test script. Refer the following figure.

Locator Types

Locator Type	Syntax	Description
id	<code>driver.findElement (By.id("ID_of_Element"))</code>	Locate by value of the "id" attribute
className	<code>driver.findElement (By.className ("Class_of_Element"))</code>	Locate by value of the "class" attribute
linkText	<code>driver.findElement (By.linkText("Text"))</code>	Locate by value of the text of the hyperlink
partialLinkText	<code>driver.findElement (By.partialLinkText ("PartialText"))</code>	Locate by value of the sub-text of the hyperlink

Locator Types

Locator Type	Syntax	Description
name	<code>driver.findElement (By.name ("Name_of_Element"))</code>	Locate by value of the "name" attribute
xpath	<code>driver.findElement (By.xpath("Xpath"))</code>	Locate by value of the xpath
cssSelector	<code>driver.findElement (By.cssSelector ("CSS Selector"))</code>	Locate by value of the CSS selector
tagName	<code>driver.findElement (By.tagName("input"))</code>	Locate by value of its tag name

Some Important Commands

<code>get()</code>	It automatically opens a new browser window and fetches the page that you specify inside its parentheses.
<code>getTitle()</code>	Needs no parameters Fetches the title of the current page Leading and trailing white spaces are trimmed Returns a null string if the page has no title
<code>getPageSource()</code>	Needs no parameters Returns the source code of the page as a String value
<code>getCurrentUrl()</code>	Needs no parameters Fetches the string representing the current URL that the browser is looking at

Some Important Commands

getText()	Fetches the inner text of the element that you specify

Navigate commands

<code>navigate().to()</code>	It automatically opens a new browser window and fetches the page that you specify inside its parentheses. It does exactly the same thing as the <code>get()</code> method.
<code>navigate().refresh()</code>	Needs no parameters. It refreshes the current page.
<code>navigate().back()</code>	Needs no parameters Takes you back by one page on the browser's history.
<code>navigate().forward()</code>	Needs no parameters Takes you forward by one page on the browser's history.

Closing and Quitting Browser Windows

<code>close()</code>	Needs no parameters It closes only the browser window that WebDriver is currently controlling.
<code>quit()</code>	Needs no parameters It closes all windows that WebDriver has opened.

Selenium-WebDriver's Drivers

WebDriver is the name of the key interface against which tests should be written, but there are several implementations. These include:

HtmlUnit Driver	This is currently the fastest and most lightweight implementation of WebDriver. As the name suggests, this is based on HtmlUnit. HtmlUnit is a java based implementation of a WebBrowser without a GUI. For any language binding (other than java) the Selenium Server is required to use this driver.
	WebDriver driver = new HtmlUnitDriver();

Selenium-WebDriver's Drivers

HtmlUnit Driver	
Pros	Cons
1.Fastest implementation of WebDriver 2.A pure Java solution and so it is platform independent. 3. Supports JavaScript	Emulates other browsers' JavaScript behaviour

Selenium-WebDriver's Drivers

Firefox Driver	
	Controls the Firefox browser using a Firefox plugin. The Firefox Profile that is used is stripped down from what is installed on the machine to only include the Selenium WebDriver.xpi (plugin). A few settings are also changed by default (see the source to see which ones) Firefox Driver is capable of being run and is tested on Windows, Mac, Linux. Currently on versions 3.6, 10, latest - 1, latest
	WebDriver driver = new FirefoxDriver();

Selenium-WebDriver's Drivers

Pros	Cons
Runs in a real browser and supports JavaScript Faster than the Internet Explorer Driver	Slower than the HtmlUnit Driver

Selenium-WebDriver's Drivers

Internet Explorer Driver	
	This driver is controlled by a .dll and is thus only available on Windows OS. Each Selenium release has its core functionality tested against versions 6, 7 and 8 on XP, and 9 on Windows7.
	WebDriver driver = new InternetExplorerDriver();

Selenium-WebDriver's Drivers

Pros	Cons
Runs in a real browser and supports JavaScript with all the quirks your end users see.	Obviously the Internet Explorer Driver will only work on Windows! Comparatively slow (though still pretty snappy :) XPath is not natively supported in most versions. Sizzle is injected automatically which is significantly slower than other browsers and slower when comparing to CSS selectors in the same browser. CSS is not natively supported in versions 6 and 7. Sizzle is injected instead. CSS selectors in IE 8 and 9 are native, but those browsers don't fully support CSS3

Selenium-WebDriver's Drivers

ChromeDriver	
	ChromeDriver is maintained / supported by the Chromium project itself. WebDriver works with Chrome through the chromedriver binary (found on the chromium project's download page). You need to have both chromedriver and a version of chrome browser installed. chromedriver needs to be placed somewhere on your system's path in order for WebDriver to automatically discover it. The Chrome browser itself is discovered by chromedriver in the default installation path. These both can be overridden by environment variables. Please refer to the wiki for more information.
	<pre>WebDriver driver = new ChromeDriver();</pre>

Selenium-WebDriver's Drivers

Pros	Cons
Runs in a real browser and supports JavaScript Because Chrome is a Webkit-based browser, the ChromeDriver may allow you to verify that your site works in Safari. Note that since Chrome uses its own V8 JavaScript engine rather than Safari's Nitro engine, JavaScript execution may differ.	Slower than the HtmlUnit Driver

Accessing Forms using Selenium WebDriver

Accessing Form Elements

Input Box

Input boxes refer to either of these two types:

Text Fields- text boxes that accept typed values and show them as they are.

Password Fields- text boxes that accept typed values but mask them as a series of special characters (commonly dots and asterisks) to avoid sensitive values to be displayed.



Accessing Forms using Selenium WebDriver

Text field

User Name:	tutorial
Password:	••••••••

Password field

Accessing Forms using Selenium WebDriver

Entering Values in Input Boxes

The **sendKeys()** method is used to enter values into input boxes.

this will type the text "tutorial" onto the
element with name-"username"

```
driver.findElement(By.name("username")).sendKeys("tutorial");
```

User
Name:

Accessing Forms using Selenium WebDriver

Deleting Values in Input Boxes

The **clear()** method is used to delete the text in an input box. **This method does not need any parameter.** The code snippet below will clear out the text "tutorial" in the User Name text box.

```
driver.findElement(By.name("userName")).clear();
```

Refer Example here-

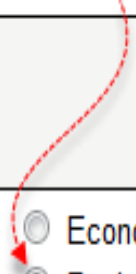
https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.0%20WebDriver_Sample_Program/3.0.2%20WebDriver_Id.java

Accessing Forms using Selenium WebDriver

Radio Button

Toggling a radio button on is done using the **click()** method.

```
driver.findElement(By.cssSelector("input[value='Business']")).click();
```



Service Class:

- ☐ Economy class
- ☒ Business class
- ☐ First class

Refer Example here-

https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.0%20WebDriver_Sample_Program/3.0.3%20WebDriver_Radio.java

Accessing Forms using Selenium WebDriver

Check Box

Toggling a check box on/off is also done using the **click()** method.

The code below will click on Facebook's "Keep me logged in" check box twice and then output the result as TRUE when it is toggled on, and FALSE if it is toggled off.

Refer Example here-

https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.0%20WebDriver_Sample_Program/3.0.4%20WebDriver_Check.java



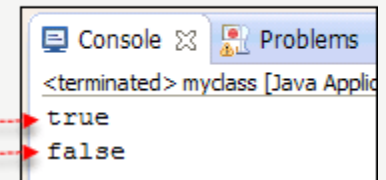
Accessing Forms using Selenium WebDriver

```
public static void main(String[] args) {  
    WebDriver driver = new FirefoxDriver();  
    String baseURL = "http://www.facebook.com";  
  
    driver.get(baseURL);  
    WebElement chkFBPersist = driver.findElement(By.id("persist_box"));  
    for(int i=0; i<2; i++){  
        chkFBPersist.click();  
        System.out.println(chkFBPersist.isSelected());  
    }  
    driver.quit();  
}
```

Email or Phone

☒ Keep me logged in

First click - checkbox
was toggled on



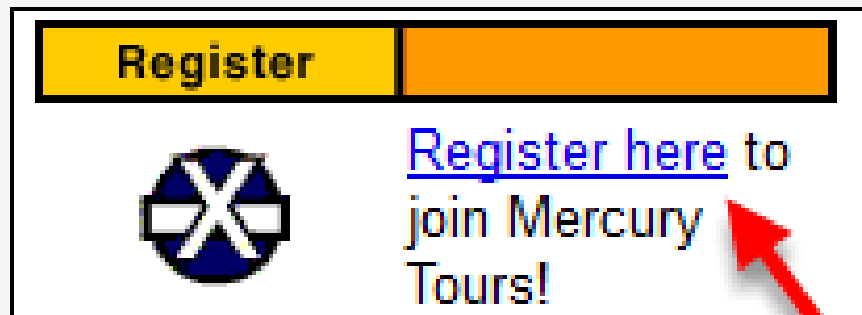
Second click - checkbox
was toggled off

Accessing Forms using Selenium WebDriver

Links

Links also are accessed by using the **click()** method.

Consider the below link found in Mercury Tours' homepage.



Accessing Forms using Selenium WebDriver

You can access this link using `linkText()` or `partialLinkText()` together with `click()`. Either of the two lines below will be able to access the "Register here" link shown above.

```
driver.findElement(By.linkText("Register here")).click();
```

```
driver.findElement(By.partialLinkText("here")).click();
```

Accessing Forms using Selenium WebDriver

Drop-Down Box

Before we can control drop-down boxes, we must do following two things :

Import the package **org.openqa.selenium.support.ui.Select**

Instantiate the drop-down box as a "Select" object in WebDriver

As an example, go to Mercury Tours' Registration page (<http://newtours.demoaut.com/mercuryregister.php>) and notice the "Country" drop-down box there.

Mercury Tours Registration Page

Country: 

```
<td>  
  <select size="1" name="country">  
</td>
```

Firebug

Accessing Forms using Selenium WebDriver

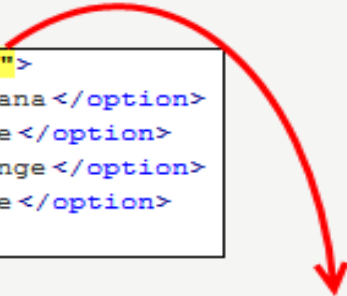
Selecting Items in a Multiple SELECT element

We can also use the **selectByVisibleText()** method in selecting multiple options in a multi SELECT element. As an example, we will take <http://jsbin.com/osebed/2> as the base URL. It contains a drop-down box that allows multiple selections at a time.

page source

```
<select id="fruits" multiple="">
  <option value="banana">Banana</option>
  <option value="apple">Apple</option>
  <option value="orange">Orange</option>
  <option value="grape">Grape</option>
</select>
```

HTML page



Banana	▲
Apple	
Orange	
Grape	▼

Accessing Forms using Selenium WebDriver

Submitting a Form

The **submit()** method is used to submit a form. This is an alternative to clicking the form's submit button. You can use submit() on any element within the form, not just on the submit button itself.

```
driver.findElement(By.name("userName")).sendKeys("tutorial");  
driver.findElement(By.name("password")).sendKeys("tutorial");  
driver.findElement(By.name("password")).submit();
```

submit() was used on the password text box instead of on the sign-in button.

Refer Example here-

https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.0%20WebDriver_Sample_Program/3.0.5%20WebDriver_Select.java

Accessing Forms using Selenium WebDriver

When `submit()` is used, WebDriver will look up the DOM to know which form the element belongs to, and then trigger its submit function.

Element	Command	Description
Input Box	<code>sendKeys()</code>	used to enter values onto text boxes
	<code>clear()</code>	used to clear text boxes of its current value
Check Box, Radio Button,	<code>click()</code>	used to toggle the element on/off

Accessing Forms using Selenium WebDriver

Drop-Down Box	<i>selectByVisibleText()/deselectByVisibleText()</i>	selects/deselects an option by its displayed text
	<i>selectByValue()/deselectByValue()</i>	selects/deselects an option by the value of its "value" attribute
	<i>selectByIndex()/deselectByIndex()</i>	selects/deselects an option by its index
	<i>isMultiple()</i>	returns TRUE if the drop-down element allows multiple selection at a time; FALSE if otherwise
	<i>deselectAll()</i>	deselects all previously selected options

Accessing Forms using Selenium WebDriver

Submit Button	<i>submit()</i>	
Links	<i>click()</i>	used to click on the link and wait for page load to complete before proceeding to the next command.

Accessing Links & Tables using Selenium Webdriver

Accessing Links

Links Matching a Criterion

Links can be accessed using an exact or partial match of their link text. The examples below provide scenarios where multiple matches would exist, and would explain how WebDriver would deal with them.

Exact Match

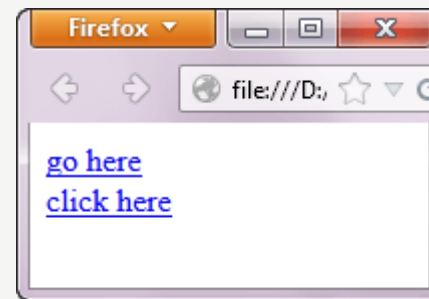
Accessing links using their exact link text is done through the `By.linkText()` method. However, if there are two links that have the very same link text, this method will only access the first one. Consider the HTML code below

Accessing Links & Tables using Selenium WebDriver

Partial Match

Accessing links using a portion of their link text is done using the **By.partialLinkText()** method. If you specify a partial link text that has multiple matches, only the first match will be accessed. Consider the HTML code below.

```
<html>
  <head>
    <title>Partial Match</title>
  </head>
  <body>
    <a href="http://www.google.com">go here</a>
    <br>
    <a href="http://www.fb.com">click here</a>
  </body>
</html>
```



Accessing Links & Tables using Selenium WebDriver

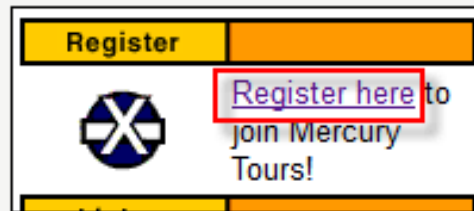
Case-sensitivity

The parameters for **By.linkText()** and **By.partialLinkText()** are both case-sensitive, meaning that capitalization matters. For example, in Mercury Tours' homepage, there are two links that contain the text "egis" - one is the "REGISTER" link found at the top menu, and the other is the "Register here" link found at the lower right portion of the page.

The link at the top menu



The link at the lower right portion of the page



Accessing Links & Tables using Selenium Webdriver

All Links

One of the common procedures in web [testing](#) is to test if all the links present within the page are working. This can be conveniently done using a combination of the **Java for-each loop** and the **By.tagName("a")** method. The WebDriver code below checks each link from the Mercury Tours homepage to determine those that are working and those that are still under construction.

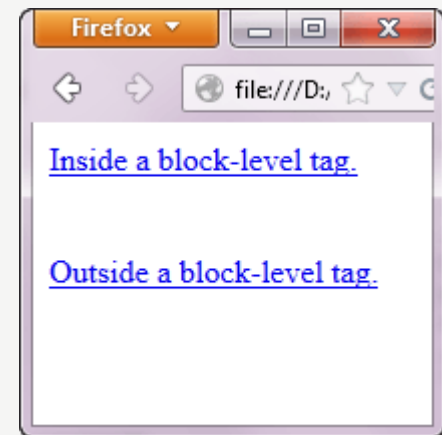
Accessing Links & Tables using Selenium Webdriver

Links Outside and Inside a Block

The latest HTML5 standard allows the `<a>` tags to be placed inside and outside of block-level tags like `<div>`, `<p>`, or `<h1>`. The `"By.linkText()"` and `"By.partialLinkText()"` methods can access a link located outside and inside these block-level elements. Consider the HTML code below.

```
<body>
  <p>
    <a href="http://www.google.com">Inside a block-level tag.</a>
  </p>

  <br>
  <a href="http://www.fb.com">
    <div>
      <span>Outside a block-level tag.</span>
    </div>
  </a>
</body>
```



Accessing Links & Tables using Selenium Webdriver

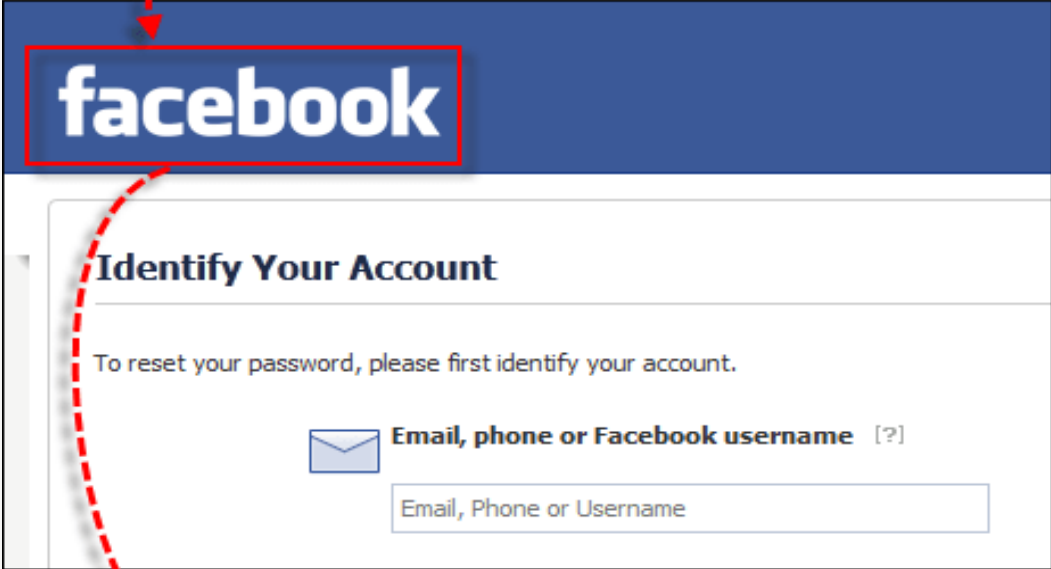
Accessing Image Links

Image links are images that act as references to other sites or sections within the same page. Since they are images, we cannot use the `By.linkText()` and `By.partialLinkText()` methods because image links basically have no link texts at all. In this case, we should resort to using either `By.cssSelector` or `By.xpath`. The first method is more preferred because of its simplicity.

In the example below, we will access the "Facebook" logo on the upper left portion of Facebook's Password Recovery page.

Accessing Links & Tables using Selenium WebDriver

this is the image link that we will access



facebook

Identify Your Account

To reset your password, please first identify your account.

Email, phone or Facebook username [?]

Email, Phone or Username

```
<a class="lfloat" title="Go to Facebook Home" href="/">
  <i class="fb_logo img sp_6jxgq1 sx_df432d">
    <u>Facebook logo</u>
  </i>
</a>
```

We will use By.cssSelector and the element's "title" attribute to access the image link. And then we will verify if we are taken to Facebook's homepage.

Accessing Links & Tables using Selenium Webdriver

Reading a Table

There are times when we need to access elements (usually texts) that are within HTML tables. However, it is very seldom for a web designer to provide an id or name attribute to a certain cell in the table. Therefore, we cannot use the usual methods such as "By.id()", "By.name()", or "By.cssSelector()". In this case, the most reliable option is to access them using the "By.xpath()" method.

Accessing Links & Tables using Selenium WebDriver

XPath Syntax

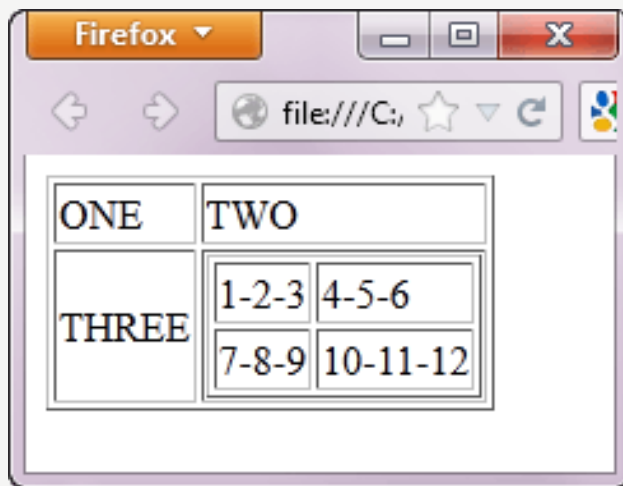
Consider the HTML code below.

```
<html>
  <head>
    <title>Sample</title>
  </head>
  <body>
    <table border="1">
      <tbody>
        <tr>
          <td>first cell</td>
          <td>second cell</td>
        </tr>
        <tr>
          <td>third cell</td>
          <td>fourth cell</td>
        </tr>
      </tbody>
    </table>
  </body>
</html>
```

Accessing Links & Tables using Selenium Webdriver

Accessing Nested Tables

The same principles discussed above applies to nested tables. **Nested tables are tables located within another table.** An example is shown below.



The screenshot shows a Firefox browser window with a file:// URL. The main content area displays a table with a nested table structure. The outer table has two columns: 'ONE' and 'TWO'. The 'TWO' column contains a nested table with two rows and two columns.

ONE	TWO				
THREE	<table border="1"><tr><td>1-2-3</td><td>4-5-6</td></tr><tr><td>7-8-9</td><td>10-11-12</td></tr></table>	1-2-3	4-5-6	7-8-9	10-11-12
1-2-3	4-5-6				
7-8-9	10-11-12				

Accessing Links & Tables using Selenium Webdriver

Using Attributes as Predicates

If the element is written deep within the HTML code such that the number to use for the predicate is very difficult to determine, we can use that element's unique attribute instead.

In the example below, the "New York to Chicago" cell is located deep into Mercury Tours homepage's HTML code.

Specials	
Atlanta to Las Vegas	\$398
Boston to San Francisco	\$513
Los Angeles to Chicago	\$168
New York to Chicago	\$198
Phoenix to San Francisco	\$213

Accessing Links & Tables using Selenium Webdriver

In this case, we can use the table's unique attribute (width="270") as the predicate. **Attributes are used as predicates by prefixing them with the @ symbol.** In the example above, the "New York to Chicago" cell is located in the first <td> of the fourth <tr>, and so our XPath should be as shown below.

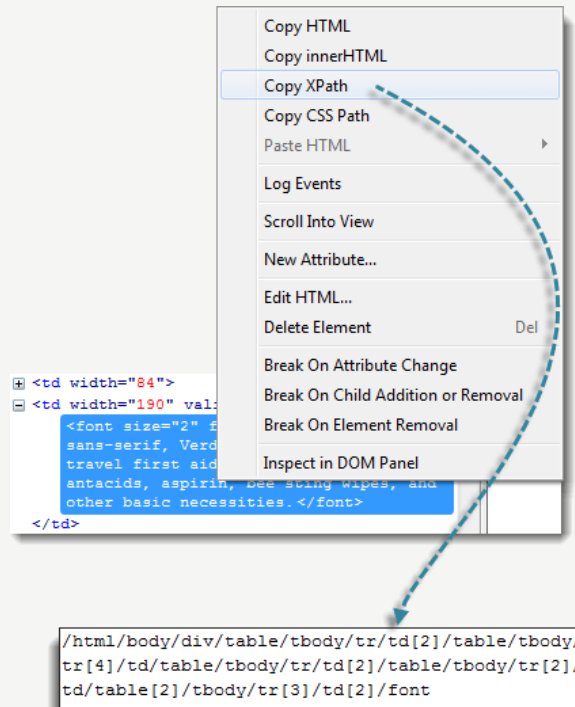
```
By.xpath("//table[@width=\"270\"]/tbody/tr[4]/td"))
```

use the escape characters here

Accessing Links & Tables using Selenium Webdriver

Shortcut: Use Firebug

If the number or attribute of an element is extremely difficult or impossible to obtain, the quickest way to generate the XPath code is thru Firebug.





Accessing Links & Tables using Selenium Webdriver

delete these

this is the first
"parent" table element

```
/html/body/div/table/tbody/tr/td[2]/table/tbody/  
tr[4]/td/table/tbody/tr/td[2]/table/tbody/tr[2]/  
td/table[2]/tbody/tr[3]/td[2]/font
```

The remaining portion of the code, trimmed
and prefixed with "//"

```
//table/tbody/tr/td[2]/table/tbody/tr[4]/td/table/  
tbody/tr/td[2]/table/tbody/tr[2]/td/table[2]/tbody/  
tr[3]/td[2]/font
```

```
By.xpath("//table/tbody/tr/td[2]"  
+ "/table/tbody/tr[4]/td"  
+ "/table/tbody/tr/td[2]"  
+ "/table/tbody/tr[2]/td"  
+ "/table[2]/tbody/tr[3]/td[2]/font"))
```

When pasted onto the By.xpath() method

How to write Dynamic XPath in Selenium

Relative XPath method

Using single attribute

```
// tagname[@attribute-name='value1'+
```

Example

```
// a *@href=http://www.google.com'+
```

```
//input*@id='name'+
```

```
//input*@name='username'+
```

```
//img*@alt='sometext'+
```


How to write Dynamic XPath in Selenium

Using multiple attribute	<pre>//tagname*@attribute1='value1'+*attribute2='value2'+ //a*@id='id1'+*@name='namevalue1'+ //img*@src='' +*@href='' +</pre>
Using contains method	<pre>Syntax //tagname*contains(@attribute,'value1')+ //input*contains(@id,'')+ //input*contains(@name,'')+ //a*contains(@href,'')+ //img*contains(@src,'')+ //div*contains(@id,'')+</pre>

How to write Dynamic XPath in Selenium

Using starts-with method	<pre>//tagname[starts-with(@attribute-name,"")+ //id[starts-with(@id,"")+ //a[starts-with(@href="")+ //img[starts-with(@src="")+ //div[starts-with(@id="")+ //input[starts-with(@id="")+ //button[starts-with(@id,"")+</pre>

How to write Dynamic XPath in Selenium

Using Following node	Xpath/following::again-ur-regular-path //input*@id=""+/following::input*1+ //a*@href=""+/following::a*1+ //img*@src=""+/following::img*1+
Using preceding node	Xpath/preceding::again-ur-regular-path //input*@id=""/ preceding::input*1+ //a[@href=""/ preceding::a*1+ //img[@src=""/ preceding::img[1]

Accessing Links & Tables using Selenium Webdriver

Refer Example here-

https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.0%20WebDriver_Sample_Program/3.0.6%20WebDriver_Table.java

Implicit Wait

Selenium WebDriver has borrowed the idea of **implicit waits** from **Watir**. This means that we can tell Selenium that we would like it to wait for a certain amount of time before throwing an **exception** that it cannot find the element on the page. We should note that implicit waits will be in place for the entire time the browser is open. This means that any search for elements on the page could take the time the implicit wait is set for.

Refer Example here-

[https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.0%20WebDriver Sample Program/3.0.7%20ImplicitWaitDemo.java](https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.0%20WebDriver%20Sample%20Program/3.0.7%20ImplicitWaitDemo.java)

Explicit Wait

It is more extendible in the means that you can set it up to wait for any condition you might like. Usually, you can use some of the prebuilt **ExpectedConditions** to wait for elements to become clickable, visible, invisible, etc.

Fluent Wait

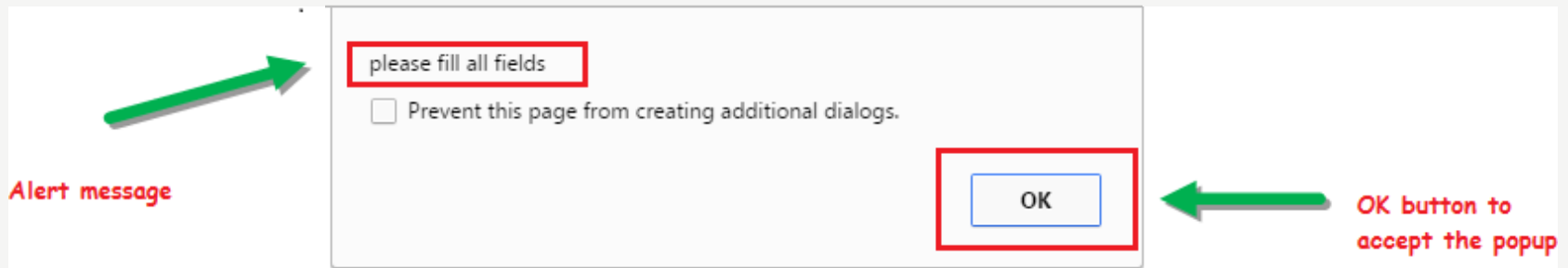
Each **FluentWait** instance defines the maximum amount of time to wait for a condition, as well as the frequency with which to check the condition. Furthermore, the user may configure the wait to ignore specific types of exceptions whilst waiting, such as **NoSuchElementExceptions** when searching for an element on the page.

Handling alerts box, Verifying Alert texts and popups

What is Alert?

Alert is a small message box which displays on-screen notification to give the user some kind of information or ask for permission to perform certain kind of operation. It may be also used for warning purpose.

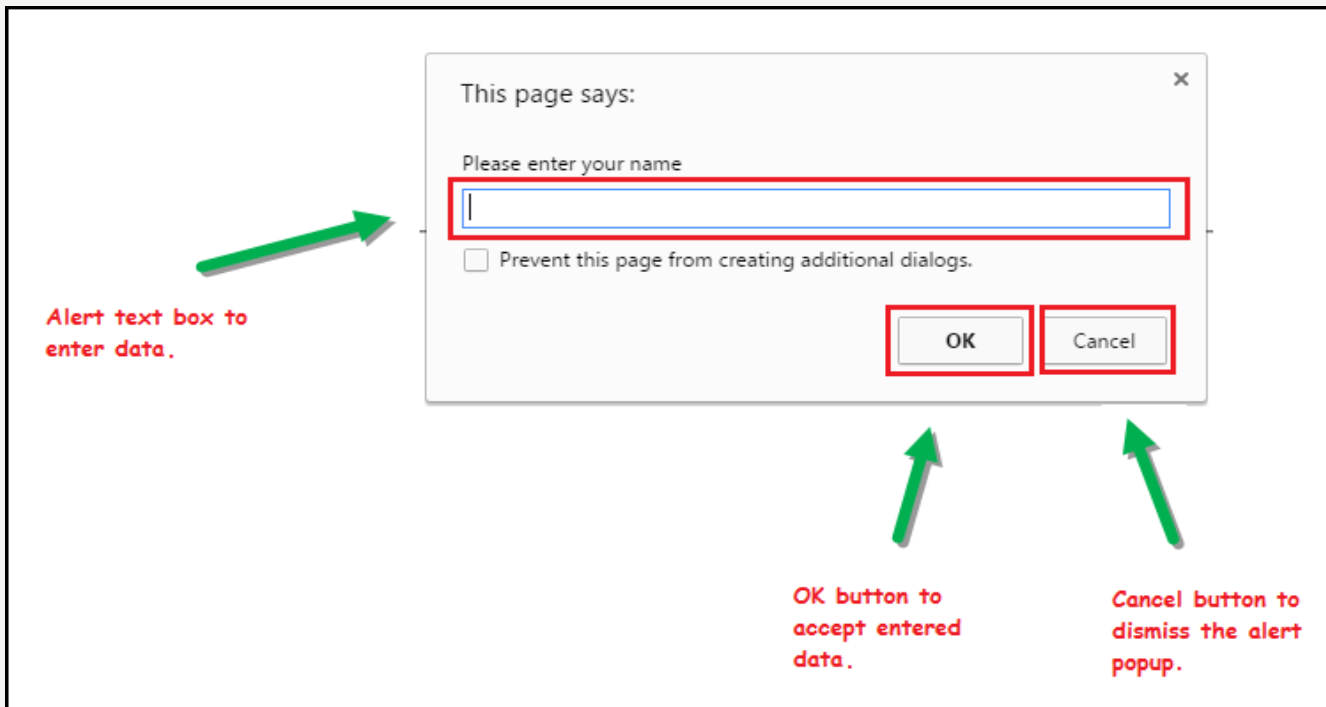
1)Simple Alert-



Handling alerts box, Verifying Alert texts and popups

2) Prompt Alert-

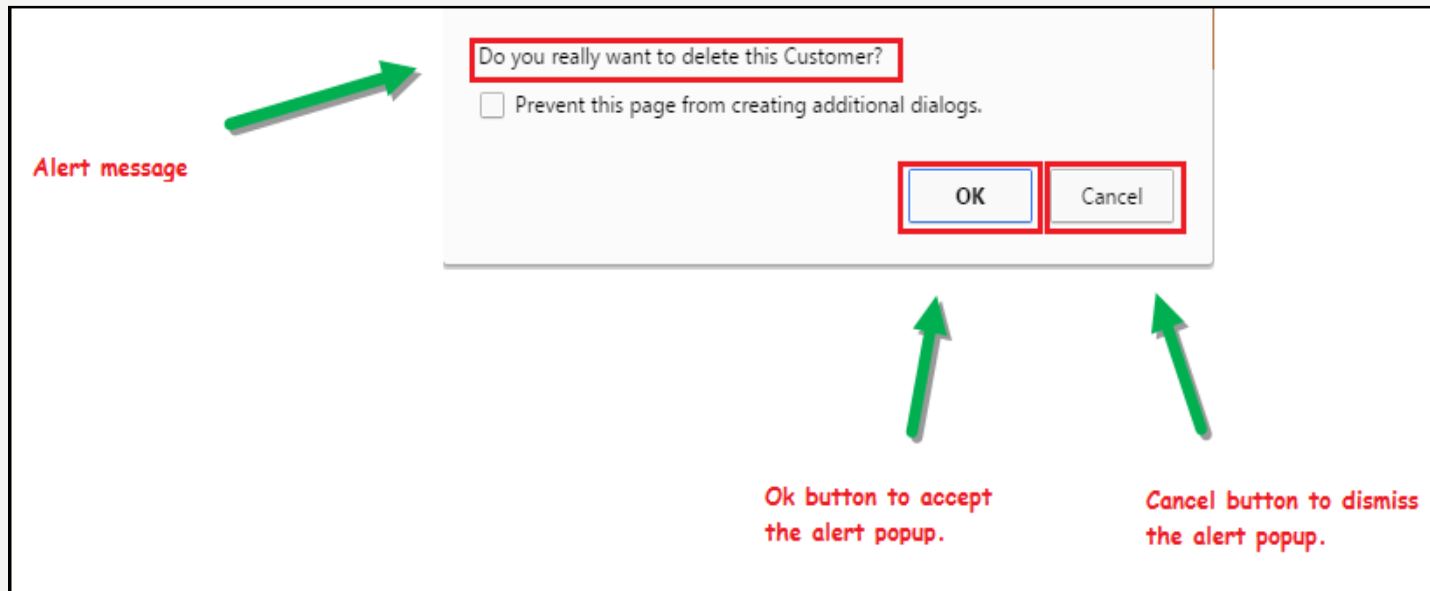
This Prompt Alert asks some input from the user and selenium webdriver can enter the text using `sendKeys("input....")`.



Handling alerts box, Verifying Alert texts and popups

3) Confirmation Alert.

This confirmation alert asks permission to do some type of operation.



Handling alerts box, Verifying Alert texts and popups

How to handle Alert in Selenium WebDriver

Alert interface provides the below few methods which are widely used in Selenium Webdriver.

1) void dismiss() // To click on the 'Cancel' button of the alert.

```
driver.switchTo().alert().dismiss();
```

2) void accept() // To click on the 'OK' button of the alert.

```
driver.switchTo().alert().accept();
```

Handling alerts box, Verifying Alert texts and popups

How to handle Alert in Selenium WebDriver

3) String getText() // To capture the alert message.

```
driver.switchTo().alert().getText();
```

4) void sendKeys(String stringToSend) // To send some data to alert box.

```
driver.switchTo().alert().sendKeys("Text");
```

You can see a number of Alert methods are displayed as shown in below screen suggested by Eclipse.

We can easily switch to alert from the main window by using Selenium's **.switchTo()** method.

Handling alerts box, Verifying Alert texts and popups

How to handle Selenium Pop-up window using Webdriver

In automation, when we have multiple windows in any web application, the activity may need to switch control among several windows from one to other in order to complete the operation. After completion of the operation, it has to return to the main window i.e. parent window.

In selenium web driver there are methods through which we can handle multiple windows.

`Driver.getWindowHandles();`

Handling Alert and Popups Example

Refer Example for Handling Alert here-

https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.0%20WebDriver_Sample_Program/3.0.10%20WebDriver_HandlingAlerts.java

Refer Example for Handling Popups here-

https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.0%20WebDriver_Sample_Program/3.0.11%20WebDriver_HandlingPopUps.java

Mouse Click & Keyboard Event: Action Class in Selenium Webdriver

Handling special keyboard and mouse events are done using the **Advanced User Interactions API**. It contains the **Actions** and the **Action** classes that are needed when executing these events.

Method	Description
<code>clickAndHold()</code>	Clicks (without releasing) at the current mouse location.
<code>contextClick()</code>	Performs a context-click at the current mouse location. (Right Click Mouse Action)
<code>doubleClick()</code>	Performs a double-click at the current mouse location.
<code>dragAndDrop(source, target)</code>	Performs click-and-hold at the location of the source element, moves to the location of the target element, then releases the mouse. Parameters: source- element to emulate button down at. target- element to move to and release the mouse at.

Mouse Click & Keyboard Event: Action Class in Selenium Webdriver

dragAndDropBy(source, x-offset, y-offset)

Performs click-and-hold at the location of the source element, moves by a given offset, then releases the mouse.

Parameters:

source- element to emulate button down at.

xOffset- horizontal move offset.

yOffset- vertical move offset.

keyDown(modifier_key)

Performs a modifier key press. Does not release the modifier key - subsequent interactions may assume it's kept pressed.

Parameters:

modifier_key - any of the modifier keys (Keys.ALT, Keys.SHIFT, or Keys.CONTROL)

keyUp(modifier_key)

Performs a key release.

Parameters:

modifier_key - any of the modifier keys (Keys.ALT, Keys.SHIFT, or Keys.CONTROL)



Mouse Click & Keyboard Event: Action Class in Selenium Webdriver

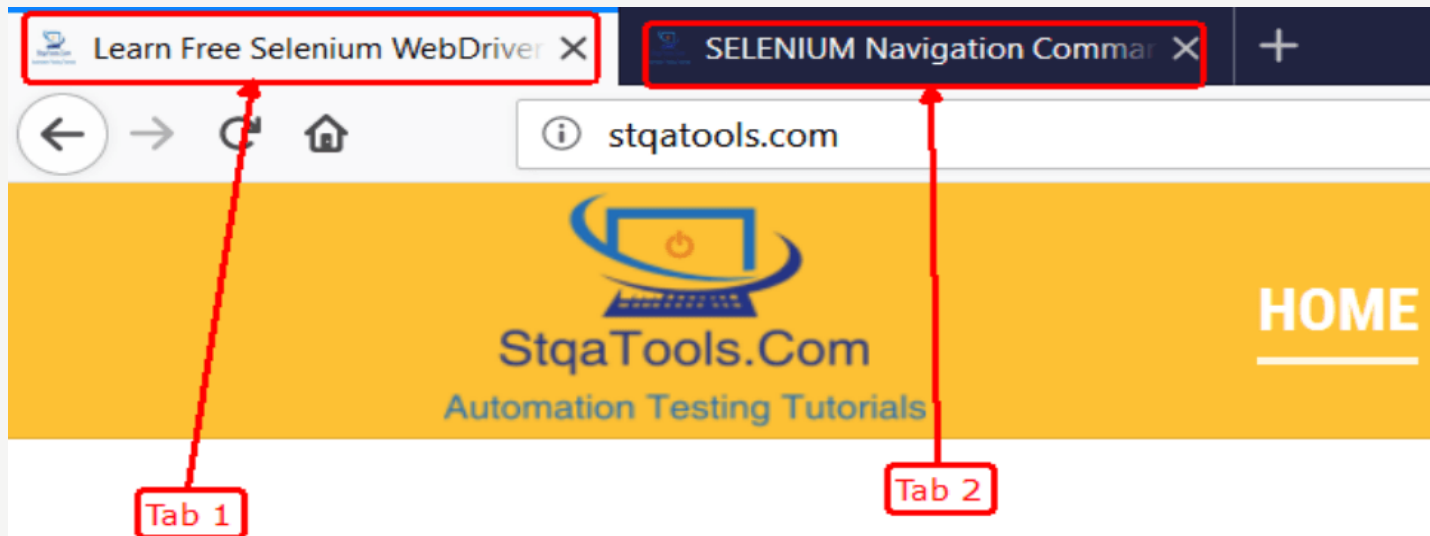
moveByOffset(x-offset, y-offset)	Moves the mouse from its current position (or 0,0) by the given offset. Parameters: x-offset- horizontal offset. A negative value means moving the mouse left. y-offset- vertical offset. A negative value means moving the mouse down.
moveToElement(toElement)	Moves the mouse to the middle of the element. Parameters: toElement- element to move to.
release()	Releases the depressed left mouse button at the current mouse location
sendKeys(onElement, charsequence)	Sends a series of keystrokes onto the element. Parameters: onElement - element that will receive the keystrokes, usually a text field charsequence - any string value representing the sequence of keystrokes to be sent

Mouse and Keyboard actions Example

https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.0%20WebDriver_Sample_Program/3.0.12%20WebDriver_KeyboardMouseAction.java

Handling Multiple Tabs in Selenium WebDriver

1. Tab is Similar to Window there is no difference with regards to Selenium.
2. New tab content is the same as automating the [window](#).
3. WebDriver Software Automation Testing Tool does not have any built-in method that we can use to open new tabs.
4. We can do the same thing to open a new tab in the Selenium WebDriver.



Handling Multiple Tabs in Selenium Webdriver Example

Ways to Window Tab Handle Using Selenium:

Switch between two tabs using switchTo()

https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.0%20WebDriver_Sample_Program/3.0.13%20WebDriver_HandlingMultipleTabs_switchTo.java

Switch between two tabs using sendKeys Actions

https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.0%20WebDriver_Sample_Program/3.0.14%20WebDriver_HandlingMultipleTabs_Actions.java

How to Identify Web Elements Using Selenium Xpath and Other Locators

What is Locator?

- Locator can be termed as an address that identifies a web element uniquely within the webpage. Locators are the HTML properties of a web element which tells the Selenium about the web element it need to perform action on.

How to Identify Web Elements Using Selenium Xpath and Other Locators

There is a diverse range of web elements. The most common amongst them are:

- Text box
- Button
- Drop Down
- Hyperlink
- Check Box
- Radio Button

How to Identify Web Elements Using Selenium Xpath and Other Locators

Types of Locators:

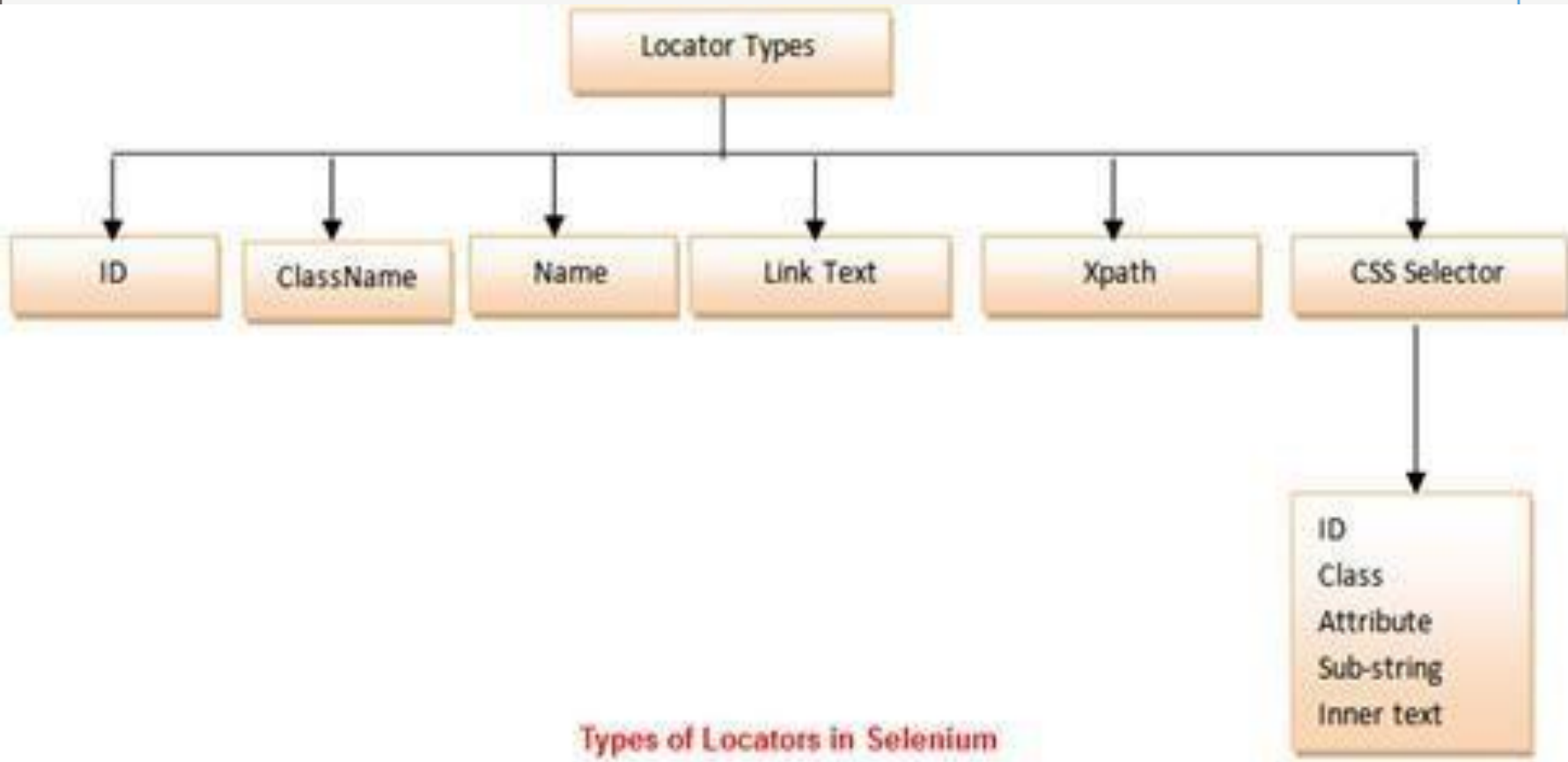
Identifying these elements has always been a very tricky subject and thus it requires an accurate and effective approach.

Thereby, we can assert that more effective the locator, more stable will be the automation script.

Essentially every Selenium command requires locators to find the web elements.

Thus, to identify these web elements accurately and precisely we have different types of locators.

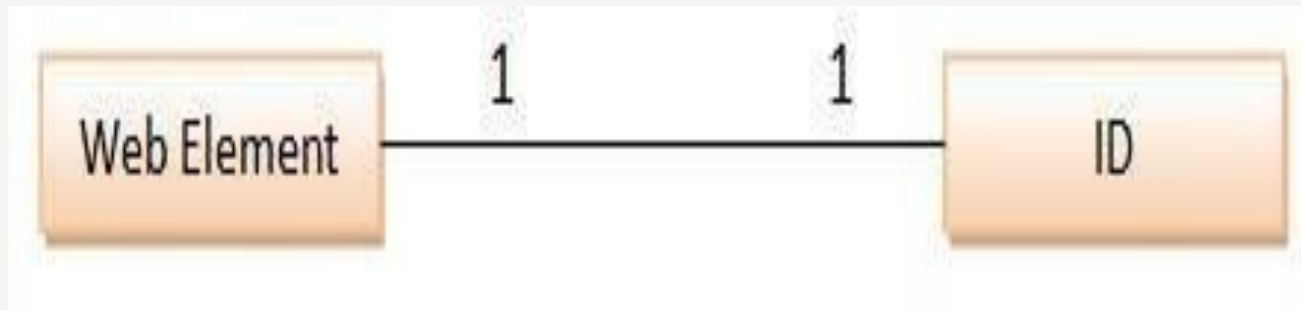
How to Identify Web Elements Using Selenium Xpath and Other Locators



How to Identify Web Elements Using Selenium Xpath and Other Locators

Using ID as a Locator

- The best and the most popular method to identify web element is to use ID. The ID of an each element is alleged to be unique.



How to Identify Web Elements Using Selenium Xpath and Other Locators

Using ClassName as a Locator

There is only a subtle difference between using ID as a locator and using classname as a locator.

In this sample, we would access “Need Help?” hyperlink present at the bottom of the login form at gmail.com.

How to Identify Web Elements Using Selenium Xpath and Other Locators

Using name as a Locator

Locating a web element using name is very much analogous to previous two locator types. The only difference lies in the syntax.

In this sample, we would access “Password” text box present in the login form at gmail.com.

Syntax: name = name of the element, In our case, the name is “Passwd”.

Verify the locator value

- **Step 1:** Type “name= Passwd” in the target box and click on the Find Button. Notice that the “Password” textbox would be highlighted.

Using Link Text as a Locator

All the hyperlinks on a web page can be indentified.

(`<a>Some Text</a>`):-

Used to create the hyperlinks on a web page and the text between opening and closing of anchor tags constitutes the link text

Using Link Text as a Locator

In this sample, we would access “Create an account” link present at the bottom of the login form at gmail.com.

Finding a link text of a web element using Firebug

- **Step 1:** Locate / inspect the web element (“Create an account” link in our case) by right clicking on the web element whose locator value we need to inspect and clicking on the option “Inspect Element with Firebug”.
- **Step 2:** Be cognizant about the text present within the `<a>` `</a>` tags and take a note of it. Hence this text will be used to identify the link on a web page uniquely.

Using Xpath as a Locator

Xpath is used to locate a web element based on its XML path. XML stands for Extensible Markup Language and is used to store, organize and transport arbitrary data. It stores data in a key-value pair which is very much similar to HTML tags. Both being mark up languages and since they fall under the same umbrella, xpath can be used to locate HTML elements.

Using Xpath as a Locator

Key Points:

The success rate of finding an element using Xpath is too high. Along with the previous statement, Xpath can find relatively all the elements within a web page. Thus, Xpaths can be used to locate elements having no id, class or name.

Creating a valid Xpath is a tricky and complex process. There are plug-ins available to generate Xpath but most of the times, the generated Xpaths fails to identify the web element correctly.

While creating xpath, user should be aware of the various nomenclatures and protocols.

Using Xpath as a Locator

Selenium Xpath Examples

Xpath Checker

Creating Xpath becomes a little simpler by using Xpath Checker. Xpath Checker is a firefox add-on to automatically generate Xpath for a web element. The add-on can be downloaded and installed like any other plug-in. The plug-in can be downloaded from “<https://addons.mozilla.org/en-US/firefox/addon/xpath-checker/>”.

As soon as the plug-in is installed, it can be seen in the context menu by right clicking any element whose Xpath we want to generate.

Locator

Note that the Xpath Checker is available for other browsers as well.

But re-iterating the fact, that most of the times, the generated Xpaths fails to identify the web element rightly. Thus, it is recommended to create our own Xpath following the pre defined rules and protocols.

In this sample, we would access “Google” image present at the top of the login form at gmail.com.

Creating a Xpath of a web element

- **Step 1:** Type “//img*@class='logo'+" i.e. the locator value in the target box within the Selenium IDE.

X-Path Example

https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.0%20WebDriver_Sample_Program/3.0.9%20WebDriver_Xpath.java

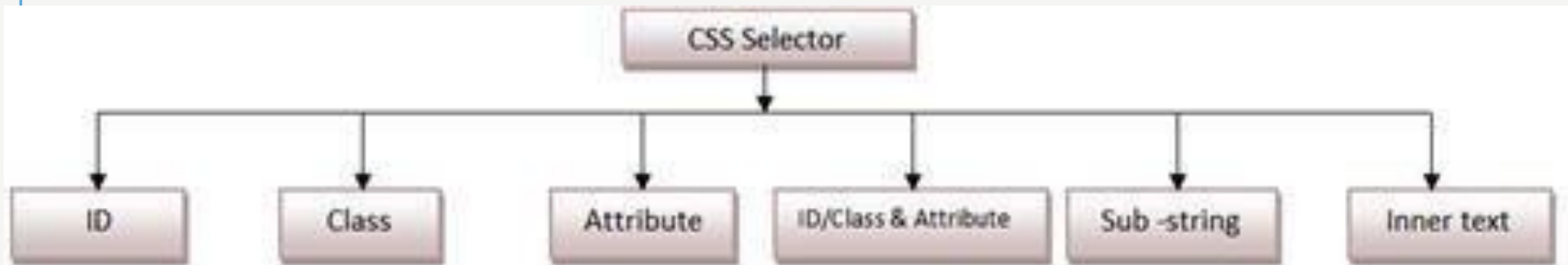
Using CSS Selector as a Locator:

CSS Selector is combination of an element selector and a selector value which identifies the web element within a web page. The composite of element selector and selector value is known as Selector Pattern.

Selector Pattern is constructed using HTML tags, attributes and their values. The central theme behind the procedure to create CSS Selector and Xpath are very much similar underlying the only difference in their construction protocol. Like Xpath, CSS selector can also locate web elements having no ID, class or Name.

So now gearing ahead, let us discuss the primitive types of CSS Selectors:

Using CSS Selector as a Locator:



CSS Selector: ID

In this sample, we would access “Email” text box present in the login form at Gmail.com.

The Email textbox has an ID attribute whose value is defined as “Email”. Thus ID attribute and its value can be used to create CSS Selector to access the email textbox.

Creating CSS Selector for web element

- **Step 1:** Locate / inspect the web element (“Email” textbox in our case) and notice that the html tag is “input” and value of ID attribute is “Email” and both of them collectively make a reference to the “Email Text box”. Hence the above data would be used to create CSS Selector.

Verify the locator value

Syntax: `css=<HTML tag><#><Value of ID attribute>`

HTML tag – It is tag which is used to denote the web element which we want to access.

– The hash sign is used to symbolize ID attribute. It is mandatory to use hash sign if ID attribute is being used to create CSS Selector.

Value of ID attribute – It is the value of an ID attribute which is being accessed. The value of ID is always preceded by a hash sign.

While specifying CSS Selector in the target text box of Selenium IDE, always remember to prefix it with “css=”. The sequence of the above artifacts is inalterable. If two or more web elements have the same HTML tag and attribute value, the first element marked in the page source will be identified.

CSS Selector: Class

We would access “Stay signed in” check box present below the login form at gmail.com.

The “Stay signed in” check box has a Class attribute whose value is defined as “remember”. Thus Class attribute and its value can be used to create CSS Selector to access the designated web element.

Locating an element using Class as a CSS Selector is very much similar to using ID, the lone difference lies in their syntax formation.

Creating CSS Selector for web element

- **Step 1:** Locate / inspect the web element (“Stay signed in” check box in our case) and notice that the html tag is “label” and value of ID attribute is “remember” and both of them collectively make a reference to the “Stay signed in check box”.

CSS Selector: Attribute

In this sample, we would access “Sign in” button present below the login form at gmail.com.

The “Sign in” button has a type attribute whose value is defined as “submit”. Thus type attribute and its value can be used to create CSS Selector to access the designated web element.

Creating CSS Selector for web element

- **Step 1:** Locate / inspect the web element (“Sign in” button in our case) and notice that the html tag is “input”, attribute is type and value of type attribute is “submit” and all of them together make a reference to the “Sign in” button.

Verify the locator value

Syntax

`css=<HTML tag><[attribute=Value of attribute]>`

Attribute – It is the attribute we want to use to create CSS Selector. It can value, type, name etc. It is recommended to choose an attribute whose value uniquely identifies the web element.

Value of attribute – It is the value of an attribute which is being accessed.

CSS Selector: ID/Class and attribute

In this sample, we would access “Password” text box present in the login form at gmail.com.

CSS Selector: Sub-string

CSS in Selenium allows matching a partial string and thus deriving a very interesting feature to create CSS Selectors using sub strings. There are three ways in which CSS Selectors can be created based on mechanism used to match the sub string.

Types of mechanisms

All the underneath mechanisms have symbolic significance.

- Match a prefix
- Match a suffix
- Match a sub string

Match a prefix

It is used to correspond to the string with the help of a matching prefix.

Syntax

`css=<HTML tag><[attribute^=prefix of the string]>`

^ – Symbolic notation to match a string using prefix.

Prefix – It is the string based on which match operation is performed. The likely string is expected to start with the specified string.

For Example: Let us consider “Password textbox”, so the corresponding CSS Selector would be:

`css=input#Passwd*name^='Pass'+`

Match a Suffix

Match a suffix

It is used to correspond to the string with the help of a matching suffix.

`css=<HTML tag><[attribute$=suffix of the string]>`

– Symbolic notation to match a string using suffix.

Suffix – It is the string based on which match operation is performed. The likely string is expected to ends with the specified string.

For Example: Lets again consider “Password textbox”, so the corresponding CSS Selector would be:

`css=input#Passwd[name$='wd'+`

Match a sub string

It is used to correspond to the string with the help of a matching sub string.

Syntax: `-css=<HTML tag><[attribute*=sub string]>`

* – Symbolic notation to match a string using sub string.

Sub string – It is the string based on which match operation is performed. The likely string is expected to have the specified string pattern.

For Example: Lets again consider “Password textbox”, so the corresponding CSS Selector would be:

`css=input#Passwd[name$='wd'+`

CSS Selector: Inner text

Inner text helps us identify and create CSS Selector using a string pattern that the HTML Tag manifests on the web page.

Consider, “Need help?” hyperlink present below the login form at gmail.com.

The anchor tag representing the hyperlink has a text enclosed within. Thus this text can be used to create CSS Selector to access the designated web element.

CSS Selector: Inner text

Syntax

css=<HTML tag><:><contains><(text)>

: – The dot sign is used to symbolize contains method

Contains – It is the value of a Class attribute which is being accessed.

Text – The text that is displayed anywhere on the web page irrespective of its location.

This is one of the most frequently used strategies to locate web element because of its simplified syntax.

Owing to the fact that creating CSS Selector and Xpath requires a lot of efforts and practice, thus the process is only exercised by more sophisticated and trained users.

CSS Selector: Inner text

Matching Text Patterns

Like locators, *patterns* are a type of parameter frequently required by Selenese commands.

Examples of commands which require are **verifyTextPresent**, **verifyTitle**, **verifyAlert**, **assertConfirmation**, and **verifyPrompt**.

And as has been mentioned above, link locators can utilize a pattern. Patterns allow you to *describe*, via the use of special characters, what text is expected rather than having to specify that text exactly.

Globbering Patterns

There are three types of patterns: *globbing*, *regular expressions*, and *exact*.

Most people are familiar with globbing as it is utilized in filename expansion at a DOS or Unix/Linux command line such as `ls *.c`. In this case, globbing is used to display all the files ending with a `.c` extension that exist in the current directory. Globbing is fairly limited. Only two special characters are supported in the Selenium implementation:

- * which translates to “match anything,” i.e., nothing, a single character, or many characters.

Globbing Patterns

[] (*character class*) which translates to “match any single character found inside the square brackets.” A dash (hyphen) can be used as a shorthand to specify a range of characters (which are contiguous in the ASCII character set). A few examples will make the functionality of a character class clear:

[aeiou] matches any lowercase vowel

[0-9] matches any digit

[a-zA-Z0-9] matches any alphanumeric character

Globbering Patterns

In most other contexts, globbing includes a third special character, the `?`. However, Selenium globbing patterns only support the asterisk and character class.

To specify a globbing pattern parameter for a Selenese command, you can prefix the pattern with a **glob:** label. However, because globbing patterns are the default, you can also omit the label and specify just the pattern itself.

Globbering Patterns

Below is an example of two commands that use globbing patterns. The actual link text on the page being tested was “Film/Television Department”; by using a pattern rather than the exact text, the **click** command will work even if the link text is changed to “Film & Television Department” or “Film and Television Department”. The glob pattern’s asterisk will match “anything or nothing” between the word “Film” and the word “Television”.

Command	Target
click	link=glob:Film*Television Department
verifyTitle	glob:*Film*Television*

Globbering Patterns

The actual title of the page reached by clicking on the link was “De Anza Film And Television Department - Menu”. By using a pattern rather than the exact text, the `verifyTitle` will pass as long as the two words “Film” and “Television” appear (in that order) anywhere in the page’s title.

For example, if the page’s owner should shorten the title to just “Film & Television Department,” the test would still pass.

Using a pattern for both a link and a simple test that the link worked (such as the `verifyTitle` above does) can greatly reduce the maintenance for such test cases.

Regular Expression Patterns

- *Regular expression* patterns are the most powerful of the three types of patterns that Selenese supports.
- Regular expressions are also supported by most high-level programming languages, many text editors, and a host of tools, including the Linux/Unix command-line utilities **grep**, **sed**, and **awk**.
- In Selenese, regular expression patterns allow a user to perform many tasks that would be very difficult otherwise.
- Whereas Selenese globbing patterns support only the ***** and **[]** (character class) features, Selenese regular expression patterns offer the same wide array of special characters that exist in JavaScript. Below are a subset of those special characters:

Exact Patterns

Exact Patterns

The **exact** type of Selenium pattern is of marginal usefulness. It uses no special characters at all. So, if you needed to look for an actual asterisk character (which is special for both globbing and regular expression patterns), the **exact** pattern would be one way to do that.

For example, if you wanted to select an item labeled “Real *” from a dropdown, the following code might work or it might not. The asterisk in the `glob:Real *` pattern will match anything or nothing. So, if there was an earlier select option labeled “Real Numbers,” it would be the option selected rather than the “Real *” option.

Exact Patterns

`select//selectglob:Real *`In order to ensure that the “Real `*`” item would be selected, the `exact:` prefix could be used to create an **exact** pattern as shown below:

`select//selectexact:Real *`But the same effect could be achieved via escaping the asterisk in a regular expression pattern:

`select//selectregexp:Real \*`It’s rather unlikely that most testers will ever need to look for an asterisk or a set of square brackets with characters inside them (the character class for globbing patterns). Thus, globbing patterns and regular expression patterns are sufficient for the vast majority of us.

CSS Selector Example

https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.0%20WebDriver_Sample_Program/3.0.8%20WebDriver_CSS.java

Commonly Used Selenium Commands

Open	opens a page using a URL.
click/clickAndWait	performs a click operation, and optionally waits for a new page to load.
verifyTitle/assertTitle	verifies an expected page title.
verifyTextPresent	verifies expected text is somewhere on the page.
verifyElementPresent	verifies an expected UI element, as defined by its HTML tag, is present on the page.
verifyText	verifies expected text and its corresponding HTML tag are present on the page.

Commonly Used Selenium Commands

verifyTable	verifies a table's expected contents.
waitForPageToLoad	pauses execution until an expected new page loads. Called automatically when clickAndWait is used.
waitForElementPresent	pauses execution until an expected UI element, as defined by its HTML tag, is present on the page.

Verifying Page Elements

Verifying UI elements on a web page is probably the most common feature of your automated tests. Selenese allows multiple ways of checking for UI elements. It is important that you understand these different methods because these methods define what you are actually testing.

Verifying Page Elements

For example, will you test that...

- an element is present somewhere on the page?
- specific text is somewhere on the page?
- specific text is at a specific location on the page?

For example, if you are testing a text heading, the text and its position at the top of the page are probably relevant for your test.

Verifying Page Elements

If, however, you are testing for the existence of an image on the home page, and the web designers frequently change the specific image file along with its position on the page, then you only want to test that *an image* (as opposed to the specific image file) exists *somewhere on the page*.

Assertion or Verification?

Choosing between “assert” and “verify” comes down to convenience and management of failures.

If you’re not on the correct page, you’ll probably want to abort your test case so that you can investigate the cause and fix the issue(s) promptly.

Verifying Page Elements

On the other hand, you may want to check many attributes of a page without aborting the test case on the first failure as this will allow you to review all failures on the page and take the appropriate action.

Effectively an “assert” will fail the test and abort the current test case, whereas a “verify” will fail the test and continue to run the test case.

Verifying Page Elements

The best use of this feature is to logically group your test commands, and start each group with an “assert” followed by one or more “verify” test commands.

An example follows:

Command	Target	Value
open	/download/	
assertTitle	Downloads	
verifyText	//h2	Downloads
assertTable	1.2.1	Selenium IDE
verifyTable	1.2.2	June 3, 2008
verifyTable	1.2.3	1.0 beta 2

Verifying Page Elements

The above example first opens a page and then “asserts” that the correct page is loaded by comparing the title with the expected value. Only if this passes will the following command run and “verify” that the text is present in the expected location.

The test case then “asserts” the first column in the second row of the first table contains the expected value, and only if this passed will the remaining cells in that row be “verified”.

Verifying Text Elements

verifyTextPresent

- The command `verifyTextPresent` is used to verify *specific text exists somewhere on the page*. It takes a single argument—the text pattern to be verified. For example:

Command	Target	Value
<code>verifyTextPresent</code>	Marketing Analysis	

Verifying Text Elements

This would cause Selenium to search for, and verify, that the text string “Marketing Analysis” appears somewhere on the page currently being tested. Use `verifyTextPresent` when you are interested in only the text itself being present on the page. Do not use this when you also need to test where the text occurs on the page.

`verifyElementPresent`

Use this command when you must test for the presence of a specific UI element, rather than its content. This verification does not check the text, only the HTML tag. One

Command	Target	Value
<code>verifyElementPresent</code>	<code>//div/p/img</code>	

Verifying Text Elements

This command verifies that an image, specified by the existence of an `<img>` HTML tag, is present on the page, and that it follows a `<div>` tag and a `<p>` tag. The first (and only) parameter is a *locator* for telling the Selenese command how to find the element. Locators are explained in the next section.

`verifyElementPresent` can be used to check the existence of any HTML tag within the page. You can check the existence of links, paragraphs, divisions `<div>`, etc. Here are a few more examples.

Verifying Text Elements

Command	Target	Value
verifyElementPresent	//div/p	
verifyElementPresent	//div/a	
verifyElementPresent	id=Login	
verifyElementPresent	link=Go to Marketing Research	
verifyElementPresent	//a[2]	
verifyElementPresent	//head/title	

Verifying Text Elements

These examples illustrate the variety of ways a UI element may be tested. In addition, locators are explained in the next section.

verifyText

Use `verifyText` when both the text and its UI element must be tested. `verifyText` must use a locator. If you choose an *XPath* or *DOM* locator, you can verify that specific text appears at a specific location on the page relative to other UI components on the page.

Command	Target	Value
<code>verifyText</code>	<code>//table/tr/td/div/p</code>	This is my text and it occurs right after the div inside the table.

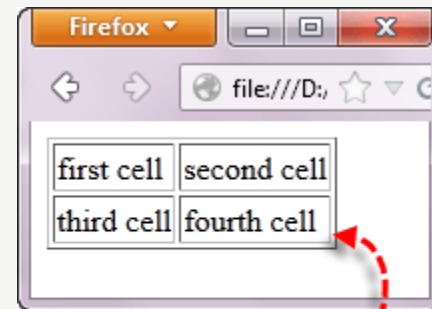
Handling Web Table in Selenium WebDriver

How to write XPath for Table

Consider the HTML code below.

```
<html>
  <head>
    <title>Sample</title>
  </head>
  <body>
    <table border="1">
      <tbody>
        <tr>
          <td>first cell</td>
          <td>second cell</td>
        </tr>
        <tr>
          <td>third cell</td>
          <td>fourth cell</td>
        </tr>
      </tbody>
    </table>
  </body>
</html>
```

We will use [XPath](#) to get the inner text of the cell containing the text "fourth cell."

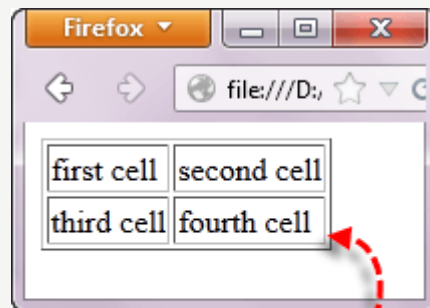


we will try to
access this cell

Handling Web Table in Selenium WebDriver

How to write XPath for Table

We will use [XPath](#) to get the inner text of the cell containing the text "fourth cell."



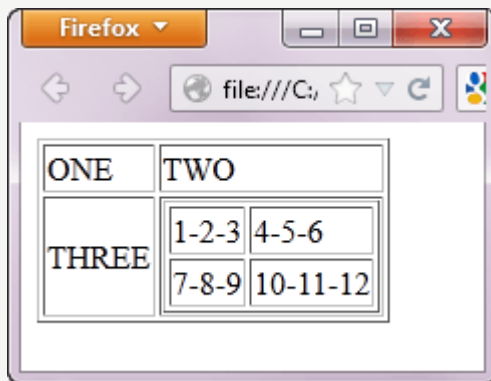
*we will try to
access this cell*

```
public static void main(String[] args) {  
    String baseUrl = "file:///C:/newhtml.html";  
    WebDriver driver = new FirefoxDriver();  
  
    driver.get(baseUrl);  
    String innerText = driver.findElement(  
        By.xpath("//table/tbody/tr[2]/td[2]")).getText();  
    System.out.println(innerText);  
    driver.quit();  
}
```


Handling Web Table in Selenium WebDriver

How to write XPath for Nested Tables-

The same principles discussed above applies to nested tables. Nested tables are tables located within another table. An example is shown here.



ONE	TWO				
THREE	<table border="1"> <tr> <td>1-2-3</td><td>4-5-6</td></tr> <tr> <td>7-8-9</td><td>10-11-12</td></tr> </table>	1-2-3	4-5-6	7-8-9	10-11-12
1-2-3	4-5-6				
7-8-9	10-11-12				

```
<html>
  <head>
    <title>Sample</title>
  </head>
  <body>
    <!--outer table-->
    <table border="1">
      <tbody>
        <tr>
          <td>ONE</td>
          <td>TWO</td>
        </tr>
        <tr>
          <td>THREE</td>
          <td>
            <!--inner table-->
            <table border="1">
              <tbody>
                <tr>
                  <td>1-2-3</td>
                  <td>4-5-6</td>
                </tr>
                <tr>
                  <td>7-8-9</td>
                  <td>10-11-12</td>
                </tr>
              </tbody>
            </table>
          </td>
        </tr>
      </tbody>
    </table>
  </body>
</html>
```

Handling Web Table in Selenium WebDriver

How to write XPath for Nested Tables

To access the cell with the text "4-5-6" using the "//parent/child" and predicate concepts from the previous section, we should be able to come up with the XPath code below.

The WebDriver code below should be able to retrieve the inner text of the cell which we are accessing.

The outer table

The inner table

`//table/tbody/tr[2]/td[2]/table/tbody/tr/td[2]`

```
public static void main(String[] args) {  
    String baseUrl = "file:///C:/newhtml.html";  
    WebDriver driver = new FirefoxDriver();  
  
    driver.get(baseUrl);  
    String innerText = driver.findElement(By  
        .xpath("//table/tbody/tr[2]/td[2]/table/tbody/tr/td[2]"))  
        .getText();  
    System.out.println(innerText);  
    driver.quit();  
}
```

Handling Web Table in Selenium WebDriver Example

https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.0%20WebDriver_Sample_Program/3.0.15%20Webdriver_WebTable.java

JUNIT

- JUnit is a unit testing framework for the Java programming language.
- JUnit is an open source framework which is used for writing & running tests.
- JUnit promotes the idea of "testing first" rather than implementing first.
- JUnit provides Annotation to identify the test methods and utility methods to assert expected results.
- JUnit has Test runners for running tests.
- JUnit tests can be organized into test suites which contains test cases and other test suites.
- JUnit Framework can be easily integrated with either of the followings:
 - Eclipse
 - Ant
 - Maven

JUnit Features

JUnit test framework provides following important features

- Fixtures
- Test suites
- Test runners
- JUnit classes

JUnit Set up

Refer this Link :

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.1%20JUnit/3.1.1%20JUnit%20SetUp/3.1.1%20JUnit%20SetUp.docx>

JUnit Assert

public class Assert **extends** java.lang.Object

This class provides a set of assertion methods useful for writing tests.

Only failed assertions are recorded.

JUnit Assert class Methods

`void assertEquals(expectedArray, resultArray);`
test whether two arrays are equal to each other.

`void assertEquals(boolean expected, boolean actual)`
Check that two primitives/Objects are equal

`void assertFalse(boolean condition)`
Check that a condition is false

`void assertNotNull(Object object)`
Check that an object isn't null.

`void assertNotSame(boolean condition)`
tests if two object references not point to the same object

JUnit Assert class Methods

`void assertNull(Object object)`

Check that an object is null

`void assertEquals(boolean condition)`

tests if two object references point to the same object

`void assertTrue(boolean condition)`

Check that a condition is true.

`void fail()`

Fails a test with no message.

Link for Example

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.1%20JUnit/3.1.2%20JUnit%20Assert%20Class/3.1.2.1%20JUnit%20Assert%20Class.java>

JUnit TestCase Methods

public abstract class TestCase extends Assert implements Test

int countTestCases()

Counts the number of test cases.

TestResult createResult()

Creates a default TestResult object.

String getName()

Gets the name of a TestCase.

TestResult run()

A convenience method to run this test.

JUnit TestCase Methods

`void run(TestResult result)`

Runs the test case and collects the results in TestResult.

`void setName(String name)`

Sets the name of a TestCase.

`void setUp()`

Sets up the fixture, for example, open a database connection.

`void tearDown()`

Tears down the fixture, for example, close a database connection.

`String toString()`

Returns a string representation of the test case.

Link for Examples of Methods

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.1%20JUnit/3.1.3%20JUnit%20Test%20Cases/3.1.3%20JUnit%20Test%20Cases.java>

TestResult Class

public class TestResult **extends** Object

Follow Link-

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.1%20JUnit/3.1.4%20JUnit%20Test%20Result/3.1.4.1%20JUnit%20Test%20Result.java>

TestSuite Class

A TestSuite is a Composite of Tests. It runs a collection of test cases.

Links for Examples

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.1%20JUnit/3.1.5%20JUnit%20Test%20Suite%20class/3.1.5.1%20Test%20Suite%20class.java>

JUnit Annotation

Annotations are meta-tags we can use to mark methods or classes.

We can annotations from JUnit to mark methods. For example, we can use `@Before` to mark a method to run before the test cases.

Annotation	Description
<code>@Test</code>	marks the public void method as a test case.
<code>@Before</code>	Annotating a method with <code>@Before</code> causes that method to be run before each Test method.
<code>@After</code>	Annotating a method with <code>@After</code> causes that method to be run after the Test method.

Annotation

Annotation	Description
@BeforeClass	Annotating a method with @BeforeClass causes it to be run once before any of the test methods in the class.
@AfterClass	call the method after all tests have finished.
@Ignore	marks to ignore the test and that test will not be executed.

Links for Examples

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.1%20JUnit/3.1.6%20JUnit%20Annotations/3.1.6.1%20JUnit%20Annotations.java>

JUnit Exceptions Test

JUnit can trace the Exception.

We can test whether the code throws desired exception or not.

The expected parameter is used along with @Test annotation.

Links for Examples

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.1%20JUnit/3.1.7%20JUnit%20Exceptions%20Test/3.1.7.1%20JUnit%20Exceptions%20Test.java>

JUnit Parameterized Test

unit 4 has introduced a new feature called Parameterized tests.

We can use Parameterized tests to run the same test using different values.

Links for Examples

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.1%20JUnit/3.1.8%20JUnit%20Parametrized%20Test/3.1.8.1%20JUnit%20Parametrized%20Test.java>

TestNG(Testing of Next generation)

Introduction

- TestNG is a testing framework inspired from JUnit and NUnit. But introducing some new functionality that make it more powerful and easier to use.
- It is an open source automated testing framework; where NG of TestNG means Next Generation. TestNG is similar to JUnit but it is much more powerful than JUnit but still it's inspired by JUnit. It is designed to be better than JUnit, especially when testing integrated classes. Pay special thanks to Cedric Beust who is the creator of TestNG.
- This eliminates most of the limitations of the older framework. It provides developer the ability to write more flexible and powerful tests with help of easy annotations, grouping, sequencing & parametrizing.

Advantages of TestNG over JUnit

- Annotations are easier to understand
- Test cases can be grouped more easily
- Parallel testing is possible
- **Annotations in TestNG are lines of code that can control how the method below them will be executed.**
- They are always preceded by the @ symbol. A very early and quick example is the one shown below.

These are 2 examples of annotations

```
@Test(priority = 0)
public void goToHomepage() {
    driver.get(baseUrl);
    Assert.assertEquals(driver.getTitle(), "Welcome: Mercury Tours");
}

@Test(priority = 1)
public void logout() {
    driver.findElement(By.LinkText("SIGN-OFF")).click();
    Assert.assertEquals("Sign-on: Mercury Tours", driver.getTitle());
}
```

The example above simply says that the method `goToHomepage()` should be executed first before `logout()` because it has a lower priority number

TestNG Test Suite

- In TestNG framework, we need to create **testng.xml** file to create and handle multiple test classes.
- This is the xml file where you will configure your test run, set test dependency, include or exclude any test, method, class or package and set priority etc.
- Implement Excel with TestNG Data Provider.

TestNG Annotations

- In the TestNG Introduction chapter we have come across different annotations used in TestNG Framework but so far we have used just three (Before, After & Test).
- All though these are the most frequently used annotations but who know how far you will go with your framework and may like to use other useful TestNG annotations.

TestNG Annotations

Sr.No.	Annotation & Description
1	@BeforeSuite The annotated method will be run only once before all tests in this suite have run.
2	@AfterSuite The annotated method will be run only once after all tests in this suite have run.
3	@BeforeClass The annotated method will be run only once before the first test method in the current class is invoked.
4	@AfterClass The annotated method will be run only once after all the test methods in the current class have run.
5	@BeforeTest The annotated method will be run before any test method belonging to the classes inside the <test> tag is run.

TestNG Annotations

Sr.No.	Annotation & Description
6	@AfterTest The annotated method will be run after all the test methods belonging to the classes inside the <test> tag have run.
7	@BeforeGroups The list of groups that this configuration method will run before. This method is guaranteed to run shortly before the first test method that belongs to any of these groups is invoked.
8	@AfterGroups The list of groups that this configuration method will run after. This method is guaranteed to run shortly after the last test method that belongs to any of these groups is invoked.
9	@BeforeMethod The annotated method will be run before each test method.
10	@AfterMethod The annotated method will be run after each test method.

TestNG Annotations

No.	Annotation & Description
11	@DataProvider Marks a method as supplying data for a test method. The annotated method must return an Object[][], where each Object[] can be assigned the parameter list of the test method. The @Test method that wants to receive data from this DataProvider needs to use a dataProvider name equals to the name of this annotation.
12	@Factory Marks a method as a factory that returns objects that will be used by TestNG as Test classes. The method must return Object[].
13	@Listeners Defines listeners on a test class.
14	@Parameters Describes how to pass parameters to a @Test method.
15	@Test Marks a class or a method as a part of the test.

TestNG Annotations

Benefits of Using Annotations-

- TestNG identifies the methods it is interested in, by looking up annotations. Hence, method names are not restricted to any pattern or format.
- We can pass additional parameters to annotations.
- Annotations are strongly typed, so the compiler will flag any mistakes right away.
- Test classes no longer need to extend anything (such as TestCase, for JUnit 3).

Link for Example

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.2%20TestNG/3.2.4%20TestNG%20Annotations/3.2.4.1%20TestNG%20Annotations.java>

TestNG - Ignore a Test

- Sometimes, it happens that our code is not ready and the test case written to test that method/code fails.
- In such cases, annotation **@Test(enabled = false)** helps to disable this test case.
- If a test method is annotated with *@Test(enabled = false)*, then the test case that is not ready to test is bypassed.
- Now, let's see @Test(enabled = false) in action.

For Example refer this link-

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.2%20TestNG/3.2.5%20TestNG%20Ignore%20a%20Test/3.2.5.1TestNG%20Ignore%20a%20Test.java>

TestNG - Dependency Test

- Sometimes, you may need to invoke methods in a test case in a particular order, or you may want to share some data and state between methods.
- This kind of dependency is supported by TestNG, as it supports the declaration of explicit dependencies between test methods.
- TestNG allows you to specify dependencies either with –
 - Using attribute *dependsOnMethods* in @Test annotations, OR.
 - Using attribute *dependsOnGroups* in @Test annotations.

For Example refer link-

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.2%20TestNG/3.2.6.1%20TestNG%20Dependancy%20Test.java>

TestNG – Parameterized Test

- Another interesting feature available in TestNG is **parametric testing**.
- In most cases, you'll come across a scenario where the business logic requires a hugely varying number of tests.
- **Parameterized tests** allow developers to run the same test over and over again using different values.
- TestNG lets you pass parameters directly to your test methods in two different ways –
 - With testng.xml
 - With Data Providers

For Example Refer link-

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.2%20TestNG/3.2.7%20TestNG%20Parameterized%20Test/3.2.7.1%20TestNG%20Parameterized%20Test.java>

TestNG Prioritizing & Sequencing

- There will be situations when you want to put number of tests under a single test class and like to run all in single shot.
- With the help of TestNG '@Test' annotations we can execute multiple tests in single TestNG file.

Link for Example

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.2%20TestNG/3.2.8%20TestNG%20Priortizing%20and%20Sequencing/3.2.8.1%20TestNG%20Priortizing%20and%20Sequencing.java>

TestNG Data Providers

- When you need to pass complex parameters or parameters that need to be created from Java (complex objects, objects read from a property file or a database, etc...), in such cases parameters can be passed using Dataproviders.
- A Data Provider is a method annotated with `@DataProvider`.
- A Data Provider returns an array of objects.

How to do it...

Link

Follow this steps to do

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.2%20TestNG/3.2.9%20TestNG%20Data%20Provider/3.2.9.1%20TestNG%20Data%20Provider.java>

TestNG Data Providers

Example Link

https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.2%20TestNG/3.2.9%20TestNG%20Data%20Provider/3.2.9.2%20TestNG_Dataprovider_Example/test-output/index.html

Working with Multiple Browsers(Cross browser Testing)

Cross Browser Testing is a type of functional test to check that your web application works as expected in different browsers.



Working with Multiple Browsers(Cross browser Testing)

Why do we need Cross Browser Testing?

A web application can be opened in any browser by the end user. For example, some people prefer to open <http://twitter.com> in **Firefox browser**, while other's can be using **Chrome browser** or **IE**.

In the diagram below you can observe that in **IE**, the login box of Twitter is not showing curve at all corners, but we are able to see it in Chrome browser.



Working with Multiple Browsers(Cross browser Testing)

So we need to ensure that the web application will work as expected in all popular browsers so that more people can access it and use it.

This motive can be fulfilled with Cross Browser Testing of the product.

Reason Cross Browser Issues

1. Font size mismatch in different browsers.
2. JavaScript implementation can be different.
3. CSS,HTML validation difference can be there.
4. Some browser still not supporting HTML5.
5. Page alignment and div size.
6. Image orientation.
7. Browser incompatibility with OS. Etc.

Working with Multiple Browsers(Cross browser Testing)

How to perform Cross Browser Testing

To execute test cases with different browsers in the same machine at the same time we can integrate Testng framework with Selenium WebDriver.

Your **testing.xml** will look like that,

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
<suite name="TestSuite" thread-count="2" parallel="tests">
  <test name="ChromeTest">
    <parameter name="browser" value="Chrome" />
    <classes>
      <class name="parallelTest.CrossBrowserScript" />
    </classes>
  </test>
  <test name="FirefoxTest">
    <parameter name="browser" value="Firefox" />
    <classes>
      <class name="parallelTest.CrossBrowserScript" />
    </classes>
  </test>
</suite>
```

Test will run parallel

1st Test name is 'ChromeTest'

Parameter is 'Chrome'

2st Test name is 'FirefoxTest'

Parameter is 'Firefox'

This is the TestNg.xml for cross Browser Testing

Working with Multiple Browsers(Cross browser Testing)

This testing.xml will map with the [Test Case](#) which will look like that

Parameter will be pass from testNG.xml

```
@BeforeTest
@Parameters("browser")
public void setup(String browser) throws Exception{
    if(browser.equalsIgnoreCase("firefox")){
        driver = new FirefoxDriver();
    }
    else if(browser.equalsIgnoreCase("chrome")){
        System.setProperty("webdriver.chrome.driver",".\\chromedriver.exe");
        driver = new ChromeDriver();
    }
}
```

check parameter value and create WebDriver according it

Here because the testing.xml has two Test tags ('ChromeTest','FirefoxTest'),this test case will execute two times for 2 different browsers.

First Test 'ChromeTest' will pass the value of parameter 'browser' as 'chrome' so ChromeDriver will be executed. This test case will run on Chrome browser.

Second Test 'FirefoxTest' will pass the value of parameter 'browser' as 'Firefox' so FirefoxDriver will be executed. This test case will run on FireFox browser.

Working with Multiple Browsers(Cross browser Testing)

Example Link

https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.2%20TestNG/3.2.10%20CrossBrowserTesting_MultipleBrowserDemo.java

Testng.xml file Link-

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-3/3.2%20TestNG/3.2.10.1%20testng.xml>

NOTE: To run the test, Right click on the **testing.xml**, Select Run As, and Click TestNG

Module -4 [Automation Frameworks]

- What is Framework
- Types of Framework
- Data Driven Framework
- Modular Driven Framework
- Keyword driven Framework
- Hybrid Framework

Apache POI (Excel)

- Most commercial automated software tools on the market support some sort of data driven testing, which allows you to automatically run a test case multiple times with different input and validation values.
- As Selenium Webdriver is more an automated testing framework than a ready-to-use tool, you will have to put in some effort to support data driven testing in your automated tests.
- Use Microsoft Excel as the format for storing parameters.
- An additional advantage of using Excel is that you can easily outsource the test data administration to someone other than yourself, someone who might have better knowledge of the test cases that need to be run and the parameters required to execute them.

Overview of Testing framework

What is Testing Framework:

- A testing framework or more specifically a testing automation framework is an execution environment for automated tests. It is the overall system in which the tests will be automated.
- It is defined as the set of assumptions, concepts, and practices that constitute a work platform or support for automated testing.

Testing framework is responsible for

- Defining the format in which to express expectations.
- Creating a mechanism to hook into or drive the application under test
- Executing the tests
- Reporting results

Properties of a testing framework:

- It is application independent.
- It is easy to expand, maintain and perpetuate/keep going.

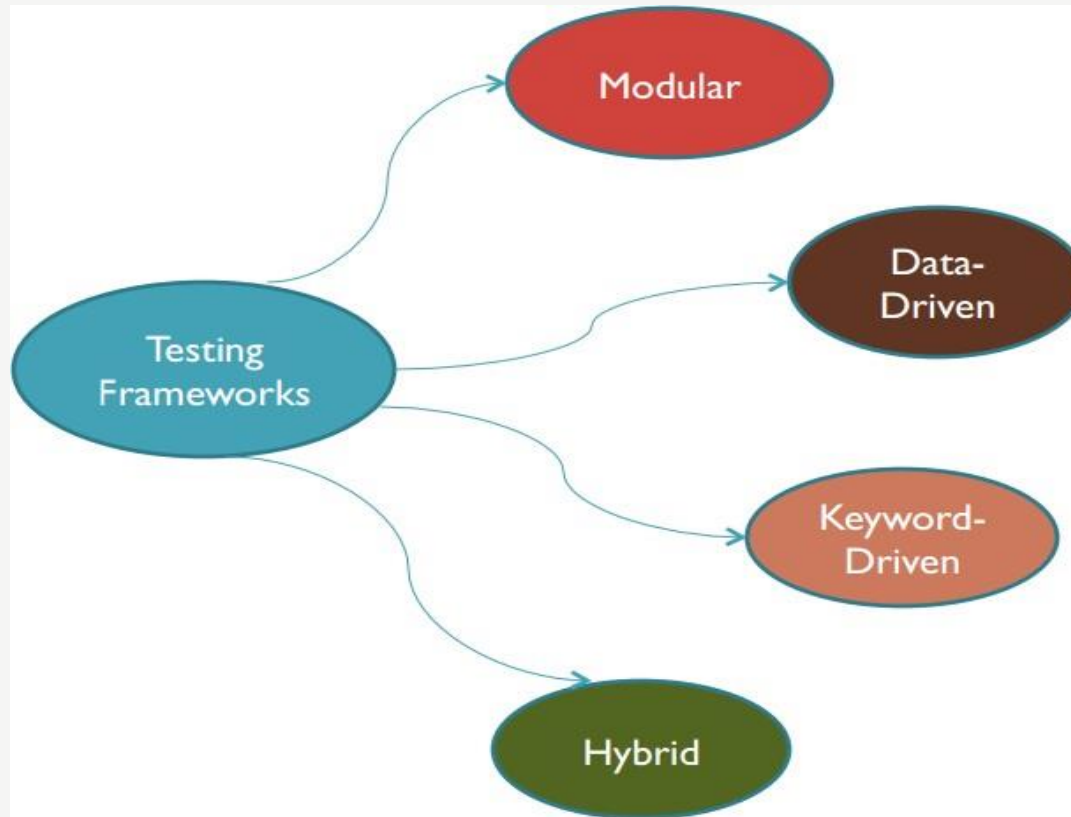
Why the testing framework is required:

- If we have a group of testers and suppose if each project implements a unique strategy then the time needed for the tester become productive in the new environment will take long.
- To handle this we cannot make changes to the automation environment for each new application that comes along.

Properties of a testing framework:

- For this purpose we use a testing framework that is application independent and has the capability to expand with the requirements of each application.
- Also an organized test framework helps in avoiding duplication of test cases automated across the application.
- In short Test frameworks helps teams organize their test suites and in turn help improve the efficiency of testing.

Types Of Testing Frameworks:



Modular Testing Framework

- The Modularity testing framework is built on the concept of abstraction.
- This involves the creation of independent scripts that represent the modules of the application under test. These modules in turn are used in a hierarchical fashion to build large test cases.
- Thus it builds an abstraction layer for a component to hide that component from the rest of the application. Thus the changes made to the other part of the application do not effect that component.

Modular Testing Framework

Example of Modular Testing Framework:

- To demonstrate the modular framework we use the calculator program.—Consider the basic functions of the calculator such as addition, subtraction, multiplication, division which are part of the Standard view.

We create scripts for these functions as follows:

Add:

Sub Main

Window Set Context, "Caption=Calculator", ""

PushButton Click, "ObjectIndex=10" 'Press 5

PushButton Click, "ObjectIndex=20" 'Press +

PushButton Click, "ObjectIndex=14" 'Press 6

PushButton Click, "ObjectIndex=21" 'Press =

Result = LabelUP (CompareProperties, "Text=11.", "UP=Object Properties")

'Compare Expected to Actual Results

End Sub

Modular Testing Framework

Advantages:

- —Modular division of scripts leads to easier maintenance and also the scalability of the automated test suites. —
- The functionality is available in easy to use test libraries so creating new driver scripts for different tests is easy and fast.

Disadvantages:

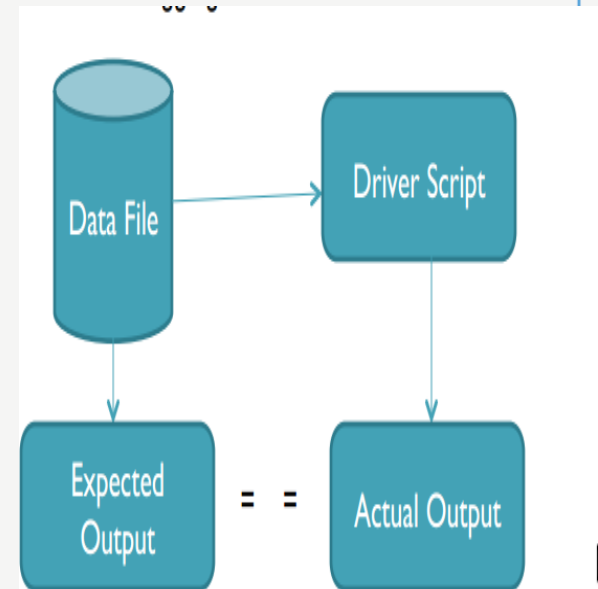
- The main problem with modular frameworks is that the test script have test data embedded in them. So when the test data needs to be updated we need to change the code of the script.
- This becomes a big problem when the test script is large. For this purpose, data- driven testing frameworks have been introduced.

Data Driven Testing

- A key benefit of automating functional testing is the ability to test large volumes of data on the system quickly.
- But you must be able to manipulate the data sets, perform calculations, and quickly create hundreds of test iterations and permutations with minimal effort.
- Test Automation Frameworks must have capability to integrate with spreadsheets and provide powerful calculation features.

Data-Driven Testing Framework

- Data driven testing is where the test input and the expected output results are stored in a separate data file (normally in a tabular format) so that a single driver script can execute all the test cases with multiple sets of data.
- The driver script contains navigation through the program, reading of the data files and logging of the test status information.



Data-Driven Testing Framework

Example of Data Driven Testing Framework

- To demonstrate the data driven testing framework we use the login page of the flight reservation.
- The first step involves creating the test data file. (testdata.csv). This data file contains the different types of input data which will be given to the driver script.

Test Case	Number1	Operator	Number2	Expected Result
Add	2	+	3	5
Subtract	3	-	2	1
Multiply	2	*	3	6
Divide	2	/	-2	-1

Data-Driven Testing Framework

In the next step we create a driver script and make references to the test data file.

```
data = open ( ' testdata.csv' ) . read ( )
```

```
l i n e s = data . s p l i t l i n e s ( )
```

#excluding the header row

for line in lines:

 Read Number1

 Read Number2

 Read Operator

 Calculate the result using the Operator on

 Number 1 and Number2

Compare the result to the expected result. Reads the data from the data file computes the value and compares it with the expected result from the data file.

Data-Driven Testing Framework

Advantages:

- This framework reduces the number of overall test scripts needed to implement all the test cases.
- Less amount of code is required to generate all the test cases.

Disadvantages:

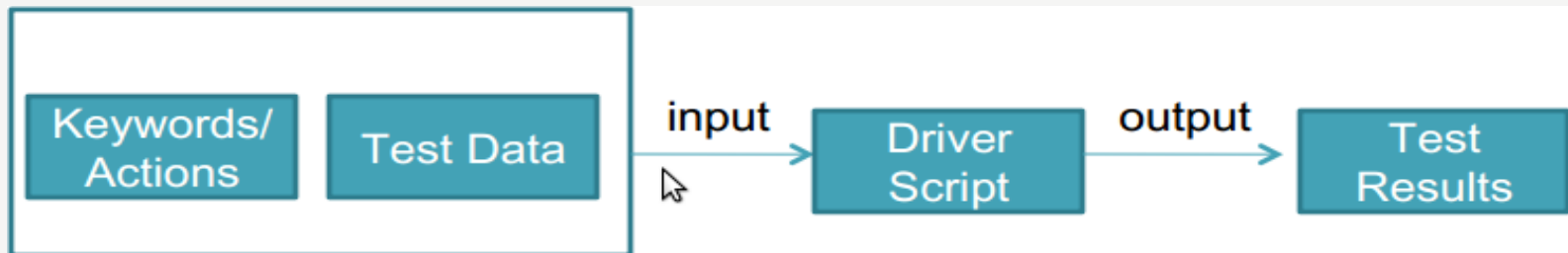
- The test cases created are similar and creating new kind of tests requires creating new driver scripts that understand different data. Thus the test data and driver scripts are strongly related that changing either requires changing the other.

• Refer this example Link-

<https://github.com/TopsCode/Automation-Testing/blob/master/Module-4/4.1%20DataDrivenExample/src/com/framework/DataDrivenDemo.java>

Keyword-Driven Testing Framework

- Keyword driven testing is an application independent framework utilizing data tables and self explanatory keywords to explain the actions to be performed on the application under test.
- Not only is the test data kept in the file but even the directives telling what to do which is in the test scripts is put in external input data file.
- These directives are called keywords. The keyword based testing is an extension to the data driven testing.



Keyword-Driven Testing Framework

Example of Keyword Driven Testing Framework:

- To demonstrate the keyword driven testing we take the actions performed by the mouse when making calculations.
- We create a table that maps the actions performed with the mouse on the window of the calculator application. In this table,
 - The windows column represents the application for which we are performing the mouse action.
 - The control column represents the control that we are clicking with the mouse.
 - The action column represents the action performed by the mouse.
 - The argument column contains the specific control value.

Keyword-Driven Testing Framework

Window	Control	Action	Arguments
Calculator	Menu		View, Standard
Calculator	Pushbutton	Click	2
Calculator	Pushbutton	Click	+
Calculator	Pushbutton	Click	3
Calculator	Pushbutton	Click	=
Calculator		Verify Result	5
Calculator		Clear	
Calculator	Pushbutton	Click	5
Calculator	Pushbutton	Click	-
Calculator	Pushbutton	Click	3
Calculator	Pushbutton	Click	=
Calculator		Verify Result	2

Keyword-Driven Testing Framework

After creating the table, we create a set of scripts for reading in the table, executing each step based on the keyword contained in the action field and logs any relevant information.

- The below pseudo code represents this test of scripts.
 - Main Script / Program
 - Connect to data tables.

Keyword-Driven Testing Framework

Read in row and parse out values.

- Pass values to appropriate functions.
- Close connection to data tables.

Advantages:

- Automation expertise is not required to maintain or create a new set of test cases.
- Keywords are reused across multiple test cases.

Disadvantages:

- The main problem is that this requires a more complicated framework than the data driven framework.
- With the keyword driven approach the test cases get longer and complex and this is due to the greater flexibility that this approach offers.

Keyword-Driven Testing Framework

- **Refer this example Link-**

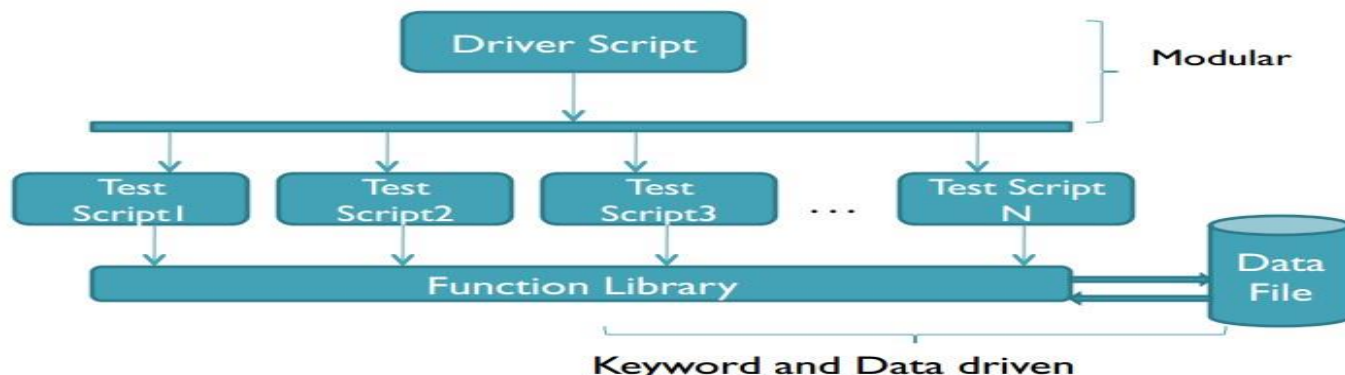
<https://github.com/TopsCode/Automation-Testing/blob/master/Module-4/4.2%20KeywordDrivenExample/test-output/index.html>

Hybrid Testing Framework

- Hybrid testing framework is the combination of modular, data-driven and keyword driven testing frameworks.
- This combination of frameworks helps the data driven scripts take advantage of the libraries which usually accompany the keyword driven testing.

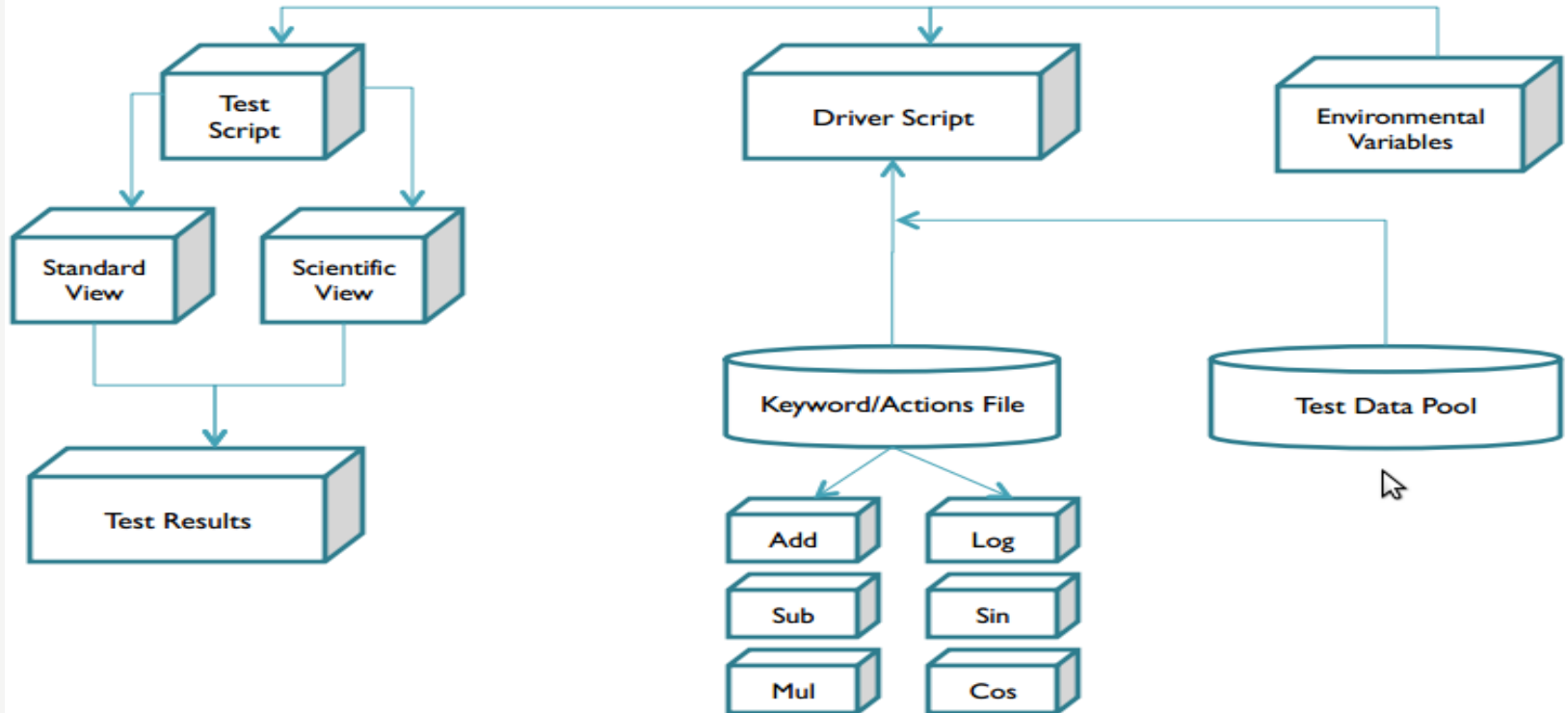
Hybrid Testing Framework

- Hybrid testing framework is the combination of modular, data-driven and keyword driven testing frameworks.
- This combination of frameworks helps the data driven scripts take advantage of the libraries which usually accompany the keyword driven testing.



Hybrid Testing Framework

The hybrid framework for the calculator can be shown as follows:



Comparison of Frameworks

Approach	Advantages	Disadvantages
Modular testing framework	Modular approach Reusable functions Hierarchical Structure	Test data within the scripts limits reusability, Test script is dependent on software.
Data driven testing framework	Improved Maintainability	Dependency on technical expertise, Test script is dependent on software.
Keyword driven testing framework	Ease of maintenance, Scalability, Less dependency of software.	Dependency on technical expertise, Requires large effort
Hybrid testing framework	Integrates the advantages of all the other frameworks.	Increased Complexity

Module – 5[Maven Project Management Tools]

- Introduction to Maven
- Insulation of Maven
- Maven Structure
- Maven Dependency
- Maven Repository
- Maven Eclipse Integration



TOPSTECHNOLOGIES

Training | Outsourcing | Placement | Study Abroad

Maven Project Management Tool

Maven Introduction

- Maven is an automation and management tool developed by Apache Software Foundation.
- It was initially released on 13 July 2004.
- In Yiddish language the meaning of Maven is "accumulator of knowledge".
- It is written in Java Language and used to build and manage projects written in C#, Ruby, Scala, and other languages.
- It allows the developer to create projects using Project Object Model and plugins.
- It helps to build projects, dependency, and documentation.
- Its development process is very similar to ANT. However, it is much advanced than ANT.
- Maven is also able to build any number of projects into desired output such as jar, war, metadata.

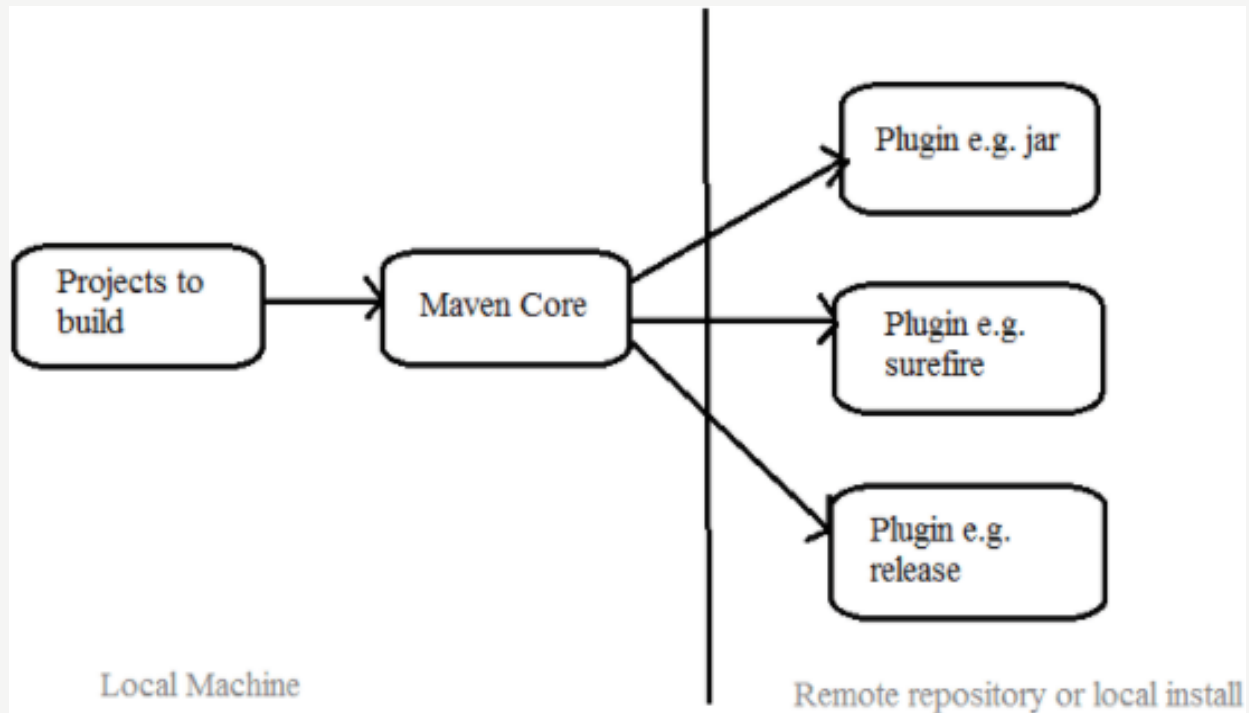
How can Maven benefit my development process?

- Maven helps the developer to create a java-based project more easily.
- Accessibility of new feature created or added in Maven can be easily added to a project in Maven configuration.
- It increases the performance of project and building process.
- The main feature of Maven is that it can download the project dependency libraries automatically.
- Below are the examples of some popular IDEs supporting development with Maven:
 - Eclipse
 - IntelliJ IDEA
 - JBuilder
 - NetBeans
 - MyEclipse

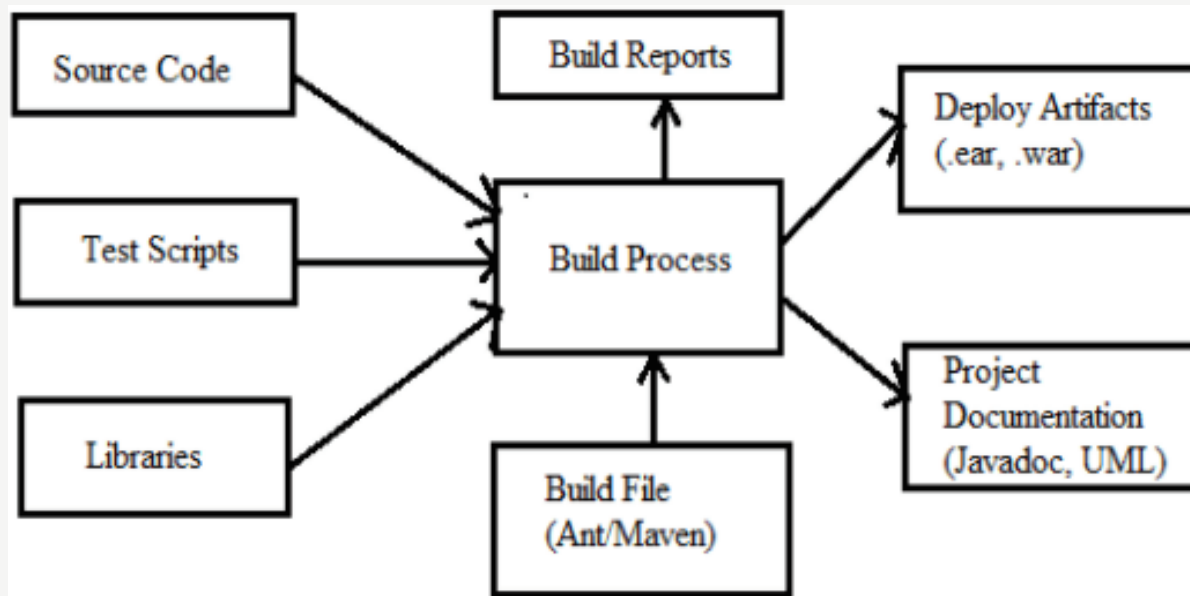
Processes which can manage using maven

- Builds
- Documentation
- Reporting
- Dependencies
- SCMs
- Releases
- Distribution
- mailing list

Maven Architecture



Maven Architecture



ANT v/s Maven

Ant	Maven
Ant doesn't has formal conventions, so we need to provide information of the project structure in build.xml file.	Maven has a convention to place source code, compiled code etc. So we don't need to provide information about the project structure in pom.xml file.
Ant is procedural , you need to provide information about what to do and when to do through code. You need to provide order.	Maven is declarative , everything you define in the pom.xml file.
It is less preferred than Maven.	It is more preferred than Ant.

ANT v/s Maven

Ant	Maven
There is no life cycle in Ant.	There is life cycle in Maven.
It is a tool box.	It is a framework .
It is mainly a build tool .	It is mainly a project management tool .
The ant scripts are not reusable .	The maven plugins are reusable .

Installation of Maven

You can download and install maven on windows, linux and MAC OS platforms.

Here, we are going to learn how to install maven on windows OS.

To install maven on windows, you need to perform following steps:

1. Download maven and extract it
2. Add JAVA_HOME and MAVEN_HOME in environment variable
3. Add maven path in environment variable
4. Verify Maven

Maven - POM

- POM stands for Project Object Model.
- It is fundamental unit of work in Maven.
- It is an XML file that resides in the base directory of the project as pom.xml.
- The POM contains information about the project and various configuration detail used by Maven to build the project(s).
- POM also contains the goals and plugins.
- While executing a task or goal, Maven looks for the POM in the current directory. It reads the POM, gets the needed configuration information, and then executes the goal.

Maven - POM

- Some of the configuration that can be specified in the POM are following –
 - project dependencies
 - plugins
 - goals
 - build profiles
 - project version
 - developers
 - mailing list
- Before creating a POM, we should first decide the project **group** (groupId), its **name** (artifactId) and its version as these attributes help in uniquely identifying the project in repository.

Maven – POM Example

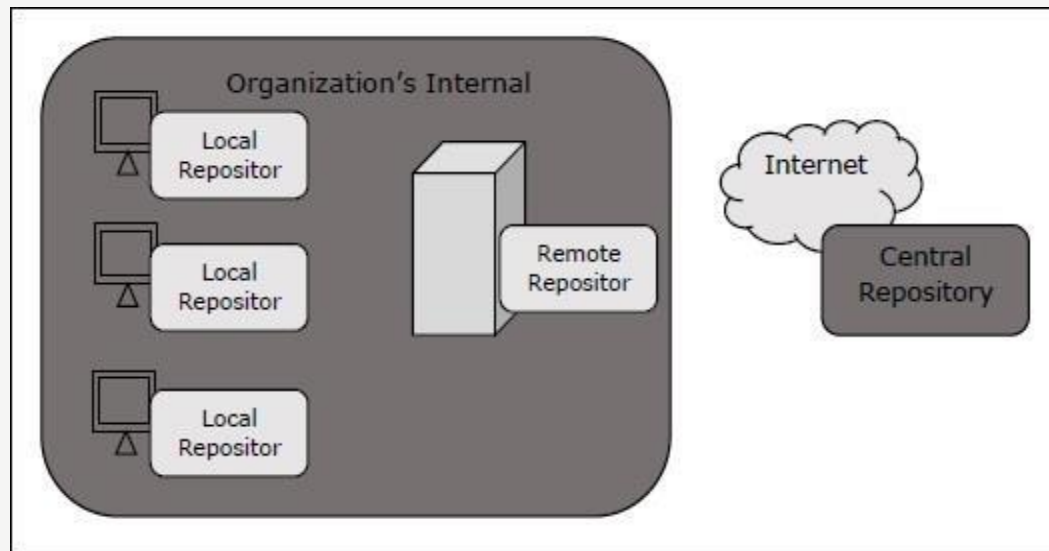
Sr.No.	Node & Description
1	Project root This is project root tag. You need to specify the basic schema settings such as apache schema and w3.org specification.
2	Model version Model version should be 4.0.0.
3	groupId This is an Id of project's group. This is generally unique amongst an organization or a project. For example, a banking group com.company.bank has all bank related projects.

Maven – POM Example

Sr.No.	Node & Description
4	artifactId This is an Id of the project. This is generally name of the project. For example, consumer-banking. Along with the groupId, the artifactId defines the artifact's location within the repository.
5	version This is the version of the project. Along with the groupId, It is used within an artifact's repository to separate versions from each other. For example – com.company.bank:consumer-banking:1.0 com.company.bank:consumer-banking:1.1.

Maven Repositories

- Repository is a directory where all the project jars, library jar, plugins or any other project specific artifacts are stored and can be used by Maven easily.
- Maven repository are of three types.-
 - local
 - central
 - remote



How to use Maven

- To configure the Maven, you need to use Project Object Model, which is stored in a pom.xml-file.
- POM includes all the configuration setting related to Maven.
- Plugins can be configured and edit in the <plugins> tag of a pom.xml file. And developer can use any plugin without much detail of each plugin.
- When user start working on Maven, it provides default setting of configuration, so the user does not need to add every configuration in pom.xml

Maven Dependencies

```
<!-- https://mvnrepository.com/artifact/javax.servlet/jsp-api -->
<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>jsp-api</artifactId>
  <version>2.0</version>
  <scope>provided</scope>
</dependency>
</dependencies>
```

Maven Dependencies

```
<build>
  <finalName>MavenWebApp</finalName>
  <plugins>
    <plugin>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.5.1</version>
      <configuration>
        <source>1.8</source>
        <target>1.8</target>
      </configuration>
    </plugin>
  </plugins>
</build>
</project>
```

Steps/process involved in building the project

- Add / Write the code for application creation and process that into source code repository
- Edit configuration / pom.XML / plugin details
- Build the application
- Save the build process output as WAR or EAR file to a local location or server
- Get the file from local location or server and deploy the file to the production site or client site
- Updated the application document with date and updated version number of the application
- create and generate a report as per the application or requirement.
- **Refer this example Link-**
<https://github.com/TopsCode/Automation-Testing/blob/master/Module-5/5.1%MavenExample/src/test/java/com/mavendemo/MavenClassDemo.java>



TOPSTECHNOLOGIES

Training | Outsourcing | Placement | Study Abroad

Thank You